Table of Contents

Title Page	1
Student Declaration	2
Table of Contents	3
Exploratory Report	4
Design Document Proposal	21
Application Development Process and Implementation	39
Testing Documentation	74
Evaluation Report	92
Application Video Demonstration	97
Appendix	98
References	102

Exploratory Report

North Sussex Judo Requirements

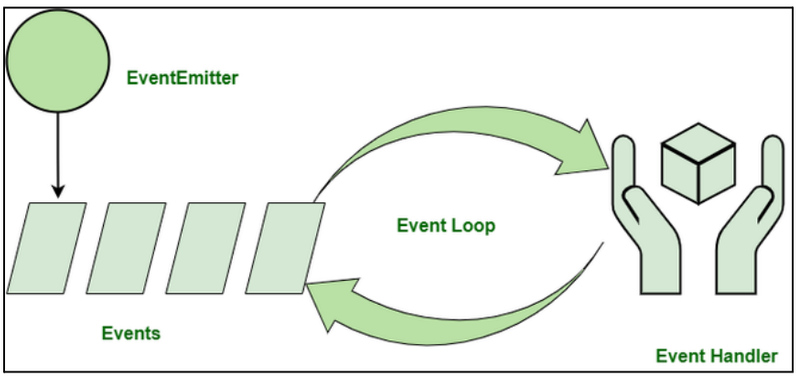
In my role as a Junior Software Developer in AQDS, I've been tasked to create a program for North Sussex Judo that will record the fees or transactions made by the athletes for each of the months. Such as the total fees of the training plan, private coaching, and competitions. In my program, the database system records all of the athletes' information while my application is designed to register athletes, manipulate, and retrieve data (CRUD). The athlete details include their name, weight and their respective weight category, training plan, private coaching, and competitions wherein the CEO gave me the freedom to design and build the application as long as there is user authentication.

On top of that, North Sussex Judo has given additional information about the training plan and weight categories with their respective prices. The given conditions are considered in the program, such as athletes can only have a maximum of 5 hours of private coaching, intermediate and elite athletes are the only ones allowed to enter the competition, and more.

Overview of Application Functionalities

The selected application I created for North Sussex Judo is an athlete management system that aims to handle administrative needs such as related to athletes' relevant information and all financial transactions. It is an application designed for administrators for NSJ, encompassing athlete registration, tracking monthly expenses, scheduling competitions, and updating athletes' activities they enrolled in. The series of interfaces on this program performs different CRUD operations effectively. Moreover, the system prioritizes data integrity and security through the implementation of data validation, authentication, and accuracy over the financial transactions. To conclude, I provided the North Sussex Judo administrators user-friendly and straightforward program to manage several aspects of the company productively. The functionalities of the program will be thoroughly discussed in the user interface section of the paper.

The Event-Driven Programming Paradigm



It is the synchronization of events that allows the software to be responsive when such an event is triggered. For example, an event where a button is clicked, wherein associated functions or methods will be executed in response to this event. Not only button clicks, but selecting an item, mouse movements (like hovering), and other user interactions (studysmarter, 2023).

This paradigm can handle infinite values or events over time without the program being blocked on execution.

According to Geeks for Geeks (2021), the following are the characteristics of an event-driven programming paradigm:

a. **Events**

   An Event plays a crucial role in this paradigm since it is the action recognized by the software as a result of a user or other source. It's the things that happen in the system that the programmer is working on and the ones that are processed and handled by the event handler.

b. **Event Emitter**

   An Event emitter is the one that signals or notifies the program that this event is triggered or activated which means this corresponding function must be executed. The program will react and respond based on the signals from the event emitter.

a. **Event loop**

   An Event loop manages and handles the flow of multiple events smoothly, binding them to their respective functions. This makes the program responsive and viable for multi-tasking, allowing it to run multiple events without waiting for one to finish. An event loop will be there to exist as it waits for user interaction or various events to occur on parts of the program.

b. **Event handler**

   An Event handler is a concept wherein when a block of code or function that is created by the programmer, will be executed automatically in response to a certain event being activated. In short, a corresponding action is called or

when an event is triggered. This allows the system to adapt dynamically to inputs without the programmer or user manually intervening.

Event-Driven Programming Paradigm in North Sussex Judo Application

My source code has effectively applied the event-driven paradigm through buttons, event handlers, and event bindings, making my program interactive and responsive to such events. Each user action triggers a particular event, and my program interacts with it according to its function. This makes my program a smoothly dynamic and well-behaved GUI application. Here are the following examples of how the paradigm works on my source code:

__Home_Window.py

```
 # Create Login button
        btn_login = tk.Button(self.home_win, text="Login", font=("Arial", 14, "bold"),
                              command=self.open_login_window)
        btn_login.grid(row=1, column=0, columnspan=2, pady=pad)

# Create a Create Account button
        btn_create = tk.Button(self.home_win, text="Create Account", font=("Arial", 14, "bold"),
                               command=self.open_user_registration)
        btn_create.grid(row=2, column=0, columnspan=2, pady=pad)
```

On my starting window, I created two buttons, the Login and Create Account buttons. These buttons have something in common which is the parameter, ‘command’, wherein this is bound to an event handler function every time the button is clicked by the user.

```
    def open_login_window(self):
        """Destroys the home window then opens the login window"""
        self.home_win.destroy()
        from Login_Window import open_login_window
        open_login_window()

    def open_user_registration(self):
        """Open the user registration window"""
        from User_Registration import open_user_registration
        open_user_registration()
```

The ‘self.open_login_window’ and ‘self.open_user_registration’ are the event handler functions bound to these buttons respectively. When the user clicks the button, it triggers specific events such as opening a login window or a user registration

window.

```
def open_home_window():
    """Open the home window by creating an instance of HomeWindow class."""
    home_win = tk.Tk()
    HomeWindow(home_win)
    home_win.mainloop()


if __name__ == "__main__":
    """Initialize the home window as the main entry point of the program"""
    open_home_window()
```

Furthermore, the main event loop is initialized by calling the 'home_win.mainloop'. The functionality of this loop is it co-exists with the program, waiting for user interactions like clicking the button. Then, if such an event occurs, the corresponding event handlers will be executed.

This is how the event-driven paradigm works on my source code, not only on the home window but all of my windows in the program are designed similarly to this. I have numerous event handlers, events, and event binding to handle user actions and execute the required functionalities of my application, including adding, editing, removing, athletes, and more.

Login_Window.py

```
        self.login_win.bind('<Return>', self.login_process)
```

It can also be seen in my login window that I included the Enter key or Return key as my keyboard bind for pressing the login button. This ensures that when the user presses the enter key, the login process will be automatically executed. This is an example of a keyboard event.

Edit_Athlete.py

```
        def tv_MouseRelease(self, event):
        """Handle selection in Treeview."""
        # Get the selected item
        selected_item = self.tv.focus()
        # Get values of the selected item
        item_values = self.tv.item(selected_item, 'values')

        # Populate Entry widgets with selected row's data
        self.entry_first_name.delete(0, END)
        self.entry_last_name.delete(0, END)
        self.entry_weight.delete(0, END)

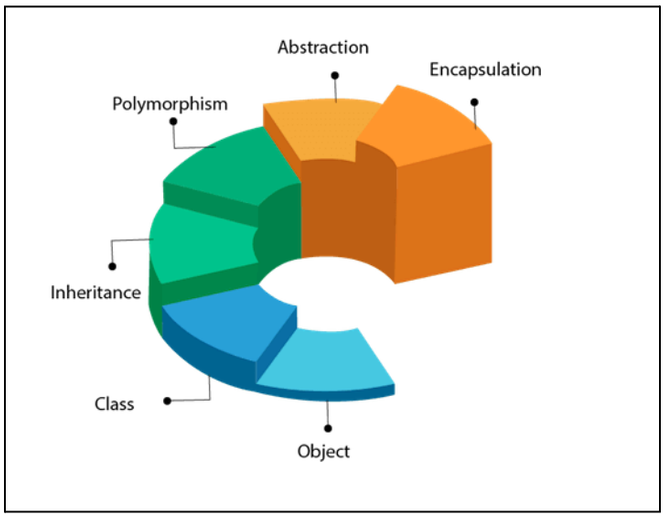
        if item_values:
            # Insert values into respective Entry widgets
            self.athlete_id.config(text=item_values[0])
            self.entry_first_name.insert(0, item_values[1])
            self.entry_last_name.insert(0, item_values[2])
            self.entry_weight.insert(0, item_values[3])
            self.weightclass_id.config(text=item_values[4])

self.tv.bind("<ButtonRelease-1>", self.tv_MouseRelease)
```

In this code snippet of my edit athlete window, the method ‘tv_MouseRelease’ is an event handler when the user releases the left mouse button after they click a row in the Treeview or ‘self.tv’, thus bound to ‘<ButtonRelease-1>’. The functionality of the method is to retrieve the data of the selected athlete and populate the entry widgets. This type of event can also be seen in the utility costs window and main window.

These are the examples of event paradigms that were applied in my program. The most common event and event handlers in my program are the button clicking and selecting in the treeview. I’ve strategically positioned numerous buttons in my main window in order to separate the widgets and distinct functionalities of the program. This approach makes it easier for me to code since it sets apart the errors to specific classes or operations, given that my module only has one class for each. Rather than compiling classes into one module, making it difficult to debug. This is why I divided my program into many windows, making it a more intuitive and user-friendly interface.

Object Oriented Programming Paradigm



It is a programming paradigm that centers and utilizes the concepts of objects and classes in programming wherein the objects are the instances of the classes and classes are the template for creating these objects. The primary idea in this paradigm is it models real-world objects that represent the interactions and relationships between these entities (Doherty, 2020). Each of these objects comprises attributes(data) and methods (functions) that encapsulate and work together as a single class, making the structure of code manageable and organised. In short, object-oriented programming is used to model real-world situations.

According to Geeks for Geeks (2016), these are the characteristics of the object-oriented programming paradigm:

a. **Class**

The class defines and encapsulates the general behavior and attributes that a whole category of objects can have. Class instances comprise methods for modification and interaction with their attributes. A class is created by the keyword “class” and a class has its attributes wherein these attributes are public and can be accessed using the dot(.) operator which also allows it to be modified.

b. **Objects**

An object is the core concept of OOP wherein it is the instance of a class, from the definition provided by the class. Upon instantiation of an object, memory is allotted to store its data and methods. Each instantiation of an object from a class would have its unique state and methods to perform specific operations, with the data and behaviors encapsulation within.

c. Data Abstraction

Data abstraction conceals complex implementation details while revealing the relevant ones. It is facilitated through abstract class, wherein it is the blueprint for subsequent child classes. It enables the programmer to outline a set of methods that must be created within any child class derived from the

abstract class. Abstract class contains one or more abstract methods, which are declared but not implemented.

d. **Encapsulation**

   Encapsulation is defined by the word itself “encapsulating”, wherein it encapsulates the data and method to work together within one unit. It imposes restrictions on direct access to variables and methods to avoid accidental modification. For example, a class has encapsulated data of functions and variables.

e. **Inheritance**

   Inheritance allows the programmer to not always start from scratch when creating a class, in a way that it writes another version of that parent class. The parent class is the original class and the inherited class is the child class. The child class takes on the attributes and methods from the parent class, but it also has the freedom to define new attributes and methods for the new class. The super() function allows one to call a method from the parent class and initialize attributes.

f. **Polymorphism**

   Polymorphism is also defined by the word itself “polymorph” which can have many forms. In programming definition, a single entity can behave in multiple ways and have different characteristics depending on the context. Operator, Function, Class, and Method Polymorphism are some of the kinds of polymorphism. For example in operator polymorphism, operators behave differently based on the data type they are applied to. Using a “+” operator on integers will have a different result as opposed to when it's used with strings.

Object-Oriented Programming Paradigm in North Sussex Judo Application

As I mentioned before, each module contains one, which maintains readability and easier to locate functionalities, especially when I make changes or debug. This is how modularity and organisation apply to my source code. It would not be a problem to add or remove aspects of any of my modules since it is separated and would not affect my whole codebase.

Importing has been easier because modular classes are reusable, wherein I can import whenever I only need to on a specific module. Each of my classes is encapsulated within a module that exposes only the necessary interfaces. My program ensures that it is scalable and flexible when new functionalities are added. The object-oriented paradigm applies to my source code, the following are some examples:

Athlete_Registration_Form.py

```
class AthleteRegistration:
    def __init__(self, reg_win):
        """Initialize the Athlete Registration Form"""
        self.reg_win = reg_win
        self.reg_win.title("Athlete Registration Form")
        self.reg_win.geometry("380x350")
```

It is shown here in my Athlete Registration Form window that the class 'AthleteRegistration' encapsulates related instance variables and methods within a single unit. Instances variables such as creating the widgets, entry fields, and the window itself. In short, the 'AthleteRegistration' class encapsulates the functionalities related to registering athletes, wherein it is supported by the constructor method '__init__' . This constructor method initializes the class attributes and the 'reg_win', and without this, the class won't operate.

```
self.reg_win
self.label_reg_date
self.entry_athlete_id
self.entry_first_name
self.entry_last_name
self.entry_birthdate
self.entry_weight
self.training_plan_var
self.training_plan_menu
```

These are instances variables which is an important component of OOP. Each of these variables corresponds to a particular instance inside the class, wherein it holds data to certain GUI elements of the program such as widgets. The 'self' is a name that is used to illustrate the instance of a class.

```
    def register_process(self):
        """Handle the registration process when the 'Register' button is clicked"""
        # Retrieving input values from the form
        reg_date = datetime.now().strftime('%Y-%m-%d')
        athlete_id = self.entry_athlete_id.get()
        birthdate = datetime.strptime(self.entry_birthdate.get(),
                                      '%d/%m/%Y').strftime('%Y-%m-%d')
        first_name = self.entry_first_name.get()
        last_name = self.entry_last_name.get()
        training_plan = self.training_plan_var.get()

        # Processing selected weight value
        selected_weight = self.entry_weight.get()
```

The process of registering athletes is encapsulated in a method. This method 'register_process' is the main functionality of the program and performs tasks based on user inputs such as executing data validation, storing data in the database, and displaying message boxes.

The objects created in this class are the label, entry, button widgets, and 'reg_win' itself which are instantiated. All of my modules have a class similar to the structure of this source code. Inside the classes, it contains the creation of widgets, while the

methods do the functionalities such as SQL commands, logical functions, and validations.

Data_Access_Layer.py

```
class DataAccessLayer(object):
    # Connection string to connect to the database
    cs = """
        Driver={ODBC Driver 17 for SQL Server};
        Server=LAPTOP-LLLCJJDT\\SQLEXPRESS;
        Database=North Sussex Judo;
        Trusted_Connection=yes;
    """

    def __init__(self):
        """Initialize DataAccessLayer instance."""
        self.cn = None  # Database connection
        self.cursor = None  # Cursor to execute SQL queries
```

The 'DataAccessLayer' class encapsulates the functionalities in relation to connecting the Python program to the SQL Database NSJ that I created. Similar to the previous example, it has a '__init__' to initialize the instance variables. This is part of the preparation for the initialization of components and storing cursor objects.

```
def open(self):
def execute(self, sql, *param):
def commit(self):
def rollback(self):
def execute_query(self, sql, *param):
def close(self):
def fetch_one(self, query, *args):
```

These are the methods encapsulated within a class wherein each has distinctive functionalities related to database operations. These methods are invoked from other modules whenever I create certain functionalities of each module. These methods also apply the concept of abstraction since they hide details relating to the interaction between the database and the program, by simply just calling the method on other modules.

These are the examples of object-oriented programming paradigms applied in my program. It is evident that OOP is applied in all of my modules and almost has the same structure for each, ensuring that is maintainable and organised.

Procedural Programming Paradigm

As the name suggests, “procedural”, the codes of the program are divided into smaller, manageable parts which are organised into a set of procedures or routines or commonly known as functions or methods (grouped by functionality). Each of these functions performs a specific task depending on context or logic (StudySmarter UK, n.d.). The code is structured in a linear, and step-by-step for the computer to immediately understand how to perform each task. This paradigm is excellent in terms of algorithms since the programmer can easily track the flow of the program, making it also easy to implement compilers and interpreters.

According to Sagar Bhatia (2022), the following key features of this paradigm include:

a. **Modularity**

   Modularity is the process of separating the program into independent components that perform different tasks. This could be separating the systems into modules or functions in order to segregate and create complex systems, while still maintaining coordination with the whole codebase. Despite the distinctive functionality of the systems, they are grouped together to achieve a common goal.

b. **Local Variable**

   Local variable is when the variable is declared under a local scope inside a method. This makes the variable limited to use. For example, when a local variable is defined under a certain method, it will limitedly work inside that method. But, when it is defined outside that certain method, it won’t work.

c. **Global Variable**

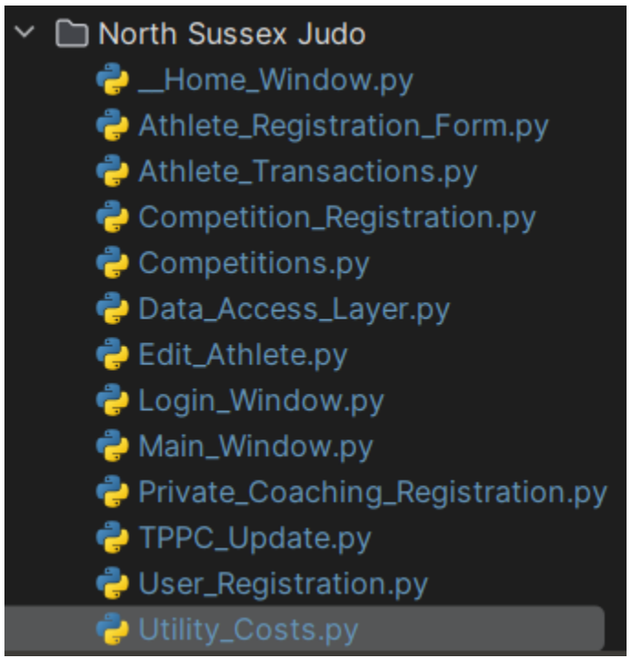
   Global Variable is when a variable is declared outside a function or method on the code base. This is the opposite of a local variable, wherein any function can use the variable within the codebase. However, bugs may be harder to debug and track since one or more variables are used globally which makes it challenging

d. Parameter Passing

   Parameter Passing is a mechanism that passes parameters to functions or procedures. This is a variable that is inside a parenthesis of the function definition. An argument, in Python, is the value that corresponds to the parameters of a certain function.

Procedural Programming Paradigm in North Sussex Judo Application

In my program, procedural programming is evident in the way how I structure my code into a top-down approach. The program is divided by procedures which are also the functions, and this contains the series of instructions to be carried out. I'll show how the characteristics of this paradigm apply to my source code.



My North Sussex Judo project is compiled into different modules, wherein each module has its unique functionality that contributes to the fulfillment of requirements for my program. Thus, showcasing the use of modularity. Then, each of my modules contains a class and these classes encapsulate methods that perform specific tasks. The functions will be executed sequentially based on the user interaction.

Athlete_Registration_Form.py

```
    def register_process(self):
        """Handle the registration process when the 'Register' button is clicked"""
        # Retrieving input values from the form
        reg_date = datetime.now().strftime('%Y-%m-%d')
        athlete_id = self.entry_athlete_id.get()
        birthdate = datetime.strptime(self.entry_birthdate.get(),
                                      '%d/%m/%Y').strftime('%Y-%m-%d')
        first_name = self.entry_first_name.get()
        last_name = self.entry_last_name.get()
        training_plan = self.training_plan_var.get()

        selected_weight = self.entry_weight.get()
```

Local variables are the most widely used variables in my source code since they are a key component for the logic of the program, SQL queries, initialization of the window, and widgets, within that local scope. For example, in my athlete registration form, these variables are declared under the 'register_process' method only, making it a local variable. This prevents unintended modification from other modules or sections of the program. These variables are associated with values which are the data of the athletes.

Data_Access_Layer.py

```
    def execute_query(self, sql, *param):
        """Execute a SQL Query and return as a list of tuples."""
        self.cursor.execute(sql, *param)
        db_set = self.cursor.fetchall()
        return db_set
```

In my data access layer module, the 'execute_query' function is widely used in all of my modules to execute SQL queries. This demonstrates parameter passing since the '*param' inside the function definition, accepts an arbitrary number of parameters to fulfill the requirement of parameters of the SQL queries, and then passes to the 'execute' method.

In this example, I'll also show how my code is structured in a step-by-step manner.

Private_Coaching_Registration.py

```
from datetime import datetime
from tkinter import *
from Data_Access_Layer import *
from tkinter import messagebox
```

First, I imported the Data Access Layer module, Tkinter library, and necessary Tkinter widgets I'll be using for the program. This makes sure that all of the resources needed to import are readily available for use and executing various codes.

```
def open_private_coaching_registration():
    """Function to create and open the Private Coaching Registration window."""
    pcr_win = Tk()
    PrivateCoachingRegistration(pcr_win)
    pcr_win.mainloop()


class PrivateCoachingRegistration:
    def __init__(self, pcr_win):
        """Initialize the PrivateCoachingRegistration class"""
        self.pcr_win = pcr_win
        self.pcr_win.title("Register Private Coaching")
        self.pcr_win.geometry("400x200")
        btn_create.grid(row=2, column=0, columnspan=2, pady=pad)
```

Second, initializing and modifying the window is an important part of the process since it is where the widgets will be placed as well as its respective functions.

```
        # Reg. Date Widgets
        lbl_reg_date = Label(self.pcr_win, text="Registration Date: ")
        lbl_reg_date.grid(row=2, column=0, padx=10, sticky="w")
        current_date = datetime.now().strftime('%d/%m/%Y')
        self.label_reg_date = Label(self.pcr_win, text=current_date)
        self.label_reg_date.grid(row=2, column=1, padx=10, sticky="w")

        # Private Coach. Date Widgets
        lbl_private_coaching_date = Label(self.pcr_win, text="Date of Private Coaching
                                          (dd/mm/yyyy): ")
        lbl_private_coaching_date.grid(row=3, column=0, padx=10, sticky="w")
        self.entry_private_coaching_date = Entry(self.pcr_win)
        self.entry_private_coaching_date.grid(row=3, column=1, padx=10, sticky="w")

        # Athlete ID Widgets
        lbl_athlete_id = Label(self.pcr_win, text="Athlete:")
        lbl_athlete_id.grid(row=4, column=0, padx=10, sticky="w")
```

Third, the GUI widgets are initialized and the widgets are in order of how they are positioned in the window accordingly. For example, the registration date comes first, then the date of private coaching, and so on.

```
        def get_athletes(self):
        """Fetch and return the list of athletes from the database."""
        dal = DataAccessLayer()
        dal.open()

        # Fetch AthleteID, FirstName, and LastName from Athletes table
        athletes = dal.execute_query("SELECT AthleteID, FirstName, LastName FROM
                                     Athletes")

        dal.close()

        # Convert the list of tuples to a list of dictionaries
        athlete_dicts = [{'AthleteID': athlete[0], 'FirstName': athlete[1], 'LastName':
                        athlete[2]} for athlete in athletes]

        return athlete_dicts


        def register_private_coaching(self):
        """Register private coaching."""
        selected_athlete_name = self.athlete_var.get()
        athlete_id = self.get_athlete_id_from_name(selected_athlete_name)
        coaching_hrs_str = self.private_coaching_var.get()

        # Remove the "hrs" string and get only the numeric
        coaching_hrs_str = coaching_hrs_str.split()[0]
```

Fourth, the program will run this ‘get_athletes’ method wherein it fetches all the athletes from the database to place it in the dropdown button so that the user can choose which athlete they will register for private coaching. After executing this, the register process will proceed and will validate the inputted data by the user. This registration process includes the execution of SQL commands and message box exception or error handling.

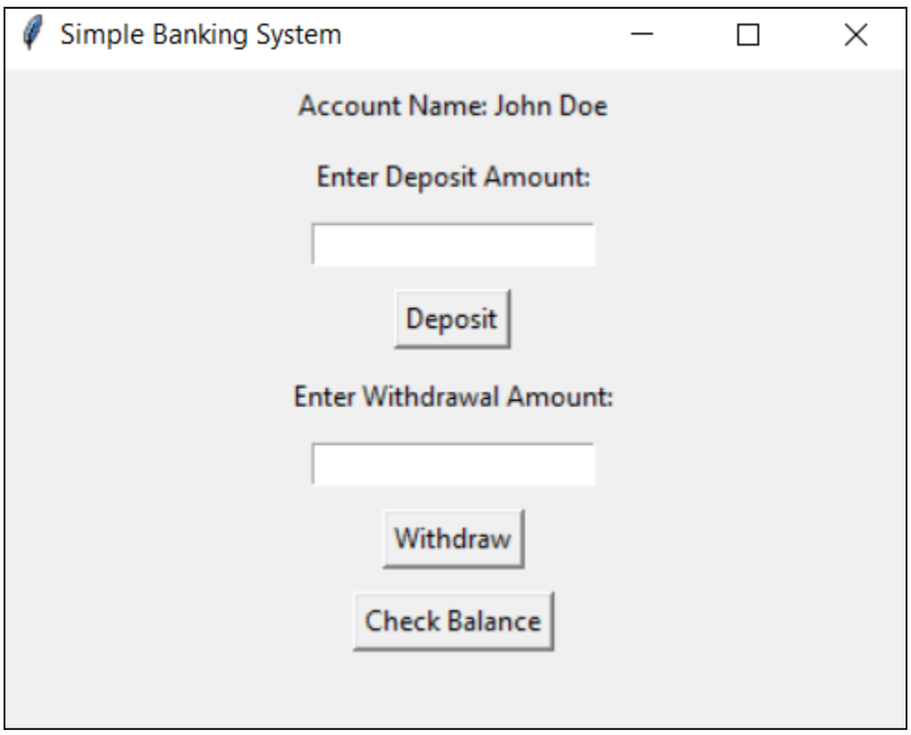
```
def get_athlete_id_from_name(self, athlete_name):
"""Fetch and return the AthleteID corresponding to the selected athlete's name."""
dal = DataAccessLayer()
dal.open()
query = "SELECT AthleteID FROM Athletes WHERE CONCAT(FirstName, ' ', LastName) =
        ?"
result = dal.execute_query(query, athlete_name)
dal.close()

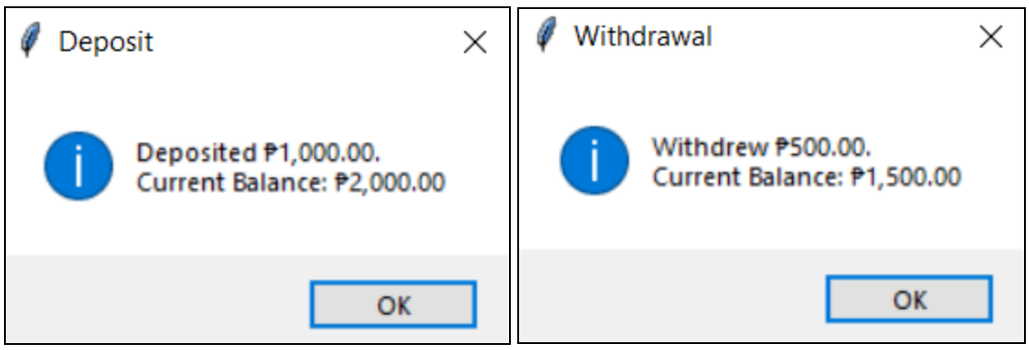
if result:
    return result[0][0]
else:
    return None
```

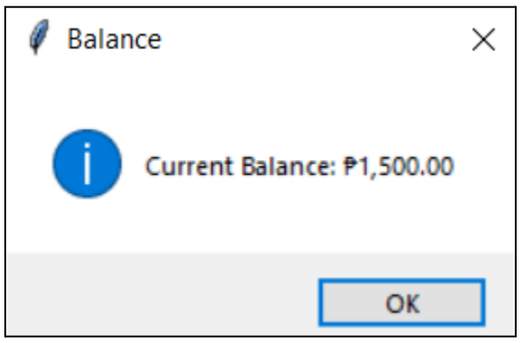
Lastly, this method will be executed once the registration process is successful. This method fetches the athlete ID that corresponds to the name that was selected.

This is an example of a source code that follows a procedural programming paradigm. Most of my modules follow this kind of structure wherein the placement of widgets and functions are executed linearly. Since my application adheres to the OOP principles, it is intuitive to structure the code in this way. The characteristics of procedural paradigms such as modularity, local variables, and parameter passing are observed in my source code.

Sample Program: Simple Banking System







I created a sample program namely, Simple Banking System, wherein I integrated the three programming paradigms. This program can deposit, withdraw, and check the current balance that the user has.

Starting with the event-driven programming paradigm. Whenever the user clicks the deposit button in the window, it triggers the deposit methods inside the classes. This demonstrates the relationship of the event and event handler in the source code, since a user interaction triggers an event, following the execution of the corresponding method.

```
class BankAccount:
    def __init__(self, account_name, initial_balance=0):
        self.account_name = account_name
        self.balance = initial_balance

    def deposit(self, amount):
        self.balance += amount
        messagebox.showinfo("Deposit", f"Deposited {locale.currency(amount,
grouping=True)}.\nCurrent Balance: {locale.currency(self.balance, grouping=True)}")


class BankingApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Simple Banking System")
        self.geometry("400x300")

        self.btn_deposit = tk.Button(self, text="Deposit", command=self.deposit)
        self.btn_deposit.pack(pady=5)

    def deposit(self):
        amount = float(self.entry_deposit.get())
        self.bank_account.deposit(amount)
```

This process is similar to the withdraw, and check balance buttons and methods. Meanwhile, the object-oriented programming paradigm is integrated by the use of classes, objects, encapsulation, abstractions, polymorphism, and inheritance in the

source code. There are two classes in this code, and both encapsulate methods related to the banking operations of the program.

```
class BankAccount:
    def __init__(self, account_name, initial_balance=0):
        self.account_name = account_name
        self.balance = initial_balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            messagebox.showinfo("Withdrawal", f"Withdrew {locale.currency(amount,
            grouping=True)}.\nCurrent Balance: {locale.currency(self.balance,
            grouping=True)}")
        else:
            messagebox.showerror("Insufficient Funds", "You do not have sufficient funds
                                 to withdraw.")
    def check_balance(self):
        messagebox.showinfo("Balance", f"Current Balance: {locale.currency(self.balance,
                            grouping=True)}")

class BankingApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Simple Banking System")
        self.geometry("400x300")

    # Deposit, withdraw, and check balance buttons and entries here
    # .....

    def deposit(self):
        amount = float(self.entry_deposit.get())
        self.bank_account.deposit(amount)

    def withdraw(self):
        amount = float(self.entry_withdraw.get())
        self.bank_account.withdraw(amount)

    def check_balance(self):
        self.bank_account.check_balance()
```

It also integrates abstraction since the BankAccount class does the internal workings of the program, while the BankingApp class interacts with the parent class by simply executing the banking operations. This provides the abstraction layer. Polymorphism is applied by the use of the same methods in two different classes but they function differently. Similarly, inheritance can be seen by the use of super().__init__() method wherein the classBankingApp inherits methods from the classBankAccount.

```
        self.bank_account = BankAccount("John Doe", 1000)
```

The use of objects is evident in the code, wherein I created a bank account name for the user, John Doe, with an initial balance of 1000.

Lastly, the procedural paradigm is evident in my code since it imports the necessary

libraries. The initialization of the window such as the widgets, buttons, and logic followed, illustrating a sequential flow of the program execution.

```
import tkinter as tk
from tkinter import messagebox
```

My source code ends with the main event loop, 'app.mainloop()', which is responsible for the opening of the program and is responsive to user interactions.

```
if __name__ == "__main__":
    app = BankingApp()
    app.mainloop()
```

Conclusion and Relationships of the Three Programming Paradigms

The three programming paradigms can be used in software development or any programming project. Programming paradigms may differ from the programming languages programmers use, however, they help all programmers identify the most ideal approach to solve a particular problem. Event-driven highlights the response to user interactions or events, OOP models real-world entities, while Procedural divides a big program into procedures to execute tasks sequentially.

Each paradigm has its unique advantage depending on the project the programmer is working on. The relationship of these three paradigms is complementary and can coexist with each other, wherein event-driven programming can be utilized with OOP for organising and managing the functionality of events, and procedural programming can exist within a class wherein it makes the methods or logic structured and efficient.

In terms of the application of the three paradigms in my North Sussex Judo codebase, the integration achieves a good balance of user-friendliness, scalability, and efficiency of the program. The structure of the code as well as the modularization enables the user to further expand the program in the future, like editing the logic inside the methods, without affecting the entire codebase.

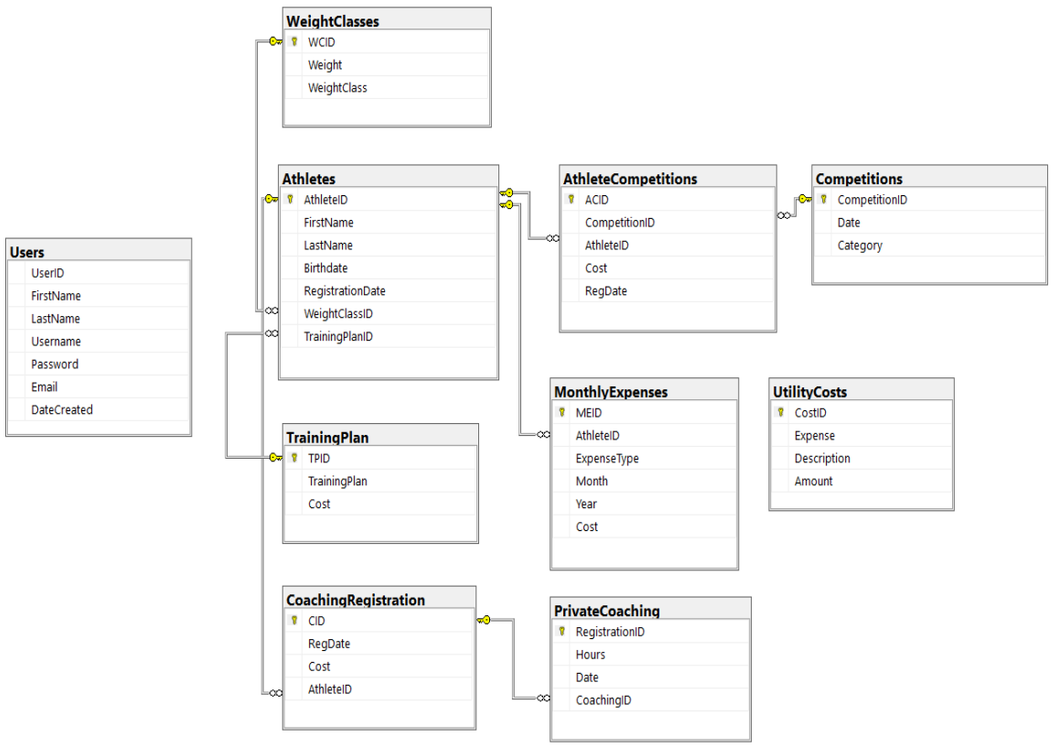
In my sample banking system program, I explained how it applied the three paradigms effectively. This makes the codebase organised, understandable, and easily maintained in case of adjustments.

Overall, the functionality of the North Sussex Judo program is comprehensive for both users and programmers since it follows the three paradigms.

Design Document Proposal

This Design Document Proposal will showcase the appropriate database schema I created for the project given.

Entity-Relationship Diagram (ERD) is a structured diagram, visualizing the database design, wherein you can pinpoint the flaws and make modifications before inserting data into the database. My schema is created with SQL Server Management Studio Management 19, wherein I can see the full picture of the relationships and columns for each table.

Database Design for North Sussex Judo

Reference Number: 1
Version Number: 5.0
System Name: North Sussex Judo
Module Name: Not Applicable

Addressing the requirements outlined by North Sussex Judo, revisions have been made to ensure proper implementation of foreign keys and primary keys on each table of the database, ensuring data integrity.

Primary keys are the unique identifier for a certain row in a table. It does not allow duplication and may serve as a reference for the foreign key (Cockroach Labs, 2023). For example, in my Athletes table in the database, each athlete has a unique athlete ID to ensure that no athlete have the same ID, especially in cases of data retrieval and manipulation.

Foreign keys serves as the link and establishes the relationship on related tables. It ensures data consistency since it references existing data, which is the primary key (Cockroach Labs, 2023). For example, in my AthleteCompetitions table, the AthleteID is the foreign key and it references from the Athletes table. This confirms this particular registered athlete is partaking in a competition.

Other than establishing relationships, I applied normalization in my database design. Normalisation is the method that aims to organise the database structure by eliminating data redundancy, which improves the data consistency (Rouse, 2019). My previous database designs will be shown in the appendices section to illustrate how I ended up on this final design, and progressively normalised. I will further discuss in the evaluation on how I can improve it.

In my design, I take note of the column names for each table and ensure that they have appropriate data types, to accommodate the data being stored. It is found in my database design to display two decimal places on numeric data such as costs, and weight, as per requirement. I will discuss how each table in the database schema aligns with the requirements given.

Data Dictionary

The data dictionary is a collection of metadata that provides the structure, format, and content of data that will be stored in my database.

Users					
Field Name	Data Type	Data Format	Field Size	Description	Example
UserID(PK)	int	numeric	5	Unique ID of Athlete	1
FirstName	nvarchar(50)	text	50	First Name of user	Jane
LastName	nvarchar(50)	text	50	Last Name of user	Doe
Username	nvarchar(50)	text	50	Chosen username of user	Jane_Doe
Password	nvarchar(50)	text	50	Chosen password of user	janedoe27
Email	nvarchar(50)	text	50	Chosen email of user	janedoe@gmail.com
DateCreated	date	YYYY-MM-DD	8	Date the user created the account	2024-01-29

	UserID	FirstName	LastName	Username	Password	Email	DateCreated
1	1	John	Doe	John_Doe	johndoe27	johndoe@gmail.com	2024-01-29
2	2	Jane	Doe	Jane_Doe	janedoe27	janedoe@gmail.com	2024-01-29

The **Users table** is for the administrators or authorized users logging in to the program and is essential for tracking the users that have access to it. The username and password will be used for the login process, thus contributing to the security and user management of the project, as specified in the requirement.

Athletes					
Field Name	Data Type	Data Format	Field Size	Description	Example
AthleteID (PK)	int	numeric	5	Unique ID of Athlete	1
FirstName	nvarchar(50)	text	50	First Name of athlete	Amanda
LastName	nvarchar(50)	text	50	Last Name of athlete	Ang
Birthdate	date	YYYY-MM-DD	8	Date of Birth of athlete	2005-07–27
RegistrationDate	date	YYYY-MM-DD	8	Date the athlete registered	2024-01-11
WeightClassID (FK)	int	numeric	5	Foreign key to WeightClasses table	1
TrainingPlanID (FK)	int	numeric	5	Foreign key to TrainingPlan table	1

	AthleteID	FirstName	LastName	Birthdate	RegistrationDate	WeightClassID	TrainingPlanID
1	1	Paul	Reed	1990-07-27	2024-01-29	1	1
2	2	John	Cruz	1985-08-07	2024-01-29	2	2

The **Athletes table** is where the athlete's personal information is stored. One of the most crucial requirements for the program is to allow the user to record and manage the athlete's information including athlete name, training plan, current weight, and competition weight category. An athlete can be registered through the Athlete Registration Form of the program wherein personal details will be entered, such as first name and last name.

The relationship between the weight class and the training plan of the athlete is established through foreign keys. WeightClassID and TrainingPlanID fields serve as the foreign key referencing the WeightClasses table and TrainingPlan table, respectively. A one-to-many relationship has been formed on these tables, wherein one weight class or training plan is associated with many athletes. My athletes table is designed to track athletes' records easily, associate athletes to their respective training plans and weight classes, and ensure data integrity through unique identifiers.

WeightClasses					
Field Name	Data Type	Data Format	Field Size	Description	Example
WCID (PK)	int	numeric	5	Unique ID of Weight Class	1
Weight	decimal(5,2)	numeric	6	Weight in kg of the athlete	70
Weight Class	nvarchar(50)	text	50	Weight category to which the athlete has been classified	Flyweight

	WCID	Weight	WeightClass
1	1	85.68	Light-Middleweight
2	2	90.00	Middleweight

The **WeightClasses table** is where the weight and corresponding weight category are assigned to that specific athlete. The requirement states that the database should record the current weight of athletes in kilograms and then assign them to the predefined weight categories.

TrainingPlan					
Field Name	Data Type	Data Format	Field Size	Description	Example
TPID (PK)	int	numeric	5	Unique ID of Training Plan	1
Training Plan	nvarchar(50)	text	50	Training plan in which the athlete is enrolled	Intermediate

Cost	decimal(5,2)	numeric	6	Cost associated with the training plan	30.00

	TPID	TrainingPlan	Cost
1	1	Intermediate	30.00
2	2	Elite	35.00

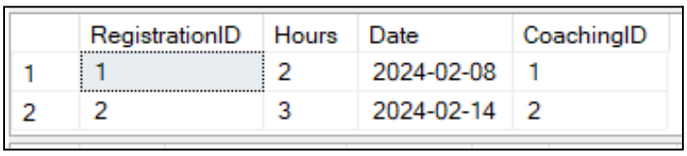
The **TrainingPlan table** is where the athlete's enrolled training program and its corresponding cost can be found. The cost of the training program will be required for calculating the total cost for the month, as required by the project. This table allows the administrator to track which athlete is enrolled in which programs.

CoachingRegistration					
Field Name	Data Type	Data Format	Field Size	Description	Example
CID (PK)	int	numeric	5	Unique ID of Coaching	1
RegDate	date	YYYY-MM-DD	8	Date the athlete registered for coaching	2024-01-09
Cost	decimal(5,2)	numeric	6	Cost associated with the private coaching	19.00
AthleteID (FK)	int	numeric	5	Foreign key to Athletes table	1

	CID	RegDate	Cost	AthleteID
1	1	2024-01-29	19.00	1
2	2	2024-01-29	28.50	2

The **CoachingRegistration table** is where the private coaching registration information of the athlete is stored. The cost can be found here wherein it is multiplied by the number of coaching hours that the athlete is enrolled. This will also be crucial for calculating the total cost for the month. The relationship established between athletes and private coaching is one-to-many since one athlete can register for multiple coaching sessions over time. This allows the flexibility of an optional process of adding an arbitrary number of private coaching sessions for each athlete, as per requirement.

PrivateCoaching					
Field Name	Data Type	Data Format	Field Size	Description	Example
RegistrationID (PK)	int	numeric	5	Unique ID of Registration	1
Hours	int	numeric	5	Number of hours for which the athlete has enrolled in private coaching	2
Date	date	YYYY-MM-DD	8	Date of the actual private coaching	2024-01-19
CoachingID (FK)	int	numeric	5	Foreign Key to CoachingRegistration table	1



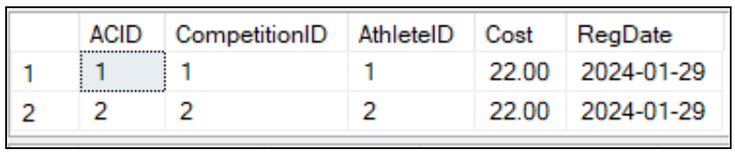
	RegistrationID	Hours	Date	CoachingID
1	1	2	2024-02-08	1
2	2	3	2024-02-14	2

The **PrivateCoaching table** is where the private coaching details are recorded such as the number of hours registered, the date of the actual coaching session, and the foreign key, CoachingID, CoachingRegistration table, that associates each private coaching session to a particular coaching registration entry.

With the CoachingRegistration and PrivateCoaching tables, NSJ can efficiently record multiple private coaching sessions for each athlete, ensuring accurate monitoring of coaching hours and corresponding costs, as per requirement.

AthleteCompetitions					
Field Name	Data Type	Data Format	Field Size	Description	Example
ACID (PK)	int	numeric	5	Unique ID of Athlete Competitions	1
CompetitionID (FK)	int	numeric	5	Foreign Key to Competitions table	1

AthleteID (FK)	int	numeric	5	Foreign Key to Athletes table	1
Cost	decimal(5, 2)	numeric	6	Cost associated with the competitions	22.00
RegDate	date	YYYY-MM-DD	8	Date the athlete registered for the competition	2024-01-09

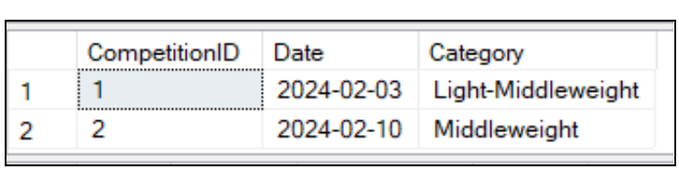


	ACID	CompetitionID	AthleteID	Cost	RegDate
1	1	1	1	22.00	2024-01-29
2	2	2	2	22.00	2024-01-29

The **AthleteCompetitions table** is where the athlete participation in competition details is recorded. The CompetitionID links to a specific competition while the AthleteID links to the specific athlete who joined that competition. The associated cost can also be found here where it will be part of the calculation of the total cost for the month. This table serves as the junction table between the Athlete and Competitions wherein it records multiple athletes participating in multiple competitions over time, establishing a many-to-many relationship.

As per requirement, NSJ requests to allow an arbitrary number of interested athletes to participate in the competition, which aligns with the structure of my database.

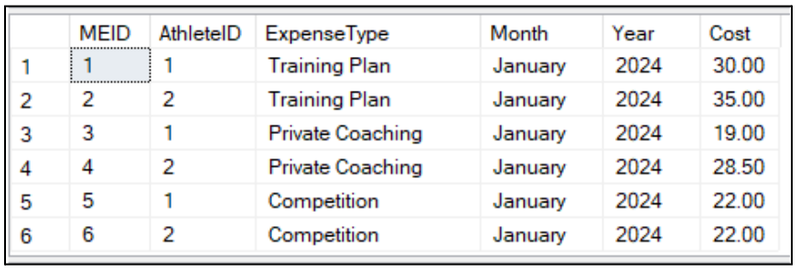
Competitions					
Field Name	Data Type	Data Format	Field Size	Description	Example
CompetitionID (PK)	int	numeric	5	Unique ID of Competition	1
Date	date	YYYY-MM-DD	8	Date of the actual competition	2024-01-27
Category	nvarchar(50)	text	50	Weight category associated with the competition	Flyweight



	CompetitionID	Date	Category
1	1	2024-02-03	Light-Middleweight
2	2	2024-02-10	Middleweight

The **Competitions table** is where the specific competition information is stored, such as the data of the actual competition as well as the category for that competition. This is where the foreign key, CompetitionID of the AthleteCompetitions table, is referencing, to track which competition the athlete joined.

MonthlyExpenses					
Field Name	Data Type	Data Format	Field Size	Description	Example
MEID (PK)	int	numeric	5	Unique ID of Monthly Expenses	1
AthleteID (FK)	int	numeric	5	Foreign Key to Athletes table	1
ExpenseType	nvarchar(50)	text	50	Type of expense: private coaching, competition, or training plan	Training Plan
Month	nvarchar(50)	text	50	Month of the athlete's transaction	January
Year	int	YYYY	4	Year of the athlete's transaction	2024
Cost	decimal(10,2)	numeric	11	Cost associated with the expense	30.00



	MEID	AthleteID	ExpenseType	Month	Year	Cost
1	1	1	Training Plan	January	2024	30.00
2	2	2	Training Plan	January	2024	35.00
3	3	1	Private Coaching	January	2024	19.00
4	4	2	Private Coaching	January	2024	28.50
5	5	1	Competition	January	2024	22.00
6	6	2	Competition	January	2024	22.00

The **MonthlyExpenses table** is where the itemized list of all costs for the month is found, which is a critical component in fulfilling the requirements for the NSJ project. The requirement also stated to compute the total cost of training and competitions for the month (additionally private coaching), wherein this is the table that the program will retrieve to calculate the overall costs of the athletes for each month. Individual transactions made by the athlete will automatically be stored here and will dictate the expense type, month, and year it was transacted, as well as the associated cost of that expense.

To conclude, this table is a comprehensive record of expenses incurred by each athlete, which is crucial for effectively tracking and generating reports of the financial activities of North Sussex Judo athletes.

UtilityCosts					
Field Name	Data Type	Data Format	Field Size	Description	Example
CostID(PK)	int	numeric	5	Unique ID of Utility Costs	1
Expense	nvarchar(50)	text	50	Type of expense: private coaching, competition, or training plan	Beginner
Description	nvarchar(70)	text	70	Information about the Expense	Training Plan
Amount	decimal(10,2)	numeric	11	Amount associated with the expense	25.00

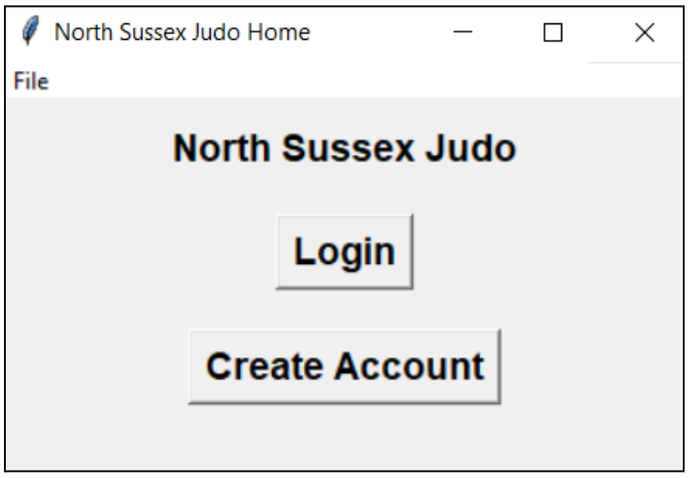
	CostID	Expense	Description	Amount
1	1	Beginner	Training Plan	100.00
2	2	Intermediate	Training Plan	120.00
3	3	Elite	Training Plan	140.00
4	4	Private Coaching	Private Tuition (per/hr)	9.50
5	5	Competition	Competition entry fee	22.00

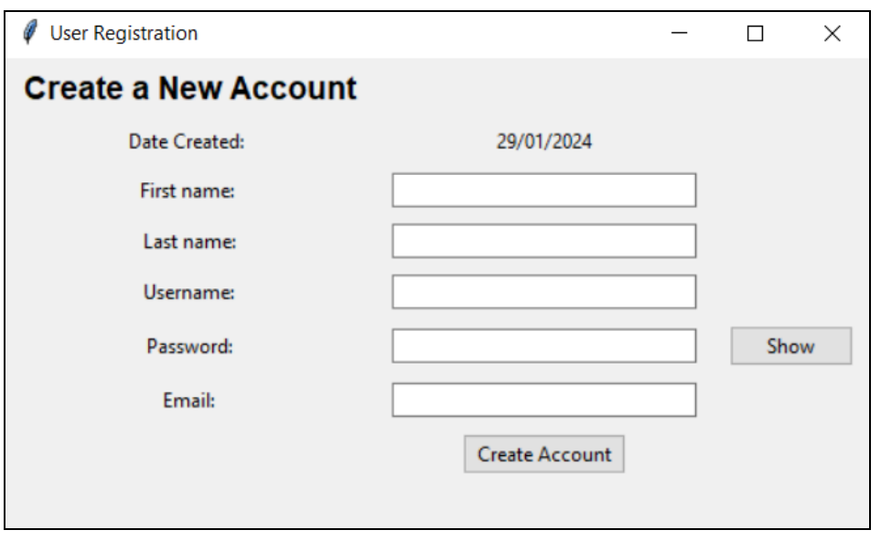
The **UtilityCosts table** is where storing expense information related to financial activities within the NSJ. The amount recorded for the corresponding expense is where the cost for training plans, competitions, and private coaching serves as the reference point for expense retrieval. It is an independent table and won't need relationships or foreign keys, simply, the program knows where to retrieve the costs from.

This ensures that the cost of various activities is accurately calculated at all transactions. Administrators can adjust the costs for the expenses based on their decision or circumstances, such as inflation, as time goes by. Thereby contributing to the efficiency, and adaptability of the program, ensuring that the cost and expenses are consistent across the entire system.

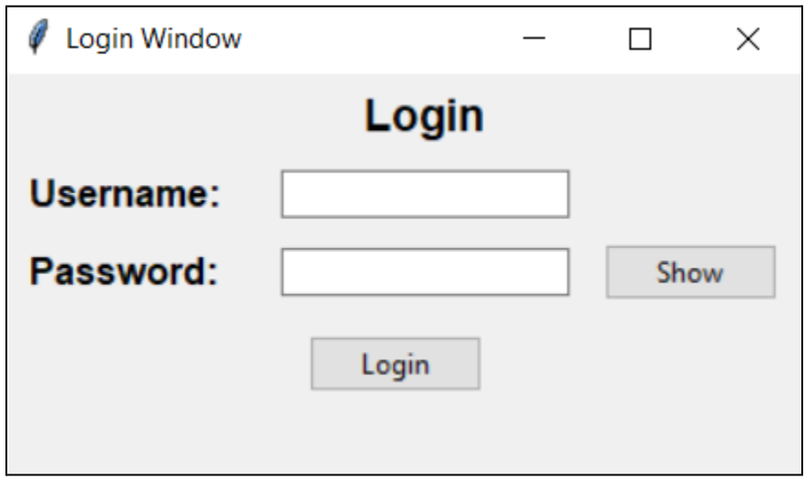
User Interfaces for North Sussex Judo

All of the functionality and operations of the programs interact with the Data Access Layer module to execute database operations securely and effectively.

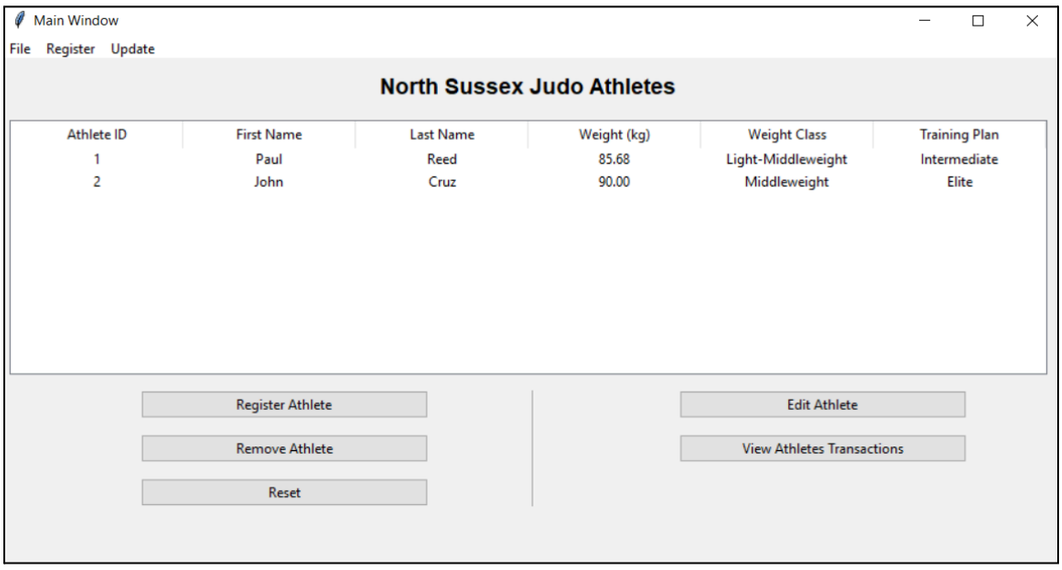
Home Window Interface

Reference Number: 2
Module Name: __Home_Window.py
The Home Window Interface serves as the starting window of the program, wherein the Login and Create Account buttons are displayed. This window fulfills the user requirements of the project for authentication and security before accessing the main functionalities and user interactions of the program. Additionally, The menu bar contains “File” wherein you have an option to exit the window.

User Registration Window Interface


Reference Number: 3
Module Name: User_Registration.py
The User Registration Interface serves as the account creation for North Sussex Judo administrators or authorized users, before logging in to the program. The date is automatically set in real-time for data integrity. Required fields such as first name, last name, username, password, and email must be filled out by the user. The password is normally hidden but has an option to show it to avoid typographical errors. Upon successful account creation, the user's data will be stored in the User's table of the database and proceed to the login window.

Login Window Interface

Reference Number: 4
Module Name: Login_Window.py
The Login Window Interface serves as the gateway for authorized users to access the program. The users are prompted to input their Username and Password fields to gain access. After successfully logging in, the user is granted access to the main window and the program functionalities. This window is important to ensure that the user accounts and athletes' data within the system are secured.

Main Window Interface



Reference Number: 5

Module Name: Main_Window.py

The Main Window Interface is the main interface for managing and performing operations related to North Sussex Judo athletes. Each button has a specific purpose and functionality, based on the specified project requirements.

a. Treeview of North Sussex Judo Athletes - this treeview is a tabular representation of athletes registered in NSJ with their necessary information for quick viewing.

b. Action buttons:

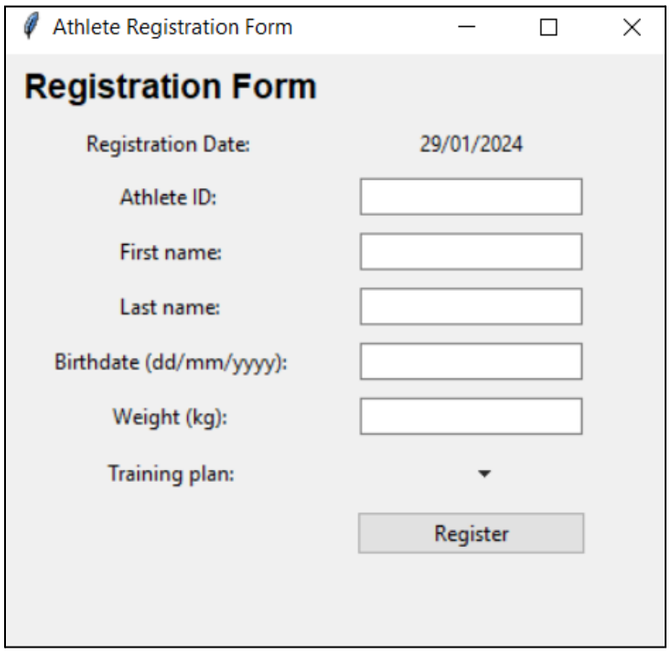
1. Register Athlete - this button directs to the Athlete Registration Window.
2. Remove Athlete - this button removes a specific athlete from the Athletes database. This functionality is for demo purposes and is not advised for real-world scenarios or databases.
3. Reset - this button refreshes the treeview.
4. Edit Athlete - this button directs to the Edit Athlete Window.
5. View Athletes Transactions - this button directs to the Athletes Trainsactions Window.

` c. Menu bar:

1. File - option to exit the main window.
2. Register:
    a. Competitions - this button directs to the Competitions Window.
    b. Private Coaching - this button directs to the Private Coaching Registration Window.
3. Update:
    a. Training Plan and Private Coaching - this button directs to the TPPC Window.

b. Utility Costs - this button directs to the Utility Costs Window.

Athlete Registration Form Interface

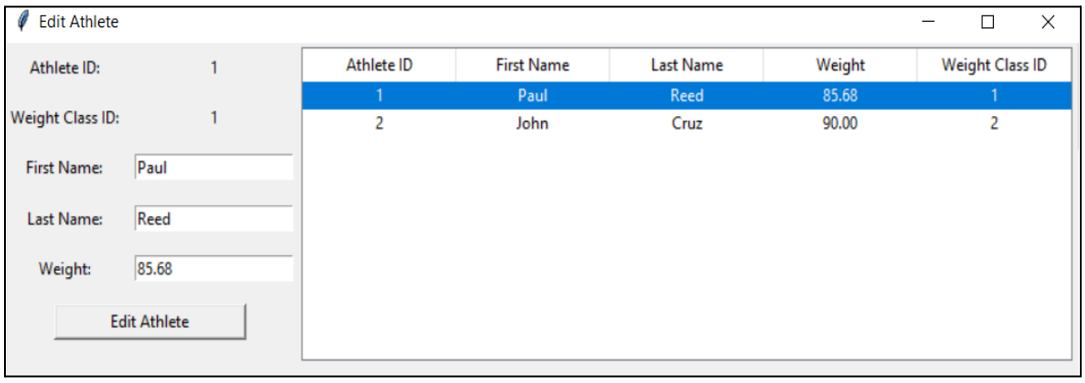


Reference Number: 6

Module Name: Athlete_Registration_Form.py

The Athlete Registration Form Window Interface facilitates the registration of the athletes by filling out the required fields, including athlete ID, first name, last name, birth date, and so on. This interface is straightforward since the fields are properly labeled. Upon clicking the register button, the athlete information will undergo data validation in order to check if the data is complete and accurate. After successful registration, the athlete's data will automatically be recorded in the Athletes table of the database.

Edit Athlete Window Interface

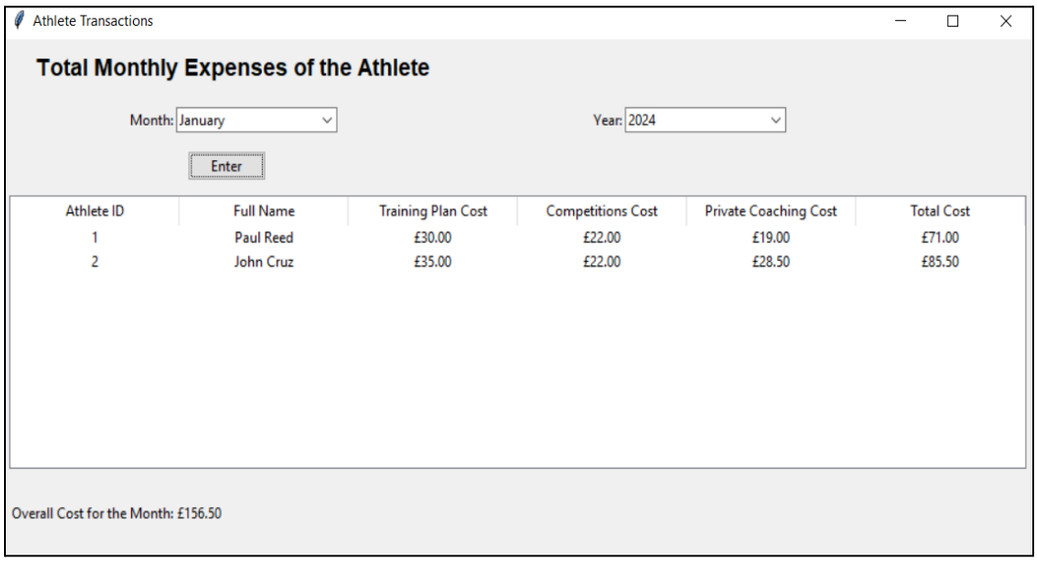


Reference Number: 7

Module Name: Edit_Athlete.py

The Edit Athlete Window allows users to edit athlete's particular information, including their first name, last name, and weight. For this to work, the user will click a row in the treeview corresponding to the athlete. Subsequently, the athlete information will automatically populate the entry widgets, making it editable. The main purpose of this window is to update athlete's information. For example, users can correct typographical errors in the athlete's name or make changes to the athlete's weight. Notably, modifying the weight on a large scale triggers an automatic realignment of the athlete's weight category.

Athlete Transactions Window Interface



Reference Number: 8

Module Name: Athlete_Transactions.py

The Athlete Transactions Window showcases the expenses incurred by the athlete on training plans, competitions, and private coaching. To compute the total cost for each athlete, the program sums up the amounts transacted on each type of expense. For this to work, the user must select a month and year to view the total expenses for each athlete during that period. Once selected, the monthly expenses of the athletes are populated in the treeview.

Below the treeview is the overall cost for the selected month. This figure computes the total expenditure made collectively by all NSJ athletes for the particular month. This was computed by summing up the total cost of each athlete. The MonthlyExpenses table of the database is where the amounts are retrieved from.

This functionality allows users to track financial transactions made on NSJ and provides an itemized list of all costs for the month, as specified in the requirements.

Private Coaching Registration Interface

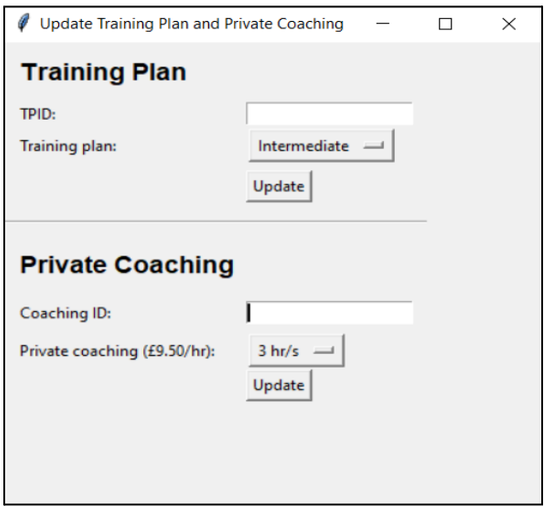


Reference Number: 9

Module Name: Private_Coaching_Registration.py

The Private Coaching Registration Window allows users to register athletes for a private coaching session. Since this is an optional process as stated in the requirement, I separated it from the Athlete Registration Form. In this private coaching form, the user can enter the date of the actual private coaching session in which the athlete will partake. The selection of athletes is facilitated through a dropdown for easier navigation. Lastly, the user can select the number of hours for the session, the dropdown is capped at a maximum of five hours as per requirement.

Update Training Plan and Private Coaching Interface

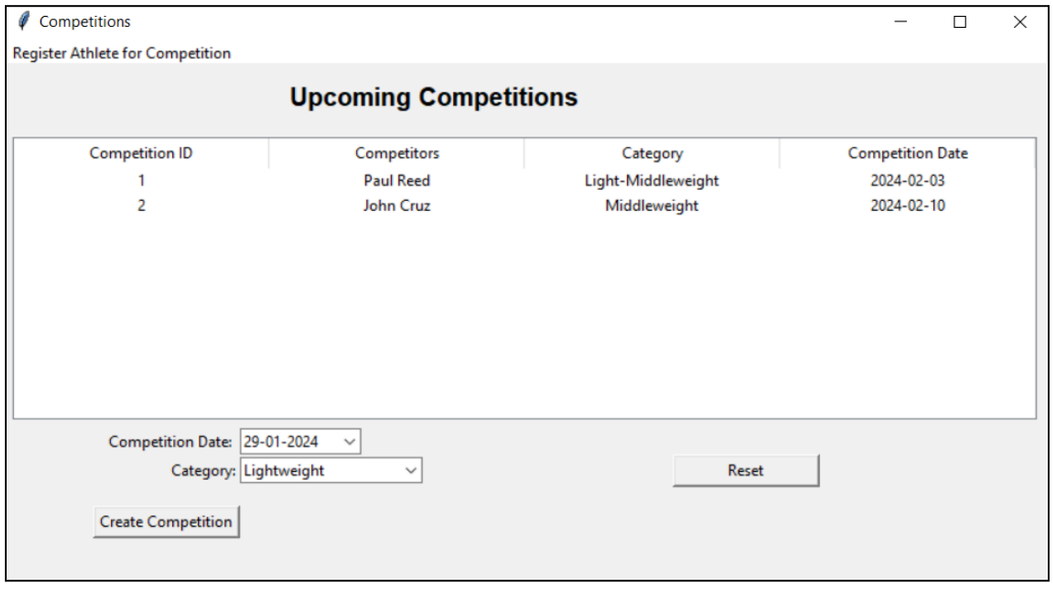


Reference Number: 10

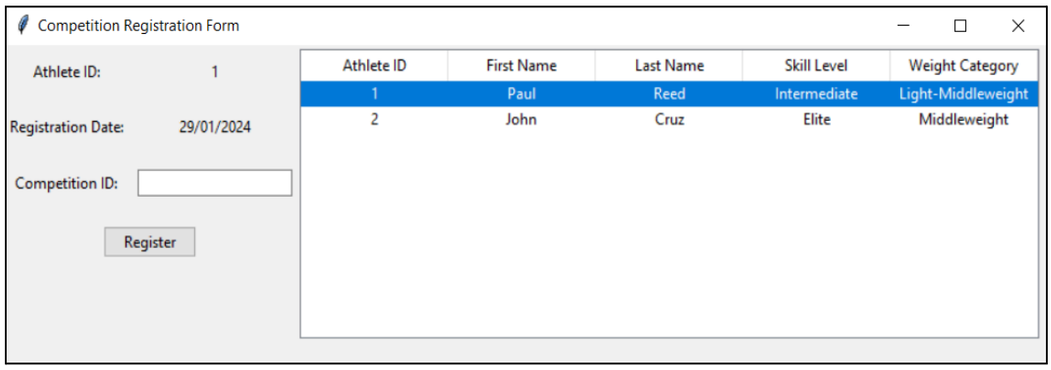
Module Name: TPPC_Update.py

The Update Training Plan and Private Coaching Interface or TPPC Update has the function of modifying the training plan and private coaching details of the specific athlete. For example, if the athlete wants to upgrade their training plan at some point in time, the user will enter their unique TPID and then select the new training plan. Similarly, for updating the private coaching, the user will enter the Coaching ID of that specific private coaching session to update the hours. The purpose of the window is to update the existing training plan or private coaching that the athlete is enrolled in.

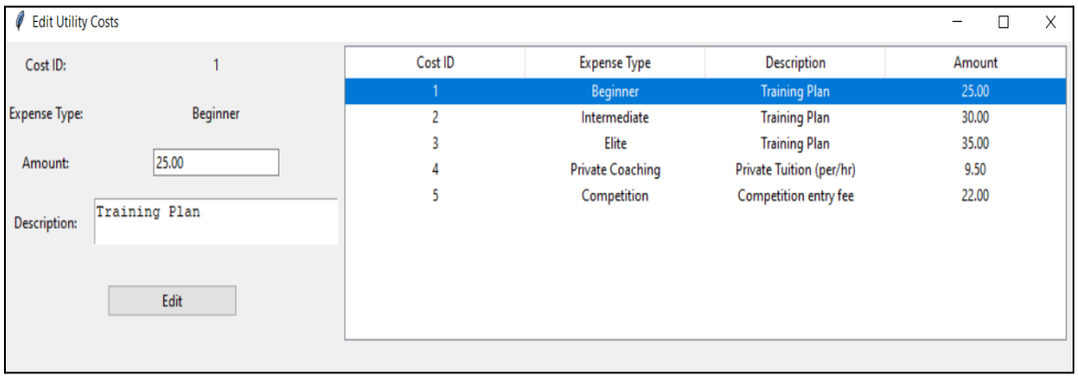
Competitions Window Interface



Reference Number: 11
Module Name: Competitions.py
The Competitions Window serves as an overview of the athletes scheduled to partake in upcoming competitions. The athletes in the Treeview are the ones who have successfully registered for a certain competition. Below the treeview, users can create a competition by selecting the date it will be held and its corresponding category. For data validation, as per requirement, the user is restricted to choosing second Saturday dates only.

Competition Registration Window Interface

Reference Number: 12
Module Name: Competition_Registration.py
The Competition Registration Window facilitates the registration of athletes in a specific competition. In the treeview, only athletes classified with the skill level of Intermediate and Elite are shown here, since it is stated in the requirement that Beginner can't participate in competitions. This helps the user to quickly identify which athlete is qualified for the competitions. Additionally, the treeview includes the weight category of the following athletes, enabling users that the category of the competition matches the weight category they belong to. For data integrity and accuracy, data validation is implemented wherein prevents athletes from joining a competition that does not match their weight category. Lastly, the user will enter the unique competition ID for the desired competition in which the athlete will partake.

Utility Costs Window Interface



Reference Number: 13

Module Name: Utility_Costs.py

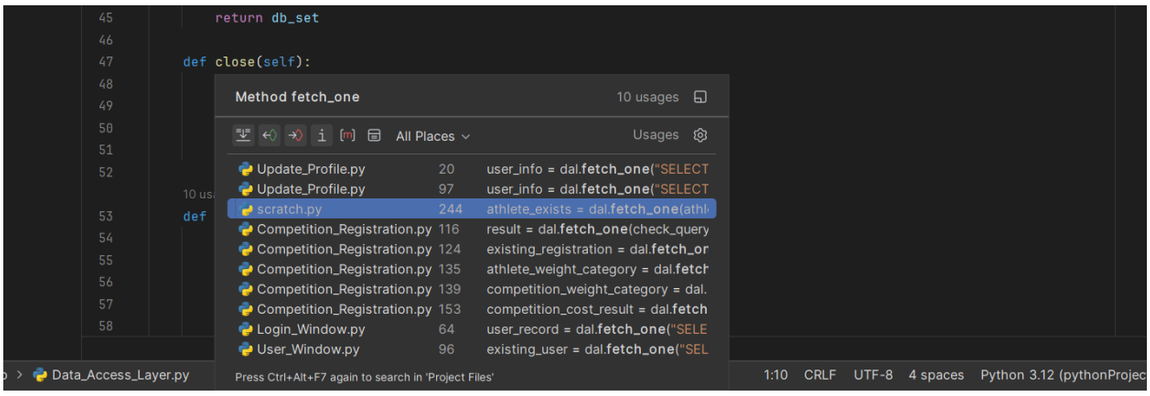
The Utility Costs Window serves as a repository for all of the expense information such as its costs and description. The information in the treeview has been produced by executing SQL commands. Although it is not explicitly stated to produce a repository for the expenses, it is a very useful functionality for the North Sussex Judo company. For example, when the administrators come up with a decision to adjust the prices of their expenses, it can easily be updated within his window.

Its main function is it act as a retrieval for the amount of a particular expense when an athlete registers for a training plan, private coaching, and competition. So for example, when the administrator decides to increase the prices, new athletes will be charged with the updated prices reflected in the database. This repository ensures consistency across the system and avoids discrepancies when calculating the total cost or monthly expenses.

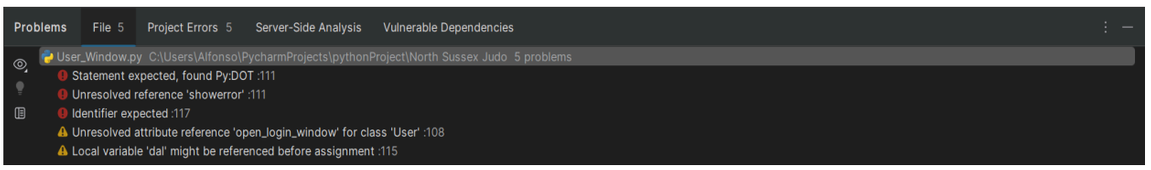
Application Development Process and Implementation

The IDE that I utilized for this program is Pycharm, specifically designed for the Python environment and it is developed by JetBrains. It is available for free download and use. The features are enough to enhance my coding experience, this includes Python code editing, database & SQL, language support, python profiler, testing, refactoring, and debugging (Jetbrains, 2019). It has a clean and user-friendly interface which makes it accessible for both beginners and developers, including a feature to customize the appearance and layout based on preference.

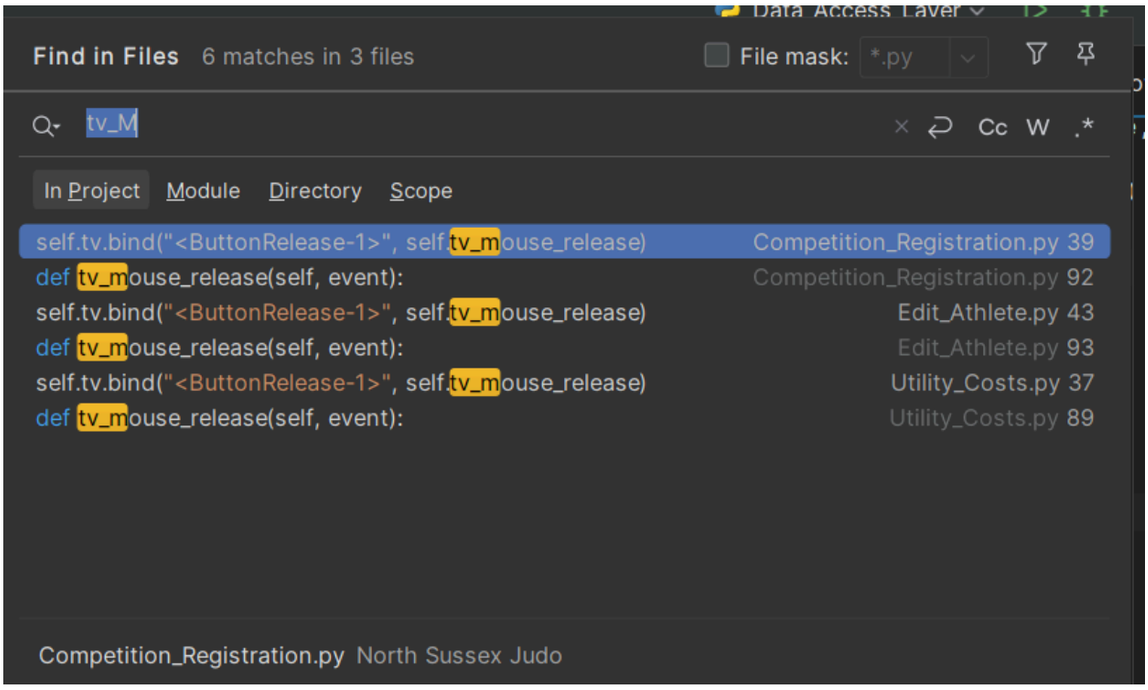
Pycharm has provided me with a lot of tools and optimization for the development process in my program. I utilized the debugging tools available, code refactoring, and also version control. With Pycharm, I can also track the number of usage of functions, which is extremely useful. For example:



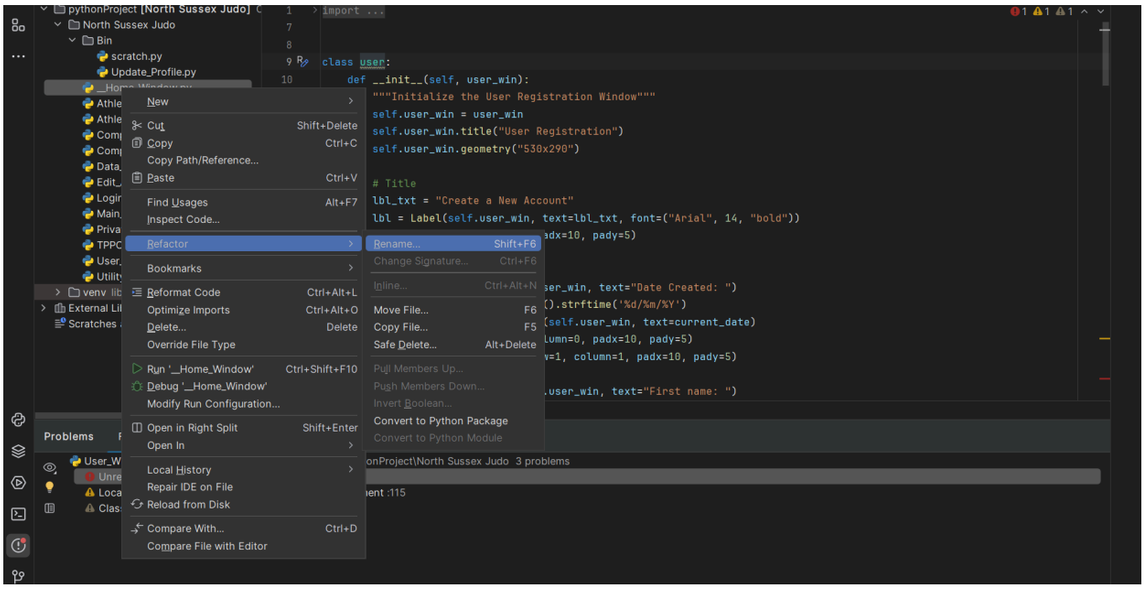
This provides ease of navigation in monitoring the usage of the function, especially since I have a lot of modules stored in my project.



Pycharm also has a console that automatically detects problems within my code, such as indentations, variables, references, code styles (PEP8), and other errors that help me enhance my code efficiently. Aside from that, it lessens my time in debugging with its automated error detection.



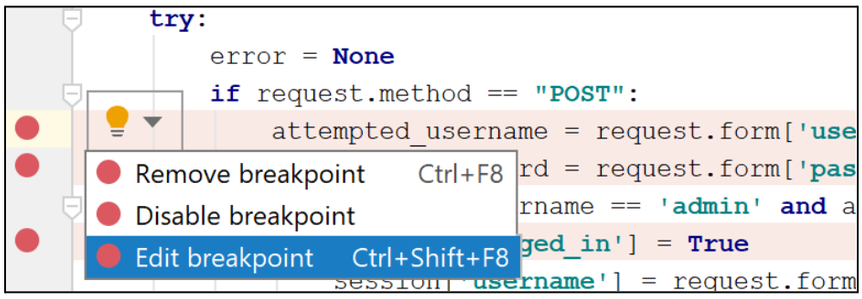
Pycharm has a feature to match a text within your project. I find this useful whenever I want to be consistent in naming my variables, as well as to navigate where the text was used and what module. I can also adjust the scope of navigation which makes my development process easier and flexible.



With Pycharm's straightforward refactoring tools, I was able to be flexible in my development process and did not come up with any issues with renaming, copying, or deleting files.

Pycharm has offered me a lot of tools that helped me boost my productivity in my development process. I will further discuss this in the following sections of the paper. Pycharm has several features or ways to debug the program in Python, depending on the error encountered.

a. **Breakpoints**



Breakpoints are special markers that tell the debugger to suspend the execution of the program at that particular point. Pycharm offers line breakpoints and exception breakpoints (Pycharm Help, n.d.).

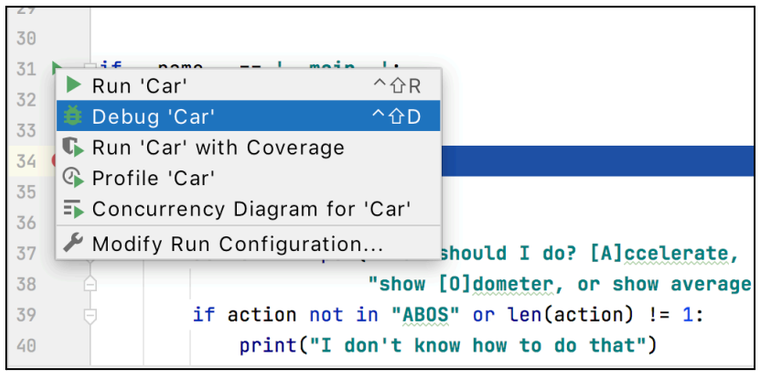
1. **Line breakpoints** stop the execution of the program upon reaching the line where the breakpoint was marked. This is the most common breakpoint used.
2. **Exception breakpoints** stop the program when specific Exceptions are thrown, which guides on exception-handling of the code.

To add or remove breakpoints, click on the gutter at the line of the code the user wants to set. Breakpoints can be configured by right-clicking and there will be options such as suspend, condition, log messages, and more. Moreover, it has a feature that displays the statuses or icons such as verified, warning, invalid, muted, disabled, and so on (Pycharm Help, n.d.).

Breakpoints can be used for debug printing rather than using print statements. It can also examine the values of variables and isolate the errors within the codebase. In short, it pinpoints the line or block of code where unexpected behavior or issues occur, making it easier and faster to troubleshoot.

I have used a breakpoint at my competition registration form when I have an unexpected behavior within the Treeview, wherein the functions won't work properly. So, I placed a breakpoint before the database operation so I could inspect the state of the program before executing the queries. In that way, I found out the error was on one of my methods inside the class and not on the try-except block.

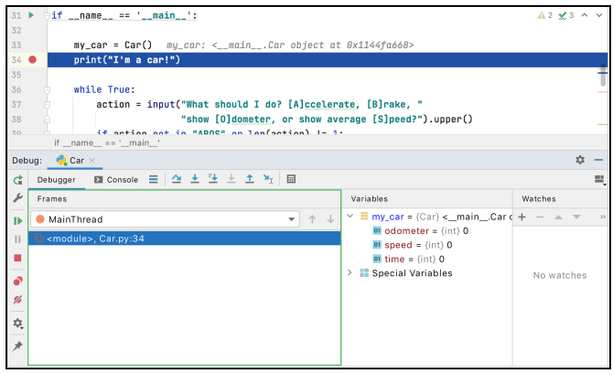
b. **Start the debugger session**



The debugger session is similar to running the program normally: clicking the "Debug" button at the toolbar or right-clicking at the gutter. There are options like pause, restart, resume, and stop the debugger session at any point to examine the state or execution. This feature can be used when debugging non-responding applications and analyse the root of the problem (IntelliJ IDEA Help, n.d.). The pause functionality lets the user evaluate the expressions without breakpoints. Lastly, it allows the user to configure pre-launch tasks like external tools and scripts before running the debugging session. Moreover, This debugger session allows the programmer to inspect and manipulate the execution in real–time.

I used the debugger session tools when I encountered opening secondary windows on my Main Window. I checked the flow of execution of the program, paused it at specific lines, and greatly examined the values. The pause functionality is useful since I don't need breakpoints and I was able to pinpoint what causes the error for the non-responding windows.

c. **Examine suspended program**



When the debugger session runs, a debug window appears and will run normally unless one of the following occurs: a breakpoint is reached, manually paused the program. Afterward, it enables the user to analyse the current state and test scenarios at runtime (IntelliJ IDEA Help, n.d.).

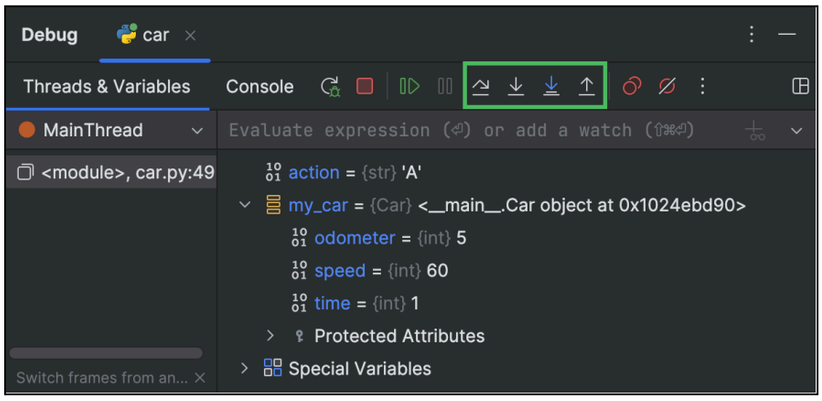
Frames are the representation of the call stack, which further displays the variables and series of method calls. This allows the user to analyse frames and method invocation during runtime. Also, it allows the user to inspect variables on that selected frame. The user can copy variables, compare

variables, and update variable values for thorough analysis (IntelliJ IDEA Help, n.d.)

Evaluating expressions can be utilized to acquire more information about the program state, and evaluation methods such as inline evaluation and arbitrary expression evaluation are available. This feature is not only important for identifying issues and validating assumptions but also for the user to understand the codebase which leads to more resolutions (JetBrains Rider Help, n.d.).

Watches monitor the value of the variable or the outcome of a complex expression during execution. Watches are evaluated continuously and provide real-time updates on values. Watches can be edited, added, copied, and deleted. Just go to the ' Threads and Variables' tab and the user can type the name of the variable he wants to watch. If an error occurs, it means that the variable is not defined (Jetbrains, n.d.).

d. **Step through the program**

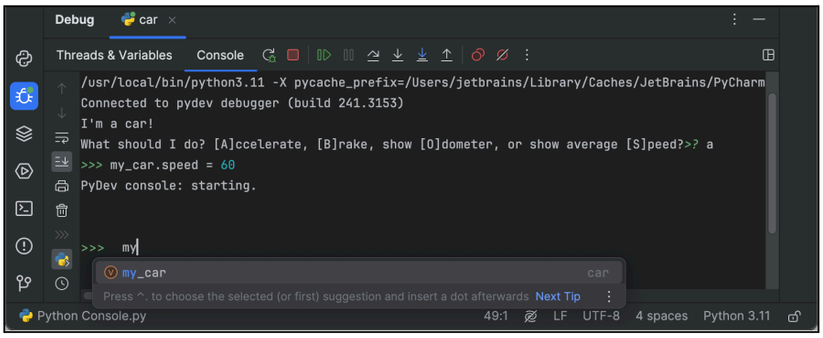


Stepping is an action that controls the program execution systematically. Pycharm offers several stepping options depending on the user's technique, which is located on the debug window toolbar. According to Jetbrains (2019), these are the following controls:

1. **Step Over** allows the debugger to move on the net line of the code, skip over the methods, and proceed to the next line of the caller method.
2. **Step Into** allows the debugger to view more details of a method and examine its behavior or inner workings.
3. **Step Out** allows the debugger to exit on the current method and return to the caller method.
4. **Run to Cursor** allows the debugger to continue the execution until it reaches the position of the caret.
5. **Force Step Into** forces the debugger to step into the method even though it is meant to be skipped.
6. **Force Run to Cursor** ignores the breakpoints and ensures that the cursor position is reached without being interrupted.

7. **Force Step Over** ignores the breakpoints within the methods called. The execution will not be interrupted even if there are breakpoints markets within the called method.
8. **Jump to Cursor** stops the execution and moves directly to the line where the caret is located. Breakpoints are ignored.

e. **Using Debug Console**



The debug console is where the output and error messages appear. The debug console automatically launches once the debugging session is started. In this console, there will be a prompt where the user can execute commands by utilizing code completion. Results will automatically displayed in the console (PyCharm Help, n.d.).

Code completion helps the user to complete names like classes, methods, and keywords visible on the codebase. Pycharm suggests code elements, reducing the typing effort and preventing errors. Types of completion are type-matching completion, statement completion, hippie completion, and more (Jetbrains, n.d.).

There are several actions available in the debug console such as navigating through command history, loading source code, toolbar buttons, and clipboard operations. The debug console is commonly useful for interactive debugging in a way of executing only a portion of the Python code during the debugging session (Pycharm Help, n.d.).

Code completion has helped me a lot in typing my codes, correcting programming syntax errors, and most importantly, suggesting the variables and methods to be used. This made my debugging faster and more efficient since it reduced the likelihood of raising an error on my codes. It also makes me aware of the code I need to modify and typos that occur while I code.

The debug console also has helped me read the error messages and what line I should pay attention to, which allows me to pinpoint and debug effectively. The debugging tools available in my IDE contributed to the efficiency of my development process of the program and debugging of errors.

Debugging Process for Secured and Robust Applications

The debugging process contributes to the security and robustness of the application since the debugger helps the programmer determine the potential weaknesses, vulnerabilities, and errors in the codebase. The utilization of debugging tools like breakpoints, step-throughs, and watches, helps application developers to realize validation flaws, potential exploits, authentication issues, and more. This lets them create a solution and implement appropriate security mechanisms and controls such as encryption, and better authentication, mitigating the unforeseen issues in the program.

For example, in my application, I built a form that registers athletes and stores it in my database. In the debugging process, I noticed that the program does not have appropriate input validation that verifies the information entered by the user. This makes the program vulnerable to hackers since harmful code can be sneaked in on these inputs. The use of a debugger will help me overcome this vulnerability as well as prevent malicious inputs that could harm my SQL database and program. Also, the debugger can be utilized to ensure that the connection between the SQL database and the program is seamless. To conclude, the debugging process ensures a safe and secure environment for the users and programmer, ensuring that exploits will not occur.

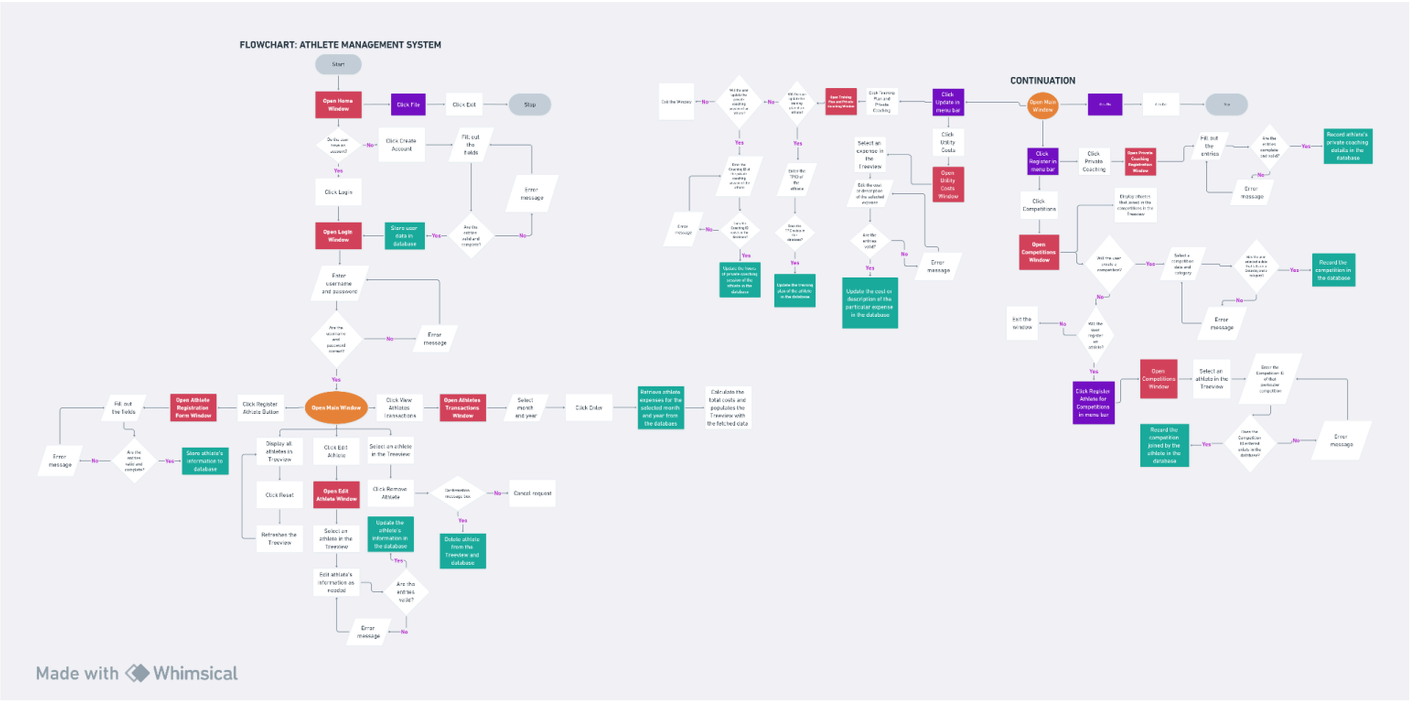
Flowchart and Algorithms

For the flowchart, I color-coded the shapes for a better understanding of the process and more readable. Red for opening a new window, purple for menu bar windows or actions, orange for the Main Window, and green for database interactions.

All of the modules in my program have an algorithm with the '__init__' method since it initializes the whole window containing the GUI widgets, Treeview, and event bindings for every button, and establishes a connection between the database and the program. Also, all of my modules have functions that open their window as needed, in order to bind it in the buttons for opening windows.

Lastly, all of my modules have try-except blocks that handle the exceptions raised, wherein my program will still continue executing even if some operations go wrong. This is better than showing tracebacks since it can sometimes be confusing and the program will completely crash. With try-except blocks, the program will continue running even if it encounters invalid input, especially since my program has different forms.

Flowchart for North Sussex Judo Program

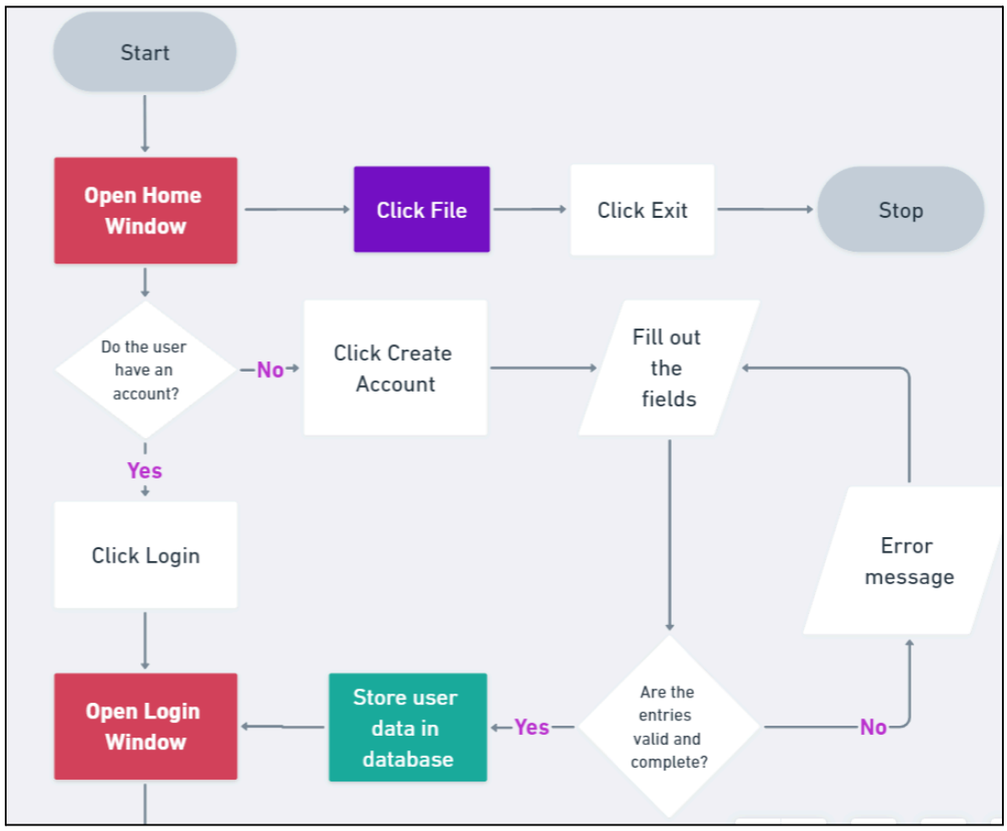


Reference Number: 14

System Name: Athlete Management System

Flowchart link: https://whimsical.com/01-04-ang-flowchart-J1BU3j2M71SKWgaLKnv4qe

User Registration Algorithm



Reference Number: 15

System Name: Athlete Management System

The Home Window is the starting window of the program. First, the user should think if they have an existing account already or not. If they have an account, they will click the login button and proceed to the Login window. This calls the 'open_login_window' function from the '__Home_Window.py'.

If not, they should click first the Create Account button and be redirected to the user registration window. The system calls the 'open_user_registration' function from the '__Home_Window.py'. The user will need to fill out all the fields. After registering, the registration will undergo data validation and check if the fields entered are valid and complete.

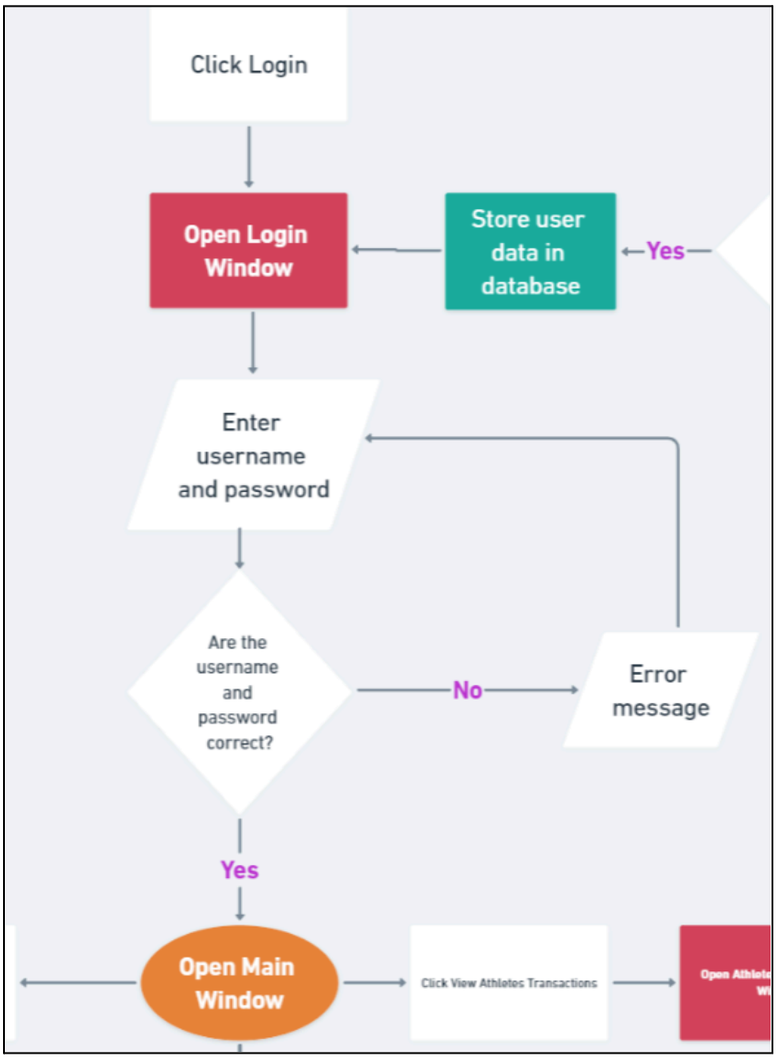
<u>Data validation includes:</u>

- Check if all of the fields are filled out.
- Check if the first name and last name do not contain digits and special characters.
- Check if '@' and '.' are in the email address entered.
- Check if the username does not exist in the database.

If the data is valid, the user's data will be recorded in the database. Afterward, the login window will automatically open. If the data is invalid, the system will pop up an error message box, requiring the user to try again.

It is also included in the Home Window wherein the program stops once the Exit is clicked from the File menu.

Login Algorithm



Reference Number: 16

System Name: Athlete Management System

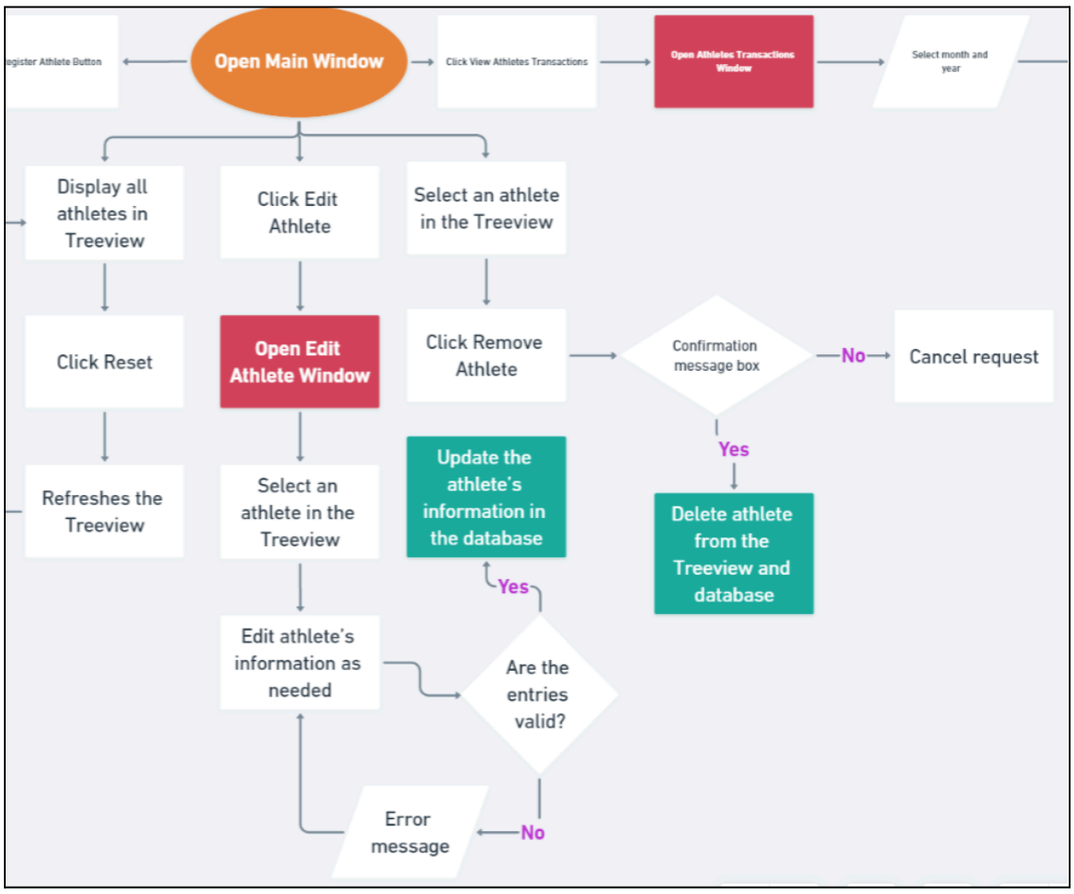
When the user has an account, the user will click the Login button in the Home Window and proceed to the Login Window. They are required to enter their username and password. After inputting their data, the user may click the login button. The login process will begin and undergo data validation.

The system will check if the username and password are correct. This is done by retrieving the credentials entered from the Users table in the database, ensuring that it corresponds to an existing account. If the username and password are correct, the system will automatically open the Main window.

Data validation includes:

- Checks if both username and password are entered.
- Check if the username and password are valid.

Reset Treeview, Edit Athlete, and Remove Athlete Algorithm



Reference Number: 17

System Name: Athlete Management System

The Main Window displays all of the athletes in the Treeview, as well as several buttons. For the Treeview to display, it opens the database connection then fetches athletes' information from the database with SQL query, the inserts it into the Treeview widgets.

Second, the Reset button simply refreshes the treeview. This is useful if there are changes made from the athlete's information, and added or removed an athlete.

Third, for the Remove Athlete button, the user will need to select an athlete in the Treeview. Afterward, the user may click the remove athlete button, prompting the user for confirmation before it is deleted. If the user confirms the deletion, the system will retrieve the athlete's ID from Treeview and then delete the athlete's data on the database. If not confirmed, it cancels the request.

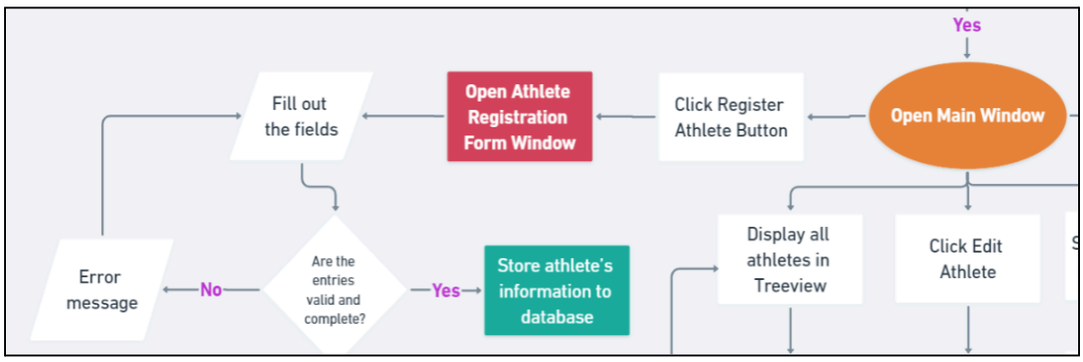
Lastly, when the Edit Athlete button is clicked, the system will open the Edit Athlete Window. Similarly, it has a tree view displaying all athletes. Same functionality as the main window's Treeview. But when the user selects an item in the treeview, it extracts the values and then populates the entry widgets with the selected athlete row. This is an algorithm of the Treeview selection handling with the 'tv_MouseRelease' method. After selecting, the user may now edit the athlete's information through the entry widgets. After editing, the entries will undergo data validation.

<u>Data validation includes:</u>

- Check if the first name and last name do not contain digits and special characters.
- Check if the weight is in numeric value only.
- Check if all fields are filled out.

If the data is valid, the system will update the athlete's data in the database. If not valid, the request is canceled.

Athlete Registration Form Algorithm



Reference Number: 18

System Name: Athlete Management System

When the Register Athlete button is clicked from the Main window, it opens a new window, Athlete Registration Form. The user will be filling out the necessary fields to register an athlete. Upon submission, the entries will

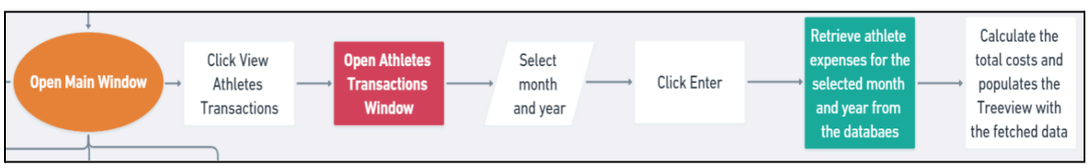
undergo data validation.

Data validation includes:

- Check if all the fields are filled out.
- Check if the Athlete ID entered is numeric.
- Check if the first name and last name don't have digits or special characters.
- Check if the weight entered is valid.

If the data is valid, the athlete's data will be stored in the database at the Athletes table and is successfully part of the North Sussex Judo athletes. The weight category will also be determined based on the weight entered. This is found executed in the 'determine_weight_class' method. The registration window will close and the user may click the Reset button below the Treeview in the main window, to display the new athlete. If the data is invalid, an error will show up and the user will need to try again.

Athlete Transactions Algorithm



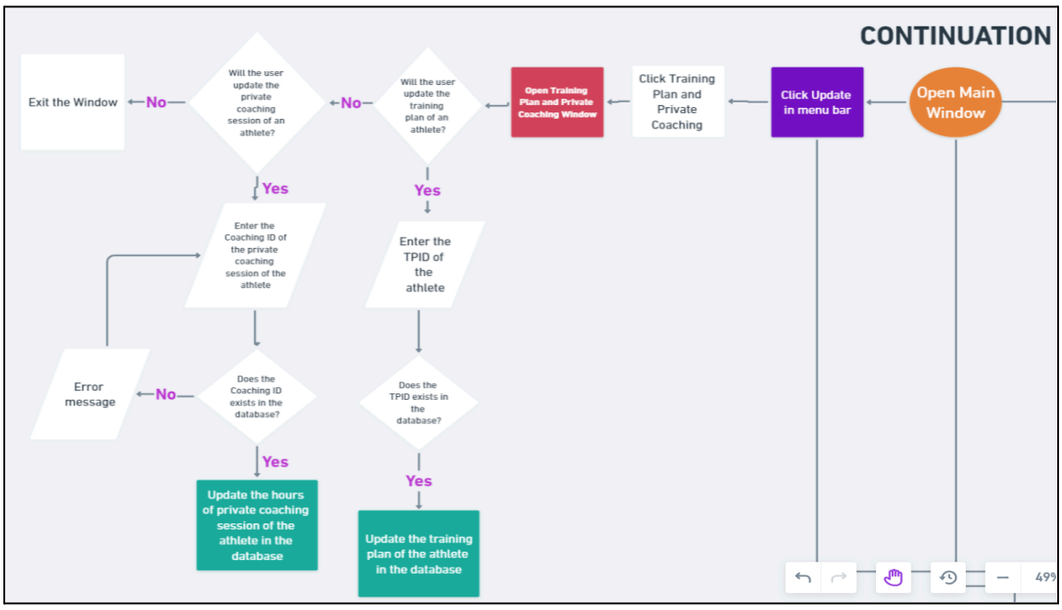
Reference Number: 19

System Name: Athlete Management System

There is a 'View Athletes Transactions' button in the Main Window, and once clicked, it will direct to a new window, Athletes Transactions Window. On the window, the user can see the amount that the athlete has transacted for the expenses over a certain period. The user must select a month and year that they would like to see the expenses incurred. Once decided, they may click the enter button.

The system will then retrieve the athlete expenses for the selected month and year from the database with the execution of the SQL query. The data will come from the MonthlyExpenses table. Calculation of total costs will be executed through SQL queries. Afterward, it will populate the treeview with the athlete's name and the computed costs for that time period. The user can now see the overall costs for the month below the treeview.

Update Training Plan and Private Coaching Algorithm



Reference Number: 20

System Name: Athlete Management System

On the Main Window, there are 3 buttons on the menu bar, one of which is the update button. Once the update button is clicked, the user may choose from the Training Plan and Private Coaching (TPPC) or the Utility Costs. Once the TPPC button is clicked, it launches a new window. In this window, the user may update the hours of coaching sessions of the athlete as well as the training plan.

If the user wants to update the training plan of the athlete, the user must enter the corresponding TPID of that athlete and the new training plan. Once the update button is clicked, the system retrieves the selected TPID and the associated training plan of the athlete from the database. The system also fetches the cost of the new training plan. Once everything is initialized, the database will be updated with the new training plan and cost in the TrainingPlan table, as well as the MonthlyExpenses table.

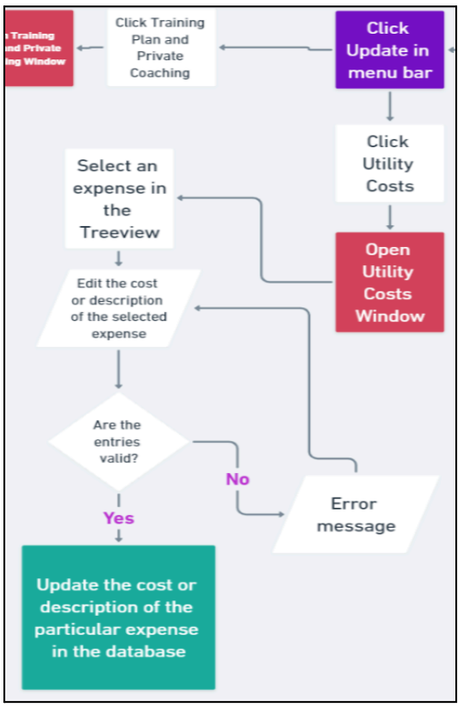
If the user wants to update the number of hours of private coaching sessions of an athlete, they may enter the Coaching ID of that particular coaching session of the athlete, then select the new number of hours. The system will retrieve the entered coaching ID and new hours and then fetch the hourly rate based on the number of hours. Once everything is initialized, the system will update the number of hours of that private coaching session, the cost, and the monthly expenses, a similar process to updating the training plan.

<u>Both of these processes undergo data validations:</u>

- Checks if both fields in the Training Plan are not empty.
- Checks if the user has selected a training plan before updating.

- Checks if the TPID exists in the database.
- Checks if both fields in Private Coaching are not empty.
- Checks if the user has selected the number of hours before updating.
- Checks if the CoachingID exists in the database.

Utility Costs Algorithm



Reference Number: 21

System Name: Athlete Management System

Under the Update menu is the Utility Costs, wherein it opens a new window, Utility Costs Window. When the window is initialized, it populates the Treeview by retrieving the data from the UtilityCosts table from the database. Similar to the Edit Athlete Window, it has the 'tv_MouseRelease' method wherein when the user selects an item in the treeview, it gets the values of the chosen item and then populates the entry widgets. This makes the data editable. Once the user updates the information in the selected item, the user may click the update button. The system will now retrieve the updated cost or description of the expense. The entries will undergo data validation.

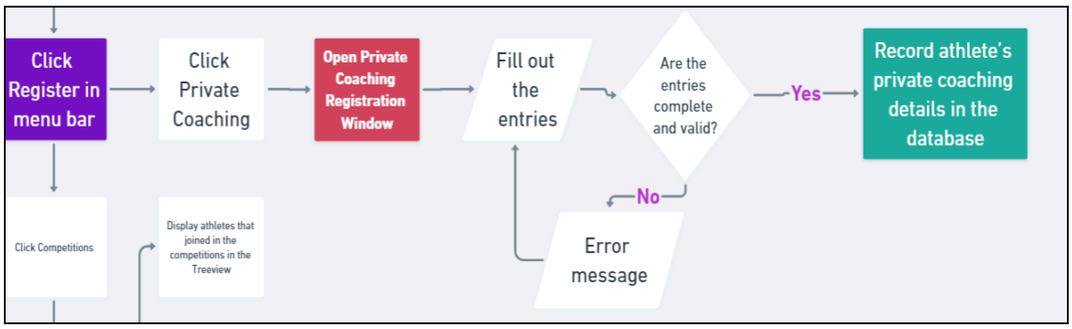
<u>Data validation includes:</u>

- Check if the amount entered does not have an alphabet.
- Check if all the fields are filled out.
- Check if the user selected a row in the Treeview.

If the data is valid, update the treeview with the modified values and execute

the update query on the UtilityCosts table. If the data is invalid, the user will need to try again.

Private Coaching Registration Algorithm



Reference Number: 22

System Name: Athlete Management System

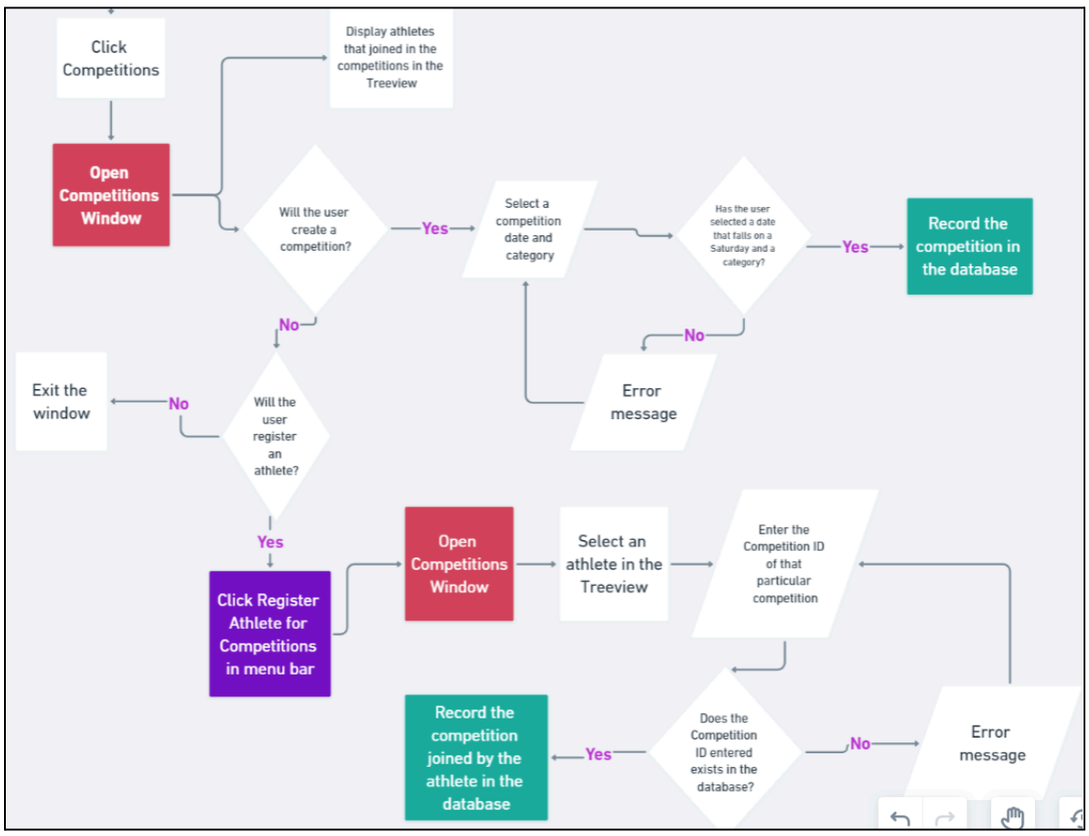
Under the Register menu of the Main Window, a Private Coaching button can be clicked. When the private coaching is clicked, it opens a new window Private Coaching Registration WIndow. During the initialization of the window, the system fetches the list of athletes such as their first name and last name from the database, to be added in the dropdown button. The user will need to fill out the entries to register the athlete for private coaching. Once fields are filled out, the user may submit the form. This activates the register process wherein the entries are retrieved and undergo data validation.

<u>Data validation includes:</u>

- Check if all of the fields are filled out.
- Check if the date of private coaching entered is invalid.

Afterward, the system calculates the cost of the private coaching based on the hours selected. If data is valid, the records are inserted into tables 'CoachingRegistration', 'PrivateCoaching', and 'MonthyExpenses' in the database. The method 'get_athlete_id_from_name' retrieves the Athlete ID of the selected athlete in the dropdown from the database. If data is invalid, it displays an error message, prompting the user to try again.

Competitions and Athlete Competition Registration Algorithm



Reference Number: 23

System Name: Athlete Management System

Under the Register menu of the Main Window, there is also a Competitions button, which directs to a new window, Competitions Window. A Treeview will display the competition information wherein the method 'display_athlete_data' fetches the competition data from the database wherein it involves joining tables of the 'AthleteCompetitions', 'Athletes', and 'Competitions' tables. Under the treeview, the user can create a competition by selecting a competition date and category. The 'get_reg_date' will be triggered once the user clicks the 'Create Competition' button. The system will validate the selected competition date and category.

<u>Data validation includes:</u>

- Check if the user selected a competition date that falls on a second Saturday (as per requirement).

If the data is valid, the competition is successfully created and will be inserted into the table, 'Competitions', in the database. If the data is invalid, the user will need to try again.

In order for the user to register an athlete for the competition, the user must click the 'Register Athlete for Competitions' in the menu bar of the Competition

Window. This opens a new window, Competition Registration. It will also display a treeview of athletes that exclusively have an Intermediate or Elite skill level. The system will fetch the necessary data from the database tables to display on the Treeview. To initialize the competition registration process, the user must first click the desired athlete or row in the Treeview they want to register for a competition. The user must enter the corresponding Competition ID of the competition. Once the register button is clicked, the data selected undergoes data validation.

Data Validation includes:

- Check if the user has selected an athlete, and entered a competition ID.
- Check if the athlete has not registered for that particular competition.
- Check if the category of the competition is compatible/matches with the weight category of the athlete.
- Check if the competition ID/competition exists in the database.

The system retrieves the athlete's weight category as well as the competition category, ensuring that the athlete is compatible with the competition. If the competition is Flyweight, the chosen athlete must also be flyweight. If the data is valid, it fetches the cost of the competition from the UtilityCosts table. Afterward, it inserts the data in the 'AthleteCompetitions' table and the 'MonthlyExpenses' table for the cost of registration. The user goes back to the Competition window and may click the reset button, in order to successfully display the athlete in the Treeview of the upcoming competitions.

User Manual

The step-by-step instructions will help the North Sussex Judo users to use and navigate the program operations properly.

1. **User Registration** - if you don't have an existing administrator or staff account, you may create an account.

Step 1: Run the program or the .exe file, directing to the Home Window.
Step 1: Click the "Create Account" button.
Step 2: Fill out the fields with valid user data.
Step 3: Click the "Create Account" button.
Step 4: Registered an account successfully and redirect to the Login Window.

2. **User Login**- if you have an existing administrator or staff account, you can access the system of the program.

Step 1: Run the program or the .exe file, directing to the Home Window.
Step 2: Click the "Login" button.
Step 3: Fill out the username and password fields with valid credentials.
Step 4: Click the "Login" button.
Step 5: Logged in successfully and redirected to the Main Window.

3. **Remove Athlete** - remove an athlete from the database.

Step 1: Select an athlete in the Treeview from the Main Window.
Step 2: Click the "Remove Athlete" button from the Main Window.
Step 3: Confirm the deletion of athlete data.
Step 4: Deletion of athlete data in the database is successful.

4. **Refresh Main Window Treeview** - refreshes the Treeview data.

Step 1: Click the "Reset" button from the Main Window.
Step 2: Treeview is successfully refreshed and updated.

4. **Athlete Registration** - register an athlete for North Sussex Judo.

Step 1: Click "Register Athlete" from the Main Window.
Step 2: Fill out the fields with valid athlete's data.
Step 3: Click "Register".
Step 4: The athlete is registered successfully.

5. **Edit Athlete's Information** - edit the name, weight, and weight class of the athlete.

Step 1: Click the “Edit Athlete” button from the Main Window
Step 2: Select an athlete from the Treeview.
Step 3: Edit the athlete’s information in the entry widgets as necessary.
Step 4: Click the “Edit Athlete” button.
Step 5: The athlete’s information is updated successfully.

6. **Create a Competition** - create a competition to be joined by athletes.

Step 1: Click the “Register” button in the menu bar of the Main Window.
Step 2: Click “Competitions”.
Step 3: Select a competition date and category. The second Saturday of the month is the only accepted date.
Step 4: Click “Create Competition”
Step 5: Competition created successfully.

7. **Register Athlete for Competition** - have an athlete partake in a competition.

Step 1: Click “Register Athlete for Competition” in the menu bar of the Competitions Window.
Step 2: Select an athlete from the Treeview to compete in the competition.
Step 3: Input the competition ID of that corresponding competition. Ensure that the weight category of the athlete is compatible with the competition category.
Step 4: Click the “Register” button.
Step 5: Registered athlete for competition successfully.
Step 6: Click the “Reset” button from the Competitions Window.
Step 7: The competition information is successfully displayed in the Treeview.

8. **Update the Training Plan** - update the training plan enrolled by the athlete.

Step 1: Click the “Update” button in the menu bar of the Main Window.
Step 2: Click “Training Plan and Private Coaching”.
Step 3: Enter the corresponding TPID of the athlete.
Step 4: Select the new training plan on the dropdown menu.
Step 5: Click the “Update” button.
Step 6: Training plan of the athlete is successfully updated.

8. **Update the Private Coaching** - update the private coaching hours of the athlete.

Step 1: Click the “Update” button in the menu bar of the Main Window.
Step 2: Click “Training Plan and Private Coaching”.
Step 3: Enter the corresponding Coaching ID of the athlete’s private coaching session.
Step 4: Select the new coaching hours on the dropdown menu.
Step 5: Click the “Update” button.
Step 6: Private coaching of the athlete is successfully updated.

9. **Update the Costs of the Expenses -** update the corresponding amount of the training plan, private coaching, and competitions.

Step 1: Click the “Update” button in the menu bar of the Main Window.
Step 2: Click “Utility Costs”.
Step 3: Select an expense in the Treeview.
Step 4: Enter the new value of the amount/cost. You may also edit the description if necessary.
Step 5: Click the “Edit” button.
Step 6: The cost of the expense or description is successfully updated.

10. **View the Athletes Transactions/Monthly Expenses** - view the monthly expenses incurred by the athlete for each month, as well as the overall cost.

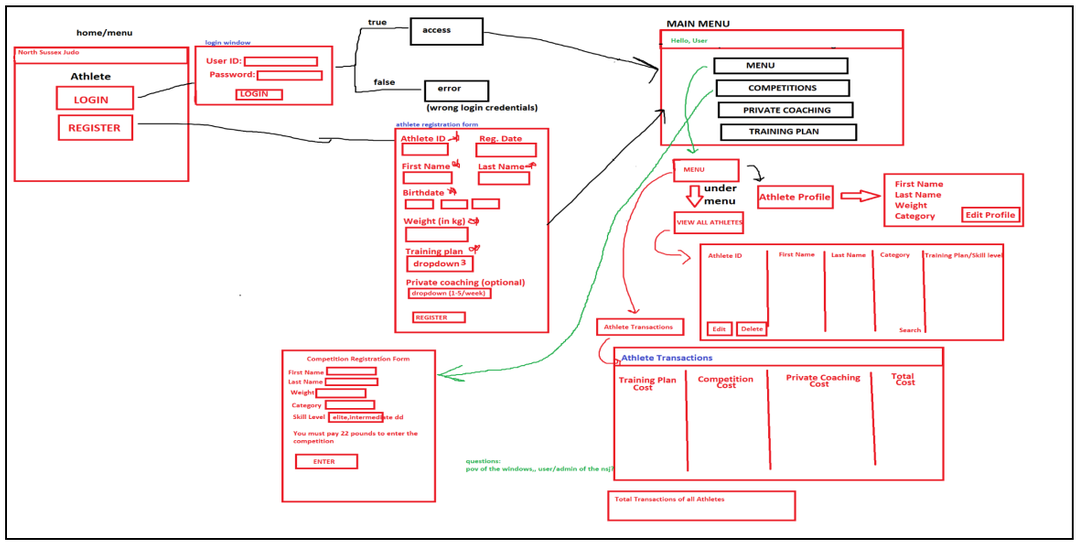
Step 1: Click the “View Athletes Transactions” button from the Main Window.
Step 2: Select a month and year.
Step 3: Click the “Enter” button.
Step 4: The Treeview successfully displays the athletes that transacted on that particular month, as well as the total amount of transactions made for each expense.

Steps on Building an Application

1. **Review and analyse the assignment brief** or the scenario that was given. Keep in mind the main purpose of the program as well as the necessary details such as the training plan, weight category, competition, and private coaching information.

2. **Create the tables and establish relationships** to design the database schema through SQL Server Management Studio, based on the North Sussex Judo requirements.

3. Before coding, **create a mockup or illustration** to have an idea of how the program will look, including the widgets, windows, purpose, and buttons. I created last December my first simple illustration with MS Paint, which helped me create my program and make revisions as needed.



Although it's messy, it greatly helped me visualize the program (GUI) and brainstorm the functions and classes I need to create. The arrows indicate the flow or the output when a button is clicked.

4. **Choose a programming language**, then install an IDE suitable for the chosen language. In this case, Pycharm applies to the Python language. Create a project file namely, North Sussex Judo.

5. **Create a Data Access Layer file.** This manages the connection between the North Sussex Judo database and the Python program. Not only that, but this serves as the foundation for database interactions that are applied to query sections of the program such as the commit, fetch, close, and open database actions. The data access layer is done by creating a class, and methods, importing pyodbc, and establishing connections.

6. **Create modules and compile them** in a Python directory within the project. The modules include:

- __Home_Window.py
- Athlete_Registration_Form.py
- Athlete_Transactions.py
- Competitions.py
- Competition_Registration.py
- Data_Access_Layer.py
- Edit_Athlete.py
- Login_Window.py
- Main_Window.py
- Private_Coaching_Registration.py
- TPPC_Update.py
- User_Registration.py
- Utility_Costs.py

7. **Import the necessary modules or libraries** such as the Tkinter library and data access layer**.** For example, in the Home Window, I imported the modules relating to the Login and Create Account window for the buttons to function.

8. **Layout the widgets or the GUI of the program.** The commonly used widgets from Tkinter are the entry, label, Treeview, combo box, and dropdown.

9. **Implement the logic behind the methods or functions created.** Use SQL commands for the **SQL queries** and ensure that the values and parameters

provided correspond to each other. The columns provided in the queries must also exist in the database schema.

10. **Write comments/documentation** on each module, class, and function for better readability and easily spot errors.

11. **Align the functionalities to the user requirements efficiently.** Ensure that the calculation of costs produces the correct value.

12. When debugging, **make sure to screenshot the error for documentation** and take note of how it was solved.

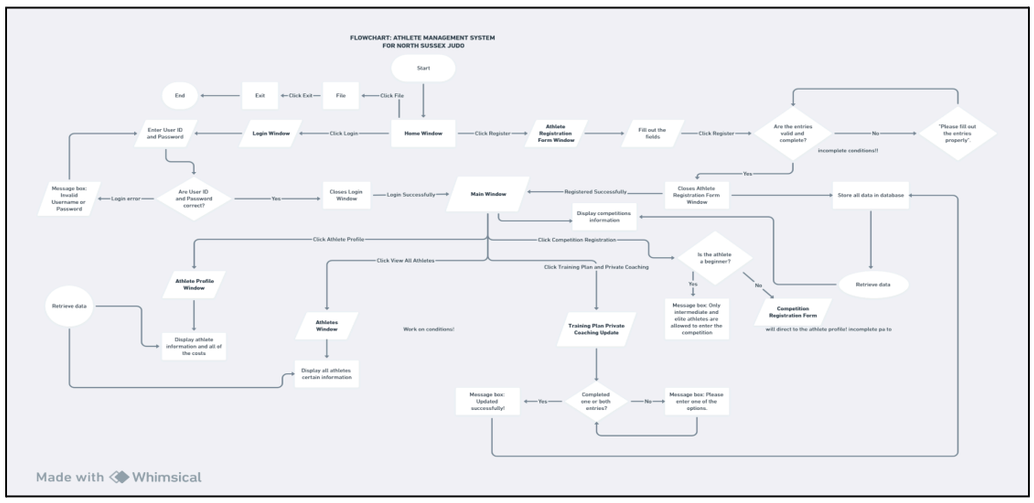
13. **Test the program with various scenarios** and when modifying changes to ensure the functionality. This could be done by clicking on the 'Run' in Pycharm. Check if the data validation database updates and error handling work well.

14. **Ask the professor for feedback**, for such errors, and how to further improve the program.

15. **Optimize the code** and make the program as efficient and straightforward as possible.

16. **Run the whole program** and check if the logic is successfully implemented as well as the GUI.

17. **Draft a flowchart for the program** during the development, to visualize the windows and functionalities completed, ensuring that nothing has been overlooked. This is my very first flowchart and I just continuously made revisions to it.



Relationship between the Algorithm and the Program Code

The relationship between the algorithm and program code is important and interdependent. An algorithm is a step-by-step procedure for solving a specific problem (Programiz, n.d.). But how will the algorithm work if it will not be executed? This is where the program code comes in. In the program code, it is where the algorithm is implemented in a specific programming language like Python.

The program code will instruct the computer to execute the algorithm and perform its tasks in order to solve the problem. This is why they are both relevant and

important to each other. An algorithm can be expressed even in our daily lives, and flowcharts, while the program code is dependent on a programming language and its corresponding syntax and semantics such as C++, Python, Java, etc. (Geeks for Geeks, 2018).

The parts of the algorithm that would remain the same are the operations for the database when it is invoked on different modules:

1. 'dal = DataAccessLayer()' - for a shortcut.
1. 'dal.open()' - opens the database.
2. Outline the SQL queries and conditional statements.
3. 'dal.execute'() - executing the SQL queries.
4. 'dal.fetch_one()' - fetches data.
5. 'dal.commit()' - to commit on the transaction.
6. 'dal.close()' - always placed at the very end of the SQL statements to close the database connection.

Also, the algorithm for the assignment of weight class remains the same in the 'EditAthletes' and 'AthleteRegistration' classes.

```
def get_weight_class(self, weight):
        """Get the weight class based on the provided weight."""
        if 1 <= weight < 73:
            return "Flyweight"
        elif 73 <= weight < 81:
            return "Lightweight"
        elif 81 <= weight < 90:
            return "Light-Middleweight"
        elif 90 <= weight < 100:
            return "Middleweight"
        elif 100 <= weight < 101:
            return "Light-Heavyweight"
        else:
            return "Heavyweight"
```

The algorithm for fetching the inputs from the user also remains the same with the '.get()' method.

```
# Retrieving input values from the form
        reg_date = datetime.now().strftime('%Y-%m-%d')
        athlete_id = self.entry_athlete_id.get()
        birthdate = datetime.strptime(self.entry_birthdate.get(),
                                      '%d/%m/%Y').strftime('%Y-%m-%d')
        first_name = self.entry_first_name.get()
        last_name = self.entry_last_name.get()
        training_plan = self.training_plan_var.get()

        selected_weight = self.entry_weight.get()
```

Commonly used GUI widgets such as 'Athlete ID', 'First Name', and 'Last Name' have the same algorithm applied across the classes.

Moreover, the methods and conditional statements used in some parts of the program for data validation are the same. '.isdigit()' used in 'AthleteRegistration' and 'UtilityCosts' classes. '.isalpha()' is used in the 'User', 'EditAthletes', and 'AthleteRegistration' classes.

Lastly, the method used for validating that all of the entries are filled out uses the 'if not all()' statement, which is evident in most of the classes in the program.

```
        # Validate athlete ID numeric
        if not athlete_id.isdigit():
            messagebox.showerror("Error", "Athlete ID must contain only numeric
                                 characters.")
            return

        # Validate first name and last name
        if not (first_name.isalpha() and last_name.isalpha()):
            messagebox.showerror("Error", "First name and last name should only contain
                                 letters.")
            return

        # Validate complete entries
        if not all((reg_date, athlete_id, birthdate, first_name, last_name, training_plan,
                    selected_weight)):
            messagebox.showerror("Error", "Please fill in all fields.")
            return
```

Many more fundamental algorithms remain the same in the program, in order to remain consistent on the logical structure, validation, and other database interactions. The functionalities of the program are modularized into methods representing the algorithm implemented.

```
def open_login_window():
    """Open the login window."""
    login_win = Tk()
    LoginWindow(login_win)
    login_win.mainloop()
```

It can also be seen in my codebase that I have a '.mainloop()' on my call function of every module so that I can open the secondary windows independently without starting from the home window of my program. Although my program runs well without the '.mainloop()', I still leave it there in case I modify a code and I want to do a quick test.

Meanwhile, the changes that would have to be made are as follows:

1. I have to adhere to the rules of Python syntax.
2. The event handling of the GUI components in the program may particularly differ depending on the functionality implemented.
3. The parameters may also differ for the instantiation of each class such as 'pcr_win', 'home_win' 'login_win', and more.

One of the biggest changes that I needed to make was the point of view of the program. At first, I thought the handlers of the program would be the athletes themselves. Yet, it was the administrators of the North Sussex Judo, that drastically changed my program. Many of the algorithms needed to be modified and even the GUI of the program.

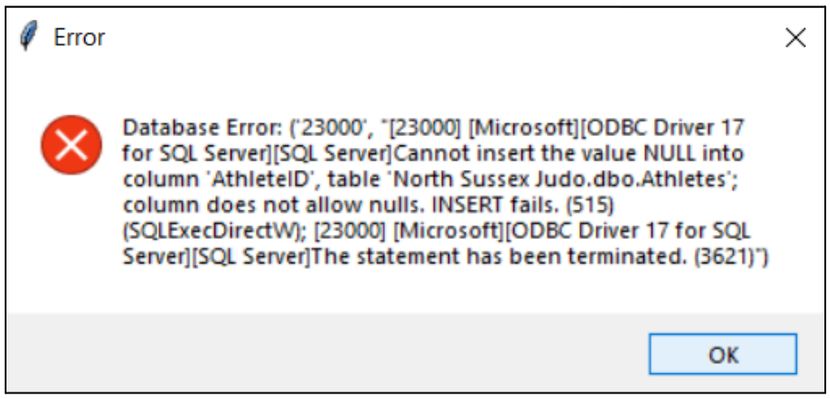
Another major change I made in the program was, at first, the calculation of the costs is hard coded, and if the costs need to change at some point in time in North Sussex Judo, it will be through modifying the codes. This would be a problem in the users' perspective. Therefore, I implemented the utility costs component in the

program, wherein all of the costs will be retrieved from the UtilityCosts table in the database. If the company decides to modify the costs of the expenses, the user may easily go to the Utility Costs window of the program.

Challenges

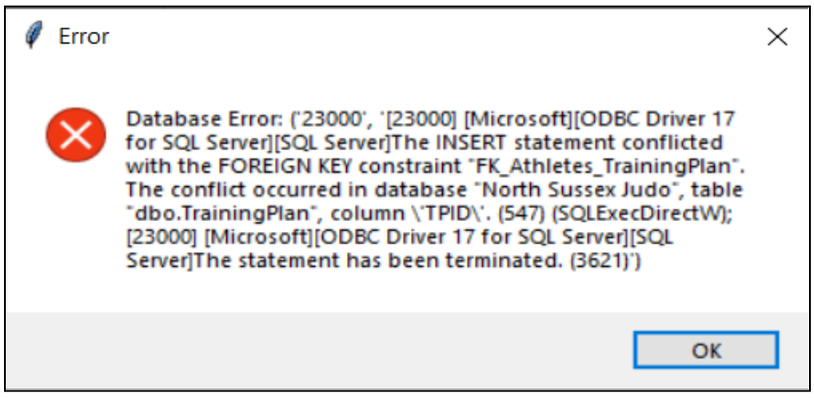
Most of the errors that I encountered in my program were related to executing the algorithm but my program code and database did not align with each other, resulting in database errors. I created except blocks and messages across the modules to handle the error in my execution.

Error #1 - Cannot Insert Null Value



The error message implies that the AthleteID column in the Athletes table cannot insert NULL since I configured the column to not allow null. I realized that the AthleteID shouldn't be null since it is a unique primary key of the Athletes table. Therefore, I figured out that the configuration I did for the AthleteID column is not the problem, but the execution of SQL queries in my program code. I resolved the error by modifying my SQL query for the insertion of data in the Athletes table as well as the order of how the SQL statements will be executed. The position of the SQL queries was a big factor in successfully inserting the data into respective tables in the database.

Error #2 - Conflicted Foreign Key



This error that I encountered is similar to the previous error, wherein there is a problem with the order of execution of the SQL queries. Since I have the TrainingPlanID as the foreign key, in my Athletes table, the appropriate order of insertion is to insert the TPID primary key first from the TrainingPlan table. Thus,

the TrainingPlanID can now reference from the TPID since the value has already been inserted into the TrainingPlan table. This emphasizes the importance of the order of insertion in the program code.

Error #3 - Keyboard Interrupt

```
    main_win.mainloop()
  File "C:\Users\Alfonso\AppData\Local\Programs\Python\Python312\Lib\tkinter\__init__.py", line 1485, in mainloop
    self.tk.mainloop(n)
KeyboardInterrupt
```

This error does not entirely affect my program in any way but it is frustrating that it appears every time I manually terminate the program. I resolved it by modifying this code:

__Home_Window.py

From this:

```
def open_home_window():
    """Open the home window by creating an instance of HomeWindow class."""
    home_win = tk.Tk()
    HomeWindow(home_win)
    home_win.mainloop()


if __name__ == "__main__":
    """Initialize the home window as the main entry point of the program"""
    home_win = tk.Tk()
    home = HomeWindow(home_win)
    home_win.mainloop()
```

To this:

```
def open_home_window():
    """Open the home window by creating an instance of HomeWindow class."""
    home_win = tk.Tk()
    HomeWindow(home_win)


if __name__ == "__main__":
    """Initialize the home window as the main entry point of the program"""
    open_home_window()
```

It is because I discovered that Tkinter can't handle well the 'home_win.mainloop' while it is inside the "__main__" block. But, when I call the function, 'open_home_window()', inside the "__main__" block, the flow of the program is handled by Tkinter smoothly and controlled even if iI manually terminate it. Also, I think my previous code is repetitive since it creates a Tk instance again inside the '__main__' block, even though I can just simply call the open function of the window. This structure does not make a Keyboard Interrupt error anymore.

Error #4 - Circular Import Error

Circular imports happen when one or two files import directly to each other or indirectly which results in an import loop (Stack Abuse, n.d.). This error has happened to me many times in my modules. Since I have a lot of modules with only one class each, it is necessary in most cases to import from module to module for a particular functionality to work successfully. A great example is my Main Window module. My main window module contains the secondary windows, which means I need to import all of those modules in the main window module so that when the particular button is clicked, that secondary window will open.

Main_Window.py

```
from tkinter import *
from tkinter import ttk, messagebox
from tkinter.ttk import *
from Data_Access_Layer import *
from Athlete_Registration_Form import open_register_window
from Edit_Athlete import open_edit_athlete
from Athlete_Transactions import open_athlete_transactions
from Competitions import open_competitions
```

In the old version of my source code, there were too many modules needed to import, thus lead a circular import issue.

```
def open_register_window():
    """Open the registration window"""
    from Athlete_Registration_Form import open_register_window
    open_register_window()


def open_edit_athlete():
    """Open the edit athlete window"""
    from Edit_Athlete import open_edit_athlete
    open_edit_athlete()


def open_athlete_transactions():
    """Open the athlete transactions window"""
    from Athlete_Transactions import open_athlete_transactions
    open_athlete_transactions()


def open_competitions():
    """Open the competitions window"""
    from Competitions import open_competitions
    open_competitions()
```

I figured out the solution by placing the import statements inside functions. This is now more organised and structured since the import statements are not at the modular level or placed at the beginning of the program. Also, the only function of these imported modules is to open their respective windows, which wouldn't impact the functionality when they are placed inside a function.

Error #5 - Logical Error

```
def open_athlete_transactions():
    """Function to create and open the AthleteTransactions window."""
    at_win = Tk()
    AthleteTransactions(at_win)
    at_win.mainloop()
```

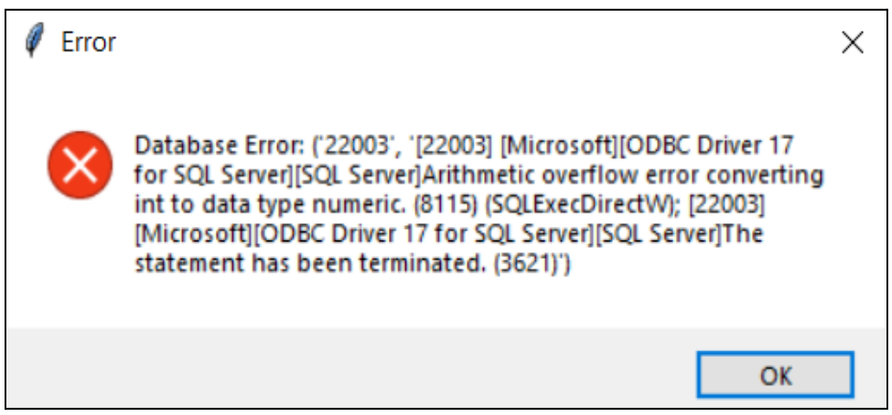
I have an error encountered which is difficult to debug since it does not have an error message, so this means there's something wrong with the structure or logic of my code. This started when I was testing a certain window of my program which is the Athletes Transactions Window. It works pretty well when I call the 'open_athletes_transactions' function in the module itself.

But I started to notice when I ran my whole program starting from the very first window, Home_Window, that the function of the Athlete Transactions Window won't entirely work. It is an odd error and when I tried to search it, no solution was found. I tried analysing and looking for the cause of that occurrence in my codebase, but everything seems to be structured well and in the same manner, so it's confusing why it won't work on one window.

```
def open_athlete_transactions():
    """Function to create and open the AthleteTransactions window."""
    at_win = Toplevel()
    AthleteTransactions(at_win)
```

Days passed and I researched Tkinter and I figured out the problem. Instead of using the 'Tk()' instance in opening the window, I used 'Toplevel()', which resolved the problem. With 'Toplevel()' widgets, it runs only as a child window.

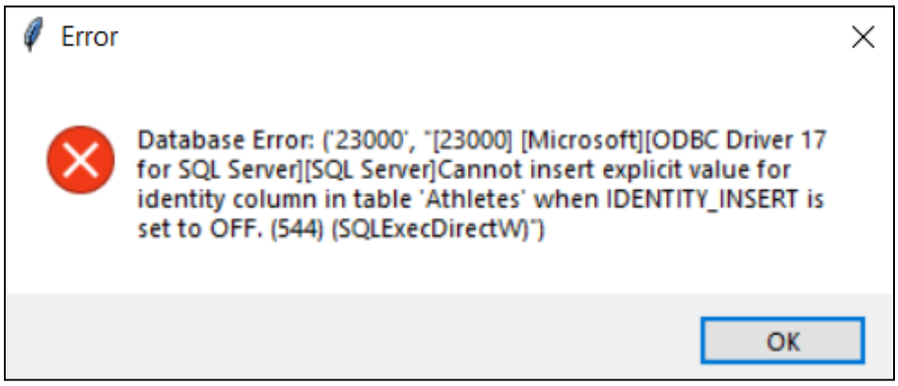
Error #6 - Converting int to Data Type Numeric



This error occurred when the data type on my program code was different from the data type I set on the specific column on my database. For example, if I converted the weight as 'int' in my code, but decimal on the database, the program will be

terminated and this error message will pop up. I resolved this problem by converting the costs and weight to float in my program, in order to accommodate the decimal data type in my database.

Error #7 - IDENTITY_INSERT is Set to OFF



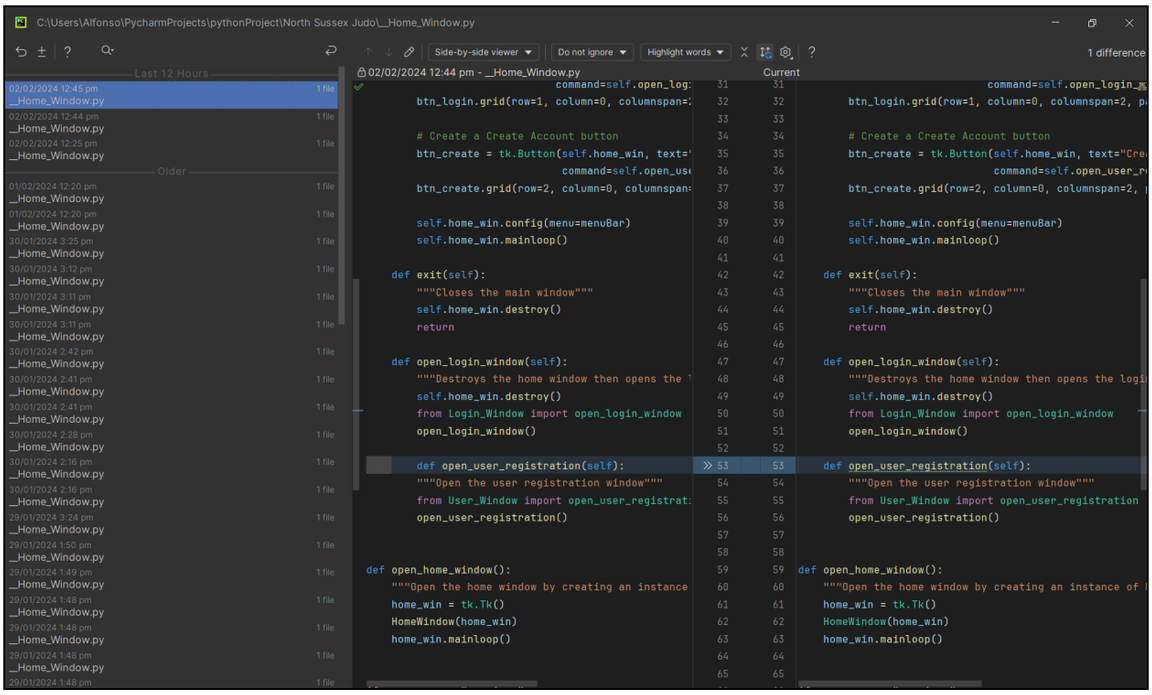
This error occurred when I executed an SQL query that insert a value in the Athlete ID even when the database is configured to generate a value automatically on that column, resulting in a conflict. However, this error was long ago and I left my Athlete ID to manually insert a value rather than auto-generated.

```
training_plan_sql = (
                "INSERT INTO TrainingPlan "
                "([TrainingPlan], Cost) "
                "OUTPUT INSERTED.TPID "
                "VALUES (?, ?);"
            )
            training_plan_id = dal.execute_query(training_plan_sql, training_plan,
                                                 training_plan_cost)[0][0]
            dal.commit()
```

On the other hand, I encountered this problem on some of my tables such as the TrainingPlan, WeightClass, and UtilityCosts, that have auto-generated IDs. I resolved this error by not inserting a value in the TPID or primary key of my training plan table. Instead, I used the 'OUTPUT INSERTED' command in order to retrieve the value that was just inserted in the TPID, and then that value will be referenced for the foreign key, TrainingPlanID, of the Athletes table that is yet to be executed.

Version Control

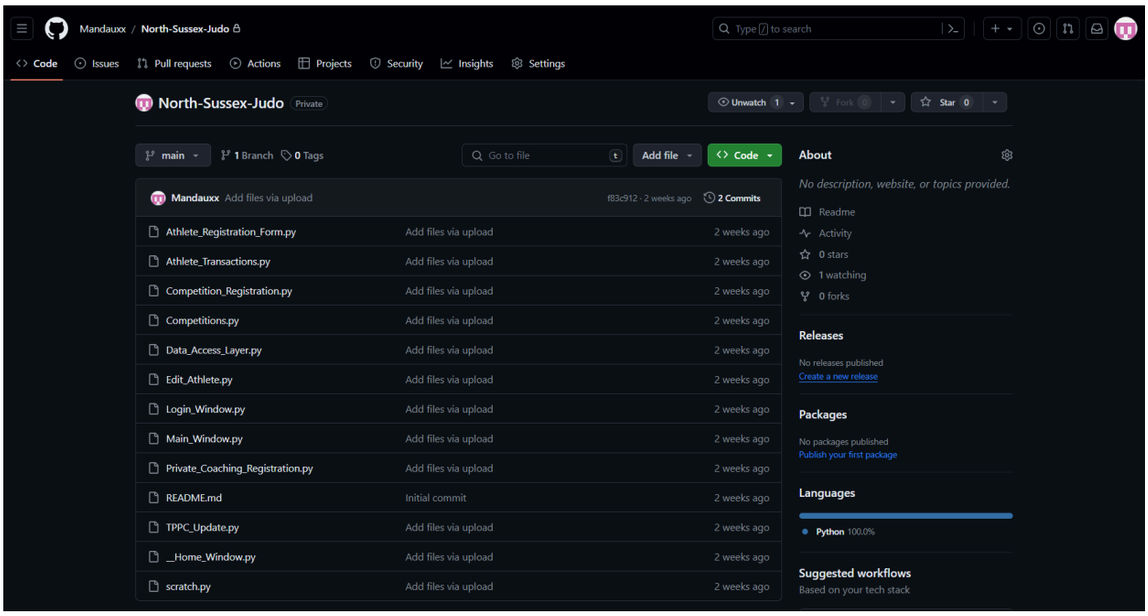
In my IDE, Pycharm, has a feature called Local History wherein it provides a side-by-side view of the changes that I have made in my code. I have greatly used this whenever I make minor revisions and I wanna revert to the previous version. For instance, I revised the algorithm of the code, but it turned out to give undesired results, therefore I just opened up the local history of the module and returned to the previous version easily.

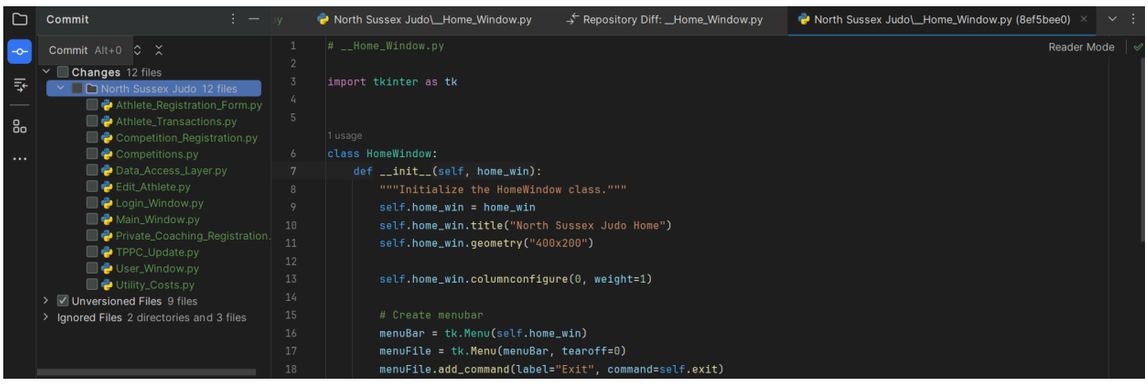


It is seen here that the different versions of my module are at the left side column, providing the date, and time it was modified. On the right side, is the changes that have been made.

Going back to the time I hadn't discovered this feature from Pycharm, I used a 'scratch.py' file wherein I just stored the previous versions of my codes. For example, I changed the GUI of a certain window because I noticed it was inefficient. The code snippet of that GUI is pasted on the scratch Python file so that I can reuse it again and serve as a reference. It acts as a backup for me in case the modified code doesn't work, and also it gives me a way to experiment with different kinds of algorithms.

Furthermore, I uploaded a private remote repository of my North Sussex Judo project in Github in order to serve as a backup and reference for previous versions of my codebase. GitHub is a platform that allows developers and programmers to create, share, and store their code in Git, which is a version control system (Github, 2023). In case my device malfunctions, I still have a repository that I can access and retrieve.





With Pycharm's feature of the git repository, I was able to store my project files in a git and was able to commit and push changes. This is also a version control system that helped me store my project source code modifications.

Coding Standards used in Source Code

Python coding standards are a set of guidelines that make our codes clear and consistent. Python Enhancement Proposal or PEP 8 is a guide that most Python programmers are instructed to follow certain code conventions and structures (van Rossum & Warsaw, 2001). This encourages programmers to write code that is easier to read and understand. Coding standards are necessary for a team since they promote collaboration and communication between developers. As an individual who is just starting out to learn programming, coding standards assisted me in creating code that is maintainable and readable. This is a good practice or habit for me to develop and carry on in order to be a successful programmer.

The following coding standards I adhered to are based on the PEP 8 (2001) website:

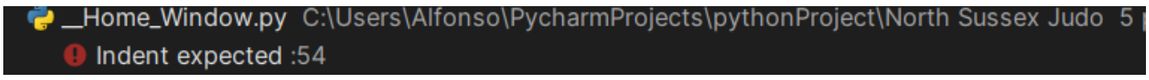
Indentation

```
1 usage
def exit(self):
    """Closes the main window"""
    self.home_win.destroy()
    return

1 usage
def open_login_window(self):
    """Destroys the home window then opens the login window"""
    self.home_win.destroy()
    from Login_Window import open_login_window
    open_login_window()

1 usage
def open_user_registration(self):
    """Open the user registration window"""
    from User_Window import open_user_registration
    open_user_registration()
```

I made sure I only used spaces for consistent alignment of codes and at the right position. In my code, I observed proper indentation of functions as well as inside the functions. This indentation can also be seen in the other modules, which makes it organise and readable.



Aside from that, I won't get confused if my indentation is correct since Pycharm has visual guidelines on the starting line and will notify me if I have an unexpected or expected indentation.

```
        try:
            self.dal.open()  # Open database connection
            insert_query = "INSERT INTO Competitions (Date, Category) VALUES (?, ?)"
            self.dal.execute(insert_query, *param: formatted_date, category)
            self.dal.commit()
            messagebox.showinfo( title: "Success",
                                 message: "Competition date registered successfully!")
            self.reset()
        except Exception as e:
            print(f"Error inserting competition date: {e}")
            messagebox.showerror( title: "Error",
                                  message: f"Error inserting competition date: {e}")
            self.dal.rollback()
        finally:
            self.dal.close()
```

Since I have try-except blocks on my modules, I still managed to organise it and double-check if the codes were working properly. Based on my coding experience, indentation can affect the behavior of the logic behind my code. I made sure that the try-except and if-else statements were properly aligned to produce the expected result.

Blank Lines

I used blank lines to organise the spacing between my codes, enhancing the readability. It is stated in PEP 8 to observe two blank lines at the top level of a class. While a single blank line for separation of methods inside the class. Two blank lines for functions outside class.



In my codebase, I made sure I observed blank lines within my modules. Each of my classes has two blank lines between the import statements and the class.



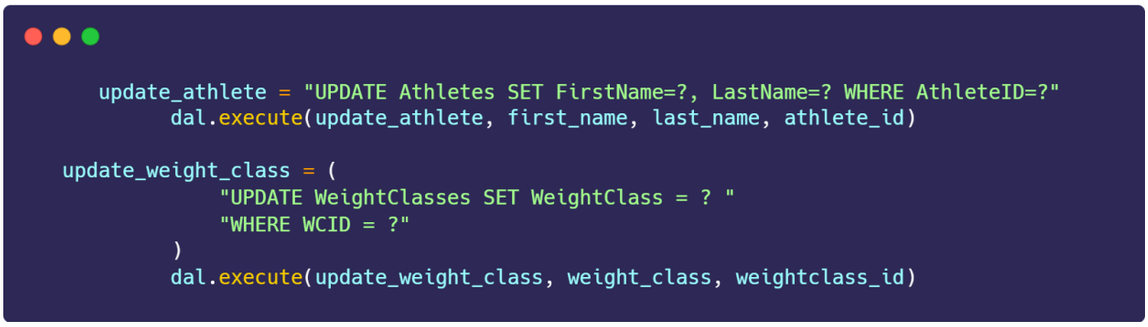
I observed two blank lines for outside functions.



Lastly, one blank line for the methods within the class.

Whitespaces

Whitespaces are important for the readability of the code in terms of spacing or separation of characters and words.



In PEP 8, it is stated that there should be no whitespace before the comma but there should be one space after the comma, which I follow across my codes notably in the execution of queries. It is evident in the code snippet that I properly observed the spacing between commas and variables.

Naming Conventions

One of the most crucial coding standards in Python is the naming convention. Beginner programmers, professionals, or even anyone should be able to understand the code by just a proper naming convention. According to PEP 8, function names, and variable names, should be in lowercase and must be separated with an underscore if it is more than one word. While class names must be capitalized. Lastly, names should be descriptive especially variable names.

```
class EditAthlete:
class AthleteRegistration:
class User:
class CompetitionRegistration:
class DataAccessLayer:
class AthleteTransactions:
```

I followed the naming convention for classes across modules wherein it should be capitalized.

```
        # Athlete ID Widgets
        lbl_athlete_id = Label(self.reg_win, text="Athlete ID: ")
        lbl_athlete_id.grid(row=2, column=0, padx=10, pady=5)
        self.entry_athlete_id = Entry(self.reg_win)
        self.entry_athlete_id.grid(row=2, column=1, padx=10, pady=5)

        # First Name Widgets
        lbl_first_name = Label(self.reg_win, text="First name:")
        lbl_first_name.grid(row=3, column=0, padx=10, pady=5)
        self.entry_first_name = Entry(self.reg_win)
        self.entry_first_name.grid(row=3, column=1, padx=10, pady=5)

        # Last Name Widgets
        lbl_last_name = Label(self.reg_win, text="Last name:")
        lbl_last_name.grid(row=4, column=0, padx=10, pady=5)
        self.entry_last_name = Entry(self.reg_win)
        self.entry_last_name.grid(row=4, column=1, padx=10, pady=5)

        # Birthdate Widgets
        lbl_birthdate = Label(self.reg_win, text="Birthdate (dd/mm/yyyy):")
        lbl_birthdate.grid(row=5, column=0, padx=10, pady=5)
        self.entry_birthdate = Entry(self.reg_win)
        self.entry_birthdate.grid(row=5, column=1, padx=10, pady=5)

        # Weight Widgets
        lbl_weight = Label(self.reg_win, text="Weight (kg):")
        lbl_weight.grid(row=6, column=0, padx=10, pady=5)
        self.entry_weight = Entry(self.reg_win)
        self.entry_weight.grid(row=6, column=1, padx=10, pady=5)
```

I followed a consistent naming convention and descriptive naming in the creation of my widgets in order to be easily understood by any user. Descriptive naming is also applied in my other variables relating to the logic. I properly used underscores in the separation of words.

```
            def display_athlete_data(self):
            def get_weight_class(self, weight):
            def edit_athlete(self):
            def display_athlete_cost(self):
            def register_process(self):
```

It is also evident in my functions that I utilized underscores and descriptive names

Comments

Comments serve as a documentation for providing explanation or context about the code block or lines of code (GeeksforGeeks, 2018). This helps debug in order to understand the context of that issue right away without wasting time. This will also help other programmers or even normal users to better understand the code.

```
        try:
            # Create Data Access Layer instance and open the connection
            dal = DataAccessLayer()
            dal.open()

            # Update Athletes table with the provided data
            update_athlete = "UPDATE Athletes SET FirstName=?, LastName=? WHERE
                             AthleteID=?"
            dal.execute(update_athlete, first_name, last_name, athlete_id)

            # Update WeightClasses table with the new weight value
            update_weight = "UPDATE WeightClasses SET Weight = ? WHERE WCID = ?"
            dal.execute(update_weight, weight, weightclass_id)

            # Update WeightClasses table with weight class based on weight range
            weight_class = self.get_weight_class(weight)
```

It is evident here in class, EditAthlete, that I have multiple comments for each purpose of that line of code. I also added a docstring for each method or function in all of the modules, providing a comprehensive description of the functionality. This makes my codebase easier to modify, navigate, and understand in all aspects.

Standardized Headers for Documentation

```
"""
Module: User_Registration.py
Date of Creation: January 29, 2024
Author: Amanda Ang

Modification History:
- Jan. 29, 2024 - created widgets or the GUI
- Jan. 29, 2024 - added SQL queries
- Jan. 31, 2024 - added data validations
- Feb. 2, 2024 - added documentation

Synopsis:
This module defines the User class, providing a user interface for user registration,
designed for NSJ administrators.
It includes functions to create a new user account, perform data validation, and toggle
password visibility.

Functions:
- __init__(self, user_win): Initializes the User class and creates the user registration
window.
- toggle_password_visibility(self): Toggles the visibility of the password.
- create_account_process(self): Processes user input, validates data, and creates a new
user account.
- open_login_window(self): Opens the login window and destroys the current window.
- open_user_registration(): Function to create and run the user registration window.

Global Variables:
None
"""
```

I also provided documentation at the beginning of every module using docstring. According to Bose (2021), this is part of the coding standards as this provides standardized headers, containing the necessary information for that module.

Testing Document

The testing process is the process that validates whether the application meets the technical requirements, is bug-free and appropriately handles exceptions and various scenarios of user interactions (GeeksforGeeks, 2017). In short, it tests the functionalities of the program created. Also, this is to check if the application has defects and if it does, the program can be further improved and fixed before it is released to the end users.

The type of testing that I will implement is system testing. System testing is a type of software testing that examines the overall functionality of the program and validate if it meets the specified requirements in various scenarios (GeeksforGeeks, 2019). This includes instances of valid and invalid inputs and its respective data validation.

This system testing is under the black box testing technique. It is under black box since it focuses on the real-world user interaction rather than how the program is built (Hamilton, 2019). This type of testing is more centered to users perspective, wherein it highlights the inputs and outputs of the program.

I chose system testing in order to analyse if the functionality of the overall system is working as expected or not, ensuring that it meets the user requirements of North Sussex Judo. For example, in my program, a data entered in one window of the system, will reflect to another window. In my case, the registration forms reflects to the athlete transactions, ensuring accurate data are stored and displayed.

This implies how parts of my program coordinate together in order to achieve expected user experience, making it suitable to apply system testing. System testing will test the behavior of the system such as how multiple components interact, through the record of test cases.

Strengths:

- I will test the whole system's functionalities such as data inputs, and validations.
- I will identify the issues and bugs that may arise in the system in order to enhance reliability.
- I will ensure that the program aligns with the intended purpose and meets the user requirements.
- I will ensure that the error-handling mechanisms are working properly and handle different scenarios.
- I will ensure that all units and components interact correctly together.

Weaknesses:

- I will not cover the several types of system testing such as:

    - Performance testing, Acceptance testing, and Security testing - it is not specified in the project requirements.
    - Usability testing - no feedback will be coming from the actual North Sussex Judo users.
- I may only have a limited test scenarios for real-world situations that may arise for the North Sussex Judo users.
- I have a limited amount of time to optimize the program as much as possible.
- No equipment and software tools that will support or automate my test cases, possibility for human error.

Test Cases

HOME WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	001	1. Click the “Login” button.	Opens the login window.	To test that the login window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	002	1. Click the “Create Account” button.	Opens the user registration window.	To test that the user registration window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	003	1.Click the “File” menu in the menu bar. 2. Click “Exit”	Exits the window and terminates the program.	To test that the program can be terminated or exited from the home window.	Successful	OK	Amanda Ang

USER REGISTRATION WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	004	1.Create an account with valid inputs. -First name: John -Last name: Doe -Username: JohnDoe -Password: johndoe -Email:johndoe@gmail.com 2. Click the “Create Account” button.	Displays user registration success message “Account created successfully.” and redirect to the Login Window.	To verify that the user registration process is successful.	Successful	OK	Amanda Ang

Feb. 3, 2024	005	1. Enter other valid inputs, but enter invalid values on first name or last name. -First name: 123John -Last name: $Doe& -Username: JohnDoe -Password: johndoe -Email:johndoe@gmail.com 2. Click the “Create Account” button.	Displays user registration error message “First name and last name cannot contain special characters or digits.".	To verify the error handling for invalid input on first name or last name.	Successful	OK	Amanda Ang
Feb. 3, 2024	006	1.Enter other valid inputs, but entered invalid email. -First name: John -Last name: Doe -Username: JohnDoe -Password: johndoe -Email: doe 2. Click the “Create Account” button.	Displays user registration error message “Invalid email address.".	To verify the error handling for invalid input of email address.	Successful	OK	Amanda Ang
Feb. 3, 2024	007	1.Enter other valid inputs, but entered an existing username. -First name: Johnny -Last name: Doe -Username: JohnDoe -Password: johnnydoe -Email: johnnydoe@gmail.com 2. Click the “Create Account” button.	Displays user registration error message “Username already exists. Please choose another username".	To verify the error handling for entering an existing username.	Successful	OK	Amanda Ang
Feb. 3, 2024	008	1. No inputs or leave one of the fields empty. -First name: -Last name: -Username: -Password: -Email: 2. Click the “Create Account” button.	Displays user registration error message “Please fill in all fields”.	To verify the error handling for an empty field/s.	Successful	OK	Amanda Ang

LOGIN WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**

Feb. 3, 2024	009	1.Input valid username and password. - Username: JohnDoe - Password: johndoe 2. Click the “Login” button.	Displays login success message "Login successful” and redirects to the Main Window.	To verify that the login process is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	010	1.Input the incorrect username and/or password -Username: john123doe -Password: doe_john! 2. Click the “Login” button.	Displays login error message “Invalid username or password. Please consider creating an account if necessary”.	To verify the error handling for invalid input.	Successful	OK	Amanda Ang
Feb. 3, 2024	011	1.Leave the username and/or password empty. -Username: -Password: 2. Click the “Login” button.	Displays login error message “Please enter both username and password”.	To verify the error handling for an empty field/s.	Successful	OK	Amanda Ang

MAIN WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	012	1. Select an athlete row from the Treeview. 2. Click the “Remove Athlete” button 3. Click “Yes” for confirmation. Click “No” to cancel deletion.	Display a confirmation message “Are you sure you want to delete this athlete?”. If the user clicks “Yes”, it displays a success message “Athlete removed successfully!”. If the user clicks “No”, it cancels the request for	Removes the selected athlete from the database.	Successful	OK	Amanda Ang

			deletion.				
Feb. 3, 2024	013	1. Click the “Reset” button.	Refreshes the athletes’ information displayed in the Treeview.	To update the athlete's information in the Treeview, if there are changes made (e.g. added an athlete, edited athlete’s information)	Successful	OK	Amanda Ang
Feb. 3, 2024	014	1.Click the “Register Athlete” button.	Opens the Athlete Registration Form Window.	To test that the athlete registration window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	015	1.Click the “Edit Athlete” button.	Opens the Edit Athlete Window.	To test that the edit athlete window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	016	1.Click the “View Athletes Transactions” button.	Opens the Athletes Transactions Window.	To test that the athlete transactions window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	017	1.Click the “Register” menu in the menu bar. 2.Click “Competitions”	Opens the Competitions Window.	To test that the competition window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	018	1.Click the “Register” menu in the menu bar. 2.Click the “Private Coaching”	Opens the Private Coaching Registration Window.	To test that the private coaching window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	019	1.Click the “Update” menu in the menu bar. 2.Click the “Training and Private Coaching”	Opens the Update Training Plan and Private Coaching Window.	To test that the update training plan and private coaching window opening is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	020	1.Click the “Update” menu in the menu bar. 2.Click the “Utility Costs”	Opens the Utility Costs Window.	To test that the utility costs window opening is successful.	Successful	OK	Amanda Ang
Feb. 3,	021	1.Click the “File” menu in	Exits the	To test that the	Successful	OK	Amanda

2024		the menu bar. 2. Click “Exit”	window and terminates the program.	program can be terminated or exited from the main window.			Ang

ATHLETE REGISTRATION FORM WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	022	1.Input valid data in all fields. - Athlete ID: 1 - First name: Amanda - Last name: Ang -Birthdate: 07/27/2005 -Weight(kg): 77 -Training plan: Intermediate 2. Click the “Register” button.	Displays athlete registration success message "Data stored successfully!”.	To verify that the athlete registration process is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	023	1.Enter other valid inputs, but entered invalid Athlete ID. - Athlete ID: aaa - First name: Amanda - Last name: Ang -Birthdate: 07/27/2005 -Weight(kg): 77 -Training plan: Intermediate 2. Click the “Register” button.	Displays athlete registration error message “Athlete ID must contain only numeric characters.”.	To verify the error handling for invalid athlete ID.	Successful	OK	Amanda Ang
Feb. 3, 2024	024	1.Enter other valid inputs, but enter invalid first name and/or last name. - Athlete ID: 1 - First name: 13Amanda - Last name: #Ang% -Birthdate: 07/27/2005 -Weight(kg): 77 -Training plan: Intermediate 2. Click the “Register”	Displays the athlete registration error message “First name and last name should only contain letters”.	To verify the error handling for invalid athlete ID.	Successful	OK	Amanda Ang

		button.					
Feb. 3, 2024	025	1.Enter other valid inputs, but entered invalid weight. - Athlete ID: 1 - First name: Amanda - Last name: Ang -Birthdate: 07/27/2005 -Weight(kg): aa -Training plan: Intermediate 2. Click “Register” button.	Displays athlete registration error message “Invalid weight entered”.	To verify the error handling for invalid weight entered.	Successful	OK	Amanda Ang
Feb. 3, 2024	026	1. No inputs or leave one of the fields empty. - Athlete ID: - First name: - Last name: -Birthdate: -Weight(kg): -Training plan: 2. Click the “Register” button.	Displays athlete registration error message “Please fill in all fields”.	To verify the error handling for an empty field/s.	Successful	OK	Amanda Ang

EDIT ATHLETE WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	027	1.Select an athlete row in the Treeview 2. Input valid values for updating the athlete’s information. Original athlete’s information: - First name: Amanda - Last name: Ang -Weight(kg): 77 (Input) Updated athlete’s information: - First name: Amandy - Last name: Ang	Displays success message "Athlete information updated successfully!”.	To verify that the updating of the athlete’s information process is successful.	Successful	OK	Amanda Ang

		- Weight(kg): 80 3. Click the “Edit Athlete” button.					
Feb. 3, 2024	028	1.Select an athlete row in the Treeview 2. Input invalid updated first name or/and last name. Original athlete’s information: - First name: Amanda - Last name: Ang -Weight(kg): 77 (Input) Updated athlete’s information: - First name: $Amandy12 - Last name: Ang33 - Weight(kg): 80 3. Click the “Edit Athlete” button.	Displays error message “First and last names should only contain letters.”.	To verify the error handling for invalid first name or/and last name.	Successful	OK	Amanda Ang
Feb. 3, 2024	029	1.Select an athlete row in the Treeview 2. Input invalid updated weight. Original athlete’s information: - First name: Amanda - Last name: Ang -Weight(kg): 77 (Input) Updated athlete’s information: - First name: Amandy - Last name: Ang - Weight(kg): aa 3. Click the “Edit Athlete” button.	Displays athlete registration error message “Weight should be a numeric value”.	To verify the error handling for invalid weight entered.	Successful	OK	Amanda Ang
Feb. 3, 2024	030	1. Don’t select an athlete row in the Treeview, or either of the fields is missing.	Displays athlete registration error message “Please fill in all fields”.	To verify the error handling for an empty field/s.	Successful	OK	Amanda Ang

		No selection of athlete row: -Athlete ID: -Weight Class ID: First name: - Last name: -Weight(kg): Left other fields empty: -Athlete ID: 1 -Weight Class ID: 1 -First name: - Last name: -Weight(kg): 3. Click the “Edit Athlete” button.					

COMPETITIONS WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	031	1.Select a competition date that falls on a second Saturday of the month. -Competition Date: Feb. 10, 2024 2.Select a competition category. -Category: Lightweight 3. Click the “Create Competition” button.	Displays competition creation success message "Competition added successfully.”.	To verify that the competition creation process is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	032	1.Select a competition date that falls on weekdays, Saturdays (except for the second), and Sundays. -Competition Date: Feb. 8, 2024 2.Select a competition category. -Category: Lightweight 3. Click the “Create Competition” button.	Displays competition creation error message "Competitions are held only on the second Saturday of each month.”.	To verify that the user is not allowed to select a date that does not fall on a second Saturday successfully.	Successful	OK	Amanda Ang

Feb. 3, 2024	033	1. Click the “Reset” button.	Refreshes the competitions’ information displayed in the Treeview.	To update the competitions information in the Treeview, if there are changes made (e.g. registered an athlete for competition)	Successful	OK	Amanda Ang

COMPETITION REGISTRATION WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	034	1.Select an athlete row in the Treeview. -Athlete ID: 1 (Lightweight) 2.Enter a valid competition ID that is compatible with the weight category of the athlete. -Competition ID: 1 (Lightweight) 3.Click the “Register” button	Displays competition registration success message "Athlete registered successfully in the competition.”.	To verify that the competition registration for athletes process is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	035	1.Select an athlete row in the Treeview. -Athlete ID:1 (Lightweight) 2. Re-register an athlete for the same competition. -Competition ID: 1 (Lightweight) 3.Click the “Register” button	Displays competition registration error message "Athlete is already registered for this competition”.	To verify that the athlete can’t re-register for the same competition.	Successful	OK	Amanda Ang
Feb. 3, 2024	036	1.Select an athlete row in the Treeview. -Athlete ID:1 (Lightweight)	Displays competition registration error message "Competition ID	To verify that the competition attempting to join by the	Successful	OK	Amanda Ang

		2. Register an athlete for a non-existing competition. -Competition ID: 999 3.Click the “Register” button	not found.”.	athlete does not exist.			
Feb. 3, 2024	037	1. Don’t select an athlete row in the Treeview. 2. Or select an athlete row, but do not enter a competition ID. 3. Click the “Register” button.	Displays competition registration error message "Please fill in all the competition registration fields.”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb 3, 2024	038	1.Select a different athlete row in the Treeview. -Athlete ID: 2 (Lightweight) 2.Register the athlete for the same competition as the other athlete. -Competition ID: 1 (Lightweight) 3. Click the “Register” button.	Displays competition registration success message "Athlete registered successfully in the competition.”.	To verify that two athletes can register for the same competition successfully.	Successful	OK	Amanda Ang
Feb. 3, 2024	039	1.Select an athlete row in the Treeview. -Athlete ID:1 (Lightweight) 2. Register an athlete to an incompatible weight category competition. -Competition ID: 3 (Middleweight) 3.Click the “Register” button	Displays competition registration error message "Invalid weight category for the competition.”.	To verify that the athlete can’t partake in a competition that is not compatible with their weight category.	Successful	OK	Amanda Ang

PRIVATE COACHING REGISTRATION WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	040	1.Enter valid inputs. -Date of Private Coaching:	Displays private coaching	To verify that the private	Successful	OK	Amanda Ang

		10/02/2024 -Athlete: Amanda Ang -Number of Hours: 2 hrs 2.Click the “Register” button	registration success message "Coaching registration and private coaching updated successfully!”.	coaching registration for athletes process is successful.			
Feb. 3, 2024	041	1.Enter valid inputs, but date of private coaching is invalid value. -Date of Private Coaching: aa -Athlete: Amanda Ang -Number of Hours: 2 hrs 2.Click the “Register” button	Displays private coaching registration error message "Error updating coaching registration:...”.	To verify the error handling for an invalid value for date of private coaching.	Successful	OK	Amanda Ang
Feb. 3, 2024	042	1.Leave all or one of the fields empty. -Date of Private Coaching: -Athlete: -Number of Hours: 2.Click the “Register” button	Displays private coaching registration success message "Please fill in all private coaching fields!”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang

UPDATE TRAINING PLAN AND PRIVATE COACHING WINDOW							
UPDATE TRAINING PLAN							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	043	1. Enter the corresponding TPID of the athlete. -TPID: 1 2.Select the new training plan -Training Plan: Elite 3.Click the “Update” button	Displays updation of the training plan success message "Training plan updated successfully.”.	To verify the process of updating the training plan of an athlete is successful.	Successful	OK	Amanda Ang

Feb. 3, 2024	044	1. Enter the corresponding TPID of the athlete. -TPID: 1 2. Do not select a training plan. -Training Plan: 3.Click the “Update” button	Displays updation of the training plan error message "Please enter both TPID and select a training plan”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	045	1. Do not enter a TPID. -TPID: 2. Select a new training plan. -Training Plan: Elite 3.Click the “Update” button	Displays updation of the training plan error message "Please enter both TPID and select a training plan”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	046	1.Leave TPID and training plan fields empty. - TPID: -Training plan: 2. Click the “Update” button.	Displays updation of the training plan error message "Please enter both TPID and select a training plan”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	047	1. Enter a non-existing TPID. -TPID: 999 2.Select the new training plan -Training Plan: Elite 3.Click the “Update” button	Displays the updation of the training plan error message "Selected TPID not found in the database”.	To verify the error handling for a non-existing TPID entered.	Successful	OK	Amanda Ang
Feb. 3, 2024	048	1. Enter an invalid TPID. -TPID: ads 2.Select the new training plan -Training Plan: Elite 3.Click the “Update” button	Displays the updation of the training plan error message "Error updating training plan:...”.	To verify the error handling for an invalid TPID entered.	Successful	OK	Amanda Ang
UPDATE PRIVATE COACHING							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**

Feb. 3, 2024	049	1. Enter the corresponding Coaching ID of the private coaching session of the athlete. -Coaching ID: 1 2.Select the new number of hours for that coaching session. -Private Coaching: 5 hr/s 3.Click the “Update” button	Displays updation of the private coaching success message "Private coaching hours updated successfully.”.	To verify the process of updating the private coaching hours of an athlete is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	050	1. Enter the corresponding Coaching ID of the private coaching session of the athlete. -Coaching ID: 1 2. Do not select new private coaching hours. -Private Coaching: 3.Click the “Update” button	Displays updation of the training plan error message "Please enter both TPID and select a training plan”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	051	1. Do not enter a Coaching ID. -Coaching ID: 2. Select the new number of hours. -Private Coaching: 2 hr/s 3.Click the “Update” button	Displays updation of the training plan error message "Please enter both TPID and select a training plan”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	052	1.Leave Coaching ID and private coaching fields empty. -Coaching ID: -Private Coaching: 2. Click the “Update” button.	Displays updation of the training plan error message "Please enter both Coaching ID and number of hours”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	053	1. Enter a non-existing Coaching ID. -Coaching ID: 999 2.Select the new private	Displays the updation of the training plan error message "Selected	To verify the error handling for a non-existing Coaching ID	Successful	OK	Amanda Ang

		coaching hours. -Private Coaching: 2 hr/s 3.Click the “Update” button	Coaching ID not found in the database”.	entered.			
Feb. 3, 2024	054	1. Enter an invalid Coaching ID. -Coaching ID: ads 2.Select the new private coaching hours. -Private Coaching: 2 hr/s 3.Click the “Update” button	Displays the updation of the training plan error message "Error updating private coaching:...”.	To verify the error handling for an invalid Coaching ID entered.	Successful	OK	Amanda Ang

UTILITY COSTS WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3, 2024	055	1. Select an expense row in the Treeview. Original expense’s information: -Cost ID: 1 -Expense Type: Beginner -Amount:100.00 -Description: Training Plan 2. Input valid values for updating the expense information. (Input) Updated expense’s information: -Cost ID: 1 -Expense Type: Beginner -Amount:120.00 -Description: Starting Training Plan 3. Click the “Edit” button.	Displays success message "Data updated successfully!”.	To verify that the updating of the expense’s information process is successful.	Successful	OK	Amanda Ang
Feb. 3, 2024	056	1. Select an expense row in the Treeview. Original expense’s information: -Cost ID: 1 -Expense Type: Beginner -Amount:100.00	Displays error message “Please fill out all the fields”.	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang

		-Description: Training Plan 2. Leave the description and/or amount empty. Original expense's information: -Cost ID: 1 -Expense Type: Beginner -Amount: -Description: 3. Click the "Edit" button.					
Feb. 3, 2024	057	1. Do not select an expense row in the Treeview -Cost ID: -Expense Type: -Amount: -Description: 2. Click the "Edit" button.	Displays error message "Please select a row in the Treeview".	To verify the error handling for an empty or missing field/s.	Successful	OK	Amanda Ang
Feb. 3, 2024	058	1. Select an expense row in the Treeview. Original expense's information: -Cost ID: 1 -Expense Type: Beginner -Amount:100.00 -Description: Training Plan 2. Input valid values for updating the expense information. (Input) Updated expense's information: -Cost ID: 1 -Expense Type: Beginner -Amount: abc -Description: Starting Training Plan 3. Click the "Edit" button.	Displays error message "Amount must be a valid number".	To verify the error handling for an invalid value of the amount entered.	Successful	OK	Amanda Ang
Feb. 3, 2024	059	1. Select an expense row in the Treeview. Original expense's information: -Cost ID: 1 -Expense Type: Beginner	Updates the amount/cost for the particular expense, and successfully integrates it in	To verify the integration and application of the updated cost in the	Successful	OK	Amanda Ang

		-Amount:100.00 -Description: Training Plan 2. Input valid values for updating the expense information. (Input) Updated expense's information: -Cost ID: 1 -Expense Type: Beginner -Amount: 150.00 -Description: Starting Training Plan 3. Click the "Edit" button. 4.Exit Utility Costs window and return to the Main Window 5.Click the "Register Athlete" button. 6. Fill out the form, and choose the Beginner training plan to try out the updated cost. 7. Click the "Register" button. 8. Check the database. Check the TrainingPlan table wherein it should record the updated cost, "150.00". 9. Or you may go to the Athlete Transactions window to check if the Training Plan Cost incurred by the athlete displayed is correct.	the program.	financial activities.			

ATHLETE TRANSACTIONS WINDOW							
Date	**Bug ID**	**Steps to Reproduce**	**Expected Result**	**Purpose**	**Actual Result**	**Status**	**Developer Assigned**
Feb. 3,	060	1. Select a month.	Displays all of	To verify the	Successful	OK	Amanda

2024		-Month: February 2. Select a year. -Year: 2024 2. Click the “Enter” button	the athletes that made transactions for that particular month, as well as the accumulated costs, total cost for each athlete, and overall cost for the month.	process generating reports or itemized list for monthly expenses, with correct calculations.			Ang

Before I did the testing process, I converted the Python script into an executable file (.exe), where I tested the whole system to ensure that the executable file worked seamlessly as running it in Pycharm. My test case tabulation includes the date, bug ID, steps to reproduce, expected result, purpose, actual result, status, and developer assigned.

- Date - the date I performed the test case.
- Bug ID - unique identifier for each test case.
- Steps to Reproduce - an outline of the steps to reproduce that certain output.
- Expected result - the expected outcome after the execution of the test case.
- Actual result - if the produced result met the expected result.
- Status - to indicate the status of the test case.
- Developer Assigned - the person who is in charge of executing the test case.

Based on my test cases, I could say my program is well-crafted for a beginner, given that this is my first programming and database project. This is The CRUD operations and connection in the database worked properly as expected, as per minimum requirement. The system is straightforward and responsive, but I can’t confirm the overall satisfaction for the end user since I didn’t include usability testing.

I encountered defects in my test cases, and most of these relate to the appropriate error message box when certain data input is entered. This means I should modify the source code and ensure that it displays the correct error message, which will undoubtedly enhance the user experience. Following that, I fixed all of the unexpected errors and outputs for each defect I faced in my test cases.

The testing process can be further improved by:

1. Automated testing. This means that a platform or software tool will streamline the testing process, reducing the potential for human error. This is more efficient since automated testing takes less time, especially in repetitive tasks (Smartbear, 2019).
2. By incorporating performance, security, and usability testing, it can improve the overall user experience of the program, improving the reliability of the program for future use (GeeksforGeeks, 2019).

3. Feedback from professional testers or developers. This is important since I can seek constructive feedback from experienced people and provide insights on how I can further improve the test case. This will also help me with unforeseen or overlooked gaps in my program.
4. A larger scope of scenarios such as diverse inputs and situations. This ensures that the program is robust and can handle different scenarios, especially in the real world. Unexpected cases or interactions from users are important to consider, making the program more realistic to use.

During the testing process, it greatly helped me identify the bugs I needed to address in my program, especially in my data validation. The testing process demonstrated the strengths and weaknesses of my program and helped me figure out the areas for improvement. Overall, the testing process was an effective tool to test the quality and functionality of my program.

Evaluation Report

Algorithm Evaluation

The algorithm I designed for North Sussex Judo was implemented through the use of Python programming language and Pycharm as my IDE. The logical algorithm that I planned from the development, is transformed into an executable and user-friendly GUI application. This project helped me understand how to properly use Python syntax as well as to implement the Tkinter library.

In order to create a user-friendly environment for my application, I implemented appropriate and concise data validations such as empty fields, numeric values, and more. It is evident in my test case that the algorithm I implemented works properly.

My algorithm includes registering athletes and handling different activities such as registering them for competitions, and private coaching. This fulfills the athlete management system that is requested by North Sussex Judo. My algorithm includes creating accounts for users and logging in for verification of user identity. On the other hand, this fulfills the user management system.

The algorithm showcased the use of SQL queries in order to connect to the database and execute different queries to store data.

With the continuous increase of data sets of athletes and North Sussex Judo users, my algorithm can handle this load but will need maintenance and re-modify to more efficient data structures. This is a weakness of my algorithm, and it may also affect the performance such as the loading time of the application.

However, maintaining and understanding the algorithm of my application will not be a big issue since the different operations and windows are separated into classes within each module. Comments and documentation are also present on each module. So, even if other programmers try to understand or alternate the algorithm of my application, it wouldn’t be such a difficult task.

North Sussex Judo user feedback is highly recommended to improve the algorithm of the application.

Relationship between Algorithms and Source Code Evaluation

The relationship between the algorithm and source code is important since the source code implements the solution for the specific problem. The algorithm is the step-by-step procedure that serves as a blueprint, while the program code translates these into steps that a computer can understand. This allows the problem to be solved, particularly relating to North Sussex Judo requirements. This relationship has played a crucial role in my development process of the application.

With the use of SQL queries, methods, import statements, and the three programming paradigms, I was able to convert the algorithm steps into code.

In my application development, I encountered some challenges in terms of converting the algorithm into code.

For example, in my initial development of athlete transactions, I had a lot of revisions on the GUI of the Athlete Transactions window since I was not sure what's the most effective and user-friendly way to view monthly expenses. I'm also not sure how I would execute properly the calculations. I overcame these problems by analysing better the North Sussex Judo goal and switching my perspective to the end-user of the application, which led me to consider their needs in designing this window. Moreover, expanding my knowledge of creating complex SQL queries has contributed to implementing this algorithm. This led me to produce the desired result of the monthly expenses.

The use of flowcharts helped me visualize the algorithm that will be implemented in the codebase.

IDE Evaluation

Features Used:

Pycharm as my IDE for the development of the North Sussex Judo program was very helpful and efficient. It has a lot of features it offers such as debugging, code editing, testing, code completion, and more, which really has sped up my development process. The development process will take more days and hours without these tools.

Pycharm also trains me to observe PEP 8 or code style formatting, which is good practice for a beginner programmer. It has a user-friendly and comfortable interface to work on, offering debugging tools for an effective resolution of bugs. Tools such as breakpoints, debugging console, and stepover.

Alternative Considered:

I considered Visual Studio as my IDE in my initial development process, but it wasn't my preference because personally it was cluttered and there were a lot of buttons, making it overwhelming compared to Pycharm's minimalist and clean interface.

Overall Experience:

If Pycharm was not used on my coding journey, it would be more difficult and less productive to analyse the issues that are coming up in my program, which will delay completing this project. Pycharm is user-friendly and I didn't have a difficult time getting used to this IDE, since the tools and functionalities are well-organized.

My codebase will be less readable and inconsistent without Pycharm since this IDE has dedicated tools that assisted me throughout my coding journey. This enhances the quality of my codes and overall user experience, given the comprehensive features, and user-friendly interface, which led me to create an efficient application.

Coding Standards Evaluation

The coding standards, following the PEP 8, were properly observed in my codebase. This includes indentation, blank lines, whitespaces, naming conventions, comments, and a standardized header for documentation. Following these coding standards enhanced the quality of my codes exceptionally since it provide context to different logic or functions and makes the codebase look cleaner.

Coding standards are necessary since they make my code more manageable, structured, and readable, which is a great practice for succeeding projects that I will create and future collaboration with other programmers.

According to Bose (2021), coding standards adhere to consistent and unified styles across the source, ensuring that it won't be difficult to understand and maintain, especially within a collaborative team environment.

Database Maintenance

The database maintenance checklist I listed is supported by the article, Lynch (2023).

Database maintenance checklist:

- ☐ Regularly backup the data in the database, especially in unexpected system failure, accidental data loss, or deletions.
    - Athletes' information and financial transactions are the most crucial data stored in the database, which is why backing it up ensures the continuity of the company, prevents the risk of data loss, and will be valuable for trend or historical analysis. In case a disaster happens, it is securely backed up.
- ☐ Perform defragmentation wherein it reorganizes the data within a database in order to speed up the system.

    - For example, if the Athletes table has fragmented for some time, the maintenance team may move or reorganise the data to a new storage location, which will resolve the fragmentation issue.
- ☐ Regularly check the security of the database and make sure the protection is active and updated.
    - Keep the database management system up to date and the team must regularly perform audits to review weaknesses or security breaches within the database.
- ☐ Check for patches and upgrades to keep up with the latest features and security protocols in the database.
    - A strong authentication mechanism must be implemented in the database management system. Keep documentation of the patches and take advantage of the latest upgrades for security features.
- ☐ Indexing can be implemented to optimize the search time or performance in database queries.
    - As the athlete records grow bigger, indexing is highly recommended to find the relevant data easily, instead of scanning every table in the database.
- ☐ Old data in the database can be archived, if no longer relevant to enhance the performance of the database
    - Archive records that are not needed for regular operations in the database. This frees up space and reduces the time to process data or execute queries. According to Marijan (2021), cloud storage can be used such as Microsoft Azure and Amazon Web Service (AWS).
- ☐ Use descriptive titles for the descriptions and titles in order to easily navigate the information somewhere in the database.
    - Don’t use vague titles in the database since it causes ambiguity. Descriptive titles speed up the search as well as provide more clarity and context behind it.
- ☐ Store the data in a single file, it lessens time on looking over the data files.
    - This makes the file more portable and easily shareable to other systems or platforms of North Sussex Judo. It also reduces the overhead of managing several files, simplifying the directories, and locating data.

Database maintenance will help the database run faster and enhance user experience, especially in important real-life operations wherein businesses heavily rely on databases (Odogwu, 2022).

In North Sussex Judo company, the technical team can perform regular maintenance in order to minimize downtime and ensure the database is readily available. When the database is readily available, it is accessible to analyse data and create various reports in order to make informed decisions, which will contribute to the company’s efficiency and success.

Database Design Evaluation

Normalization reduces the chances of data error and redundancy, consumes less disk space, increases performance, and simplifies data management (GeeksforGeeks, 2021). Without normalisation, problems will arise such as inserting, and updating anomalies, and data inconsistencies. A common challenge in real-world scenarios of database systems includes scalability, since increasing data volumes and users are happening each day, suggesting the need for performance and optimisation techniques (Nexus, 2019).

The database design that I created for North Sussex Judo is well-structured and applies normalization by analysing the flaws through the diagram and making changes to my tables as needed during my development process. This makes my database design more flexible than my previous versions of denormalised databases. My database design established necessary tables that are relevant to fulfill the North Sussex Judo requirements. I ensure the appropriate foreign and primary keys are successfully established between tables.

Although, I can't guarantee that my database design is properly normalised for scalability and performance purposes. In the future, there will be continuously growing athlete records, suggesting the need for modifications. Thus, this is a potential weakness of my database design, its robustness, and scalability, since I have not performed any performance testing or optimisation. These modifications may even affect the existing data in the database.

In database management systems, it is crucial to track financial transactions and generate reports, and then analyse the data with the use of advanced SQL queries in order to make informed decisions (LearnSQL.com, 2021). This highlights the relevance of DBMS in ensuring the accuracy of financial reports in different industries.

In my database design, the North Sussex Judo administrators can effectively track and manage athletes' information, and their transacted amount of expenses such as the training plans, competitions, and private coaching, which meets the requirements of the project. Individual transactions can be seen on the MonthlyExpenses table which plays a crucial role in monitoring the financial activities of athletes, providing also an itemised list of all the costs incurred for the month. In this way, financial reports can be generated in case the North Sussex Judo company needs to.

Registration dates may contribute to providing critical insights in terms of tracking data and audit trails. It also serves as a documentation or log of actions especially in inserting new data or making changes in the database (Smartsheet, 2019). In my database design, the integration of the registration dates in multiple tables is useful for monitoring the date that the transaction was made and may serve as a reference for tracking the date the athlete entered North Sussex Judo. My database design can be further improved or integrated in the future by implementing audit trails for documentation and analysis.

Login authentication is a barrier in the system wherein it ensures that only authorized users can access the program or resources. In DBMS, this provides confidentiality of the sensitive information stored in the system, avoiding the risk of exploitation cases (loginradius, n.d.). The Users table provides a layer of authentication for North Sussex Judo users, which fulfills the security requirement of the program, but it won't be resilient enough to handle attacks or vulnerabilities. My program security can be further solidified by enforcing better security measures and multi-factor authentication in the future, such as two-step verification (Microsoft, 2022), in order to protect the users' and athletes' data.

The database can be further enhanced by implementing database maintenance since it is important for the longevity of the database and to be adaptable in all scenarios. Also, as mentioned in the maintenance, performing indexing techniques helps accommodate more athletes and users at some point in time. These areas of improvement will contribute to the efficiency of the North Sussex Judo database. End-user feedback, as I mentioned, is important in order to understand the real-world problems. With this, I can draw solutions on how to improve the user experience of the program

Modify the table columns to avoid data redundancy. For example, in my Training Plan and Utility Costs tables, the costs are redundant. Applying normalisation principles frees up storage space and avoids discrepancies (GeeksforGeeks, 2021). Lastly, data validation of my program should be optimized and robust in order to make the inserted data consistent and clean. These are the areas of improvement to take into consideration in order to achieve a secure, and streamlined database.

Application Video Demonstration

Video demo file name: 01 - 04 - ANG, AMANDA - DEMO

Google drive link for the program: 01-04 - ANG, AMANDA - PROGRAM - Google Drive

Note/Correction from the Demo:
(Since revisions were made on some data validations after I made the video)

- Revised data validation for athlete registration window: if any or all of the fields are left empty in the form, the message box: "Please fill in all fields" will appear. Not specifically if last name or first name are left empty.

- Revised Data validation for private coaching registration window: if the birthdate is left empty while the other fields have been filled, the message box: "Please fill in all fields" will still appear. So, if any or all of the fields are left empty, the same message box will appear.

- Revised data validation for update training plan and private coaching window: only modified the grammar of the error messages.

Appendix (if applicable)

```sql
SELECT * FROM Athletes
SELECT * FROM WeightClasses
SELECT * FROM TrainingPlan
SELECT * FROM CoachingRegistration
SELECT * FROM PrivateCoaching
SELECT * FROM AthleteCompetitions
SELECT * FROM Competitions
SELECT * FROM MonthlyExpenses
SELECT * FROM UtilityCosts
Select * FROM Users

delete from Competitions
delete from athletecompetitions
DELETE FROM MonthlyExpenses
DELETE FROM CoachingRegistration
DELETE FROM PrivateCoaching
DELETE FROM TrainingPlan;
DELETE FROM Athletes
Delete FRom WeightClasses

Delete From Users
Delete from UtilityCosts

--For Resetting Identity Specification
DBCC CHECKIDENT ('AthleteCompetitions', RESEED, 0);
DBCC CHECKIDENT ('Competitions', RESEED, 0);
DBCC CHECKIDENT ('MonthlyExpenses', RESEED, 0);
DBCC CHECKIDENT ('WeightClasses', RESEED, 0);
DBCC CHECKIDENT ('TrainingPlan', RESEED, 0);
DBCC CHECKIDENT ('CoachingRegistration', RESEED, 0);
DBCC CHECKIDENT ('PrivateCoaching', RESEED, 0);

DBCC CHECKIDENT ('Users', RESEED, 0);
DBCC CHECKIDENT ('UtilityCosts', RESEED, 0);

--INSERT INITIAL COSTS FOR THE FOLLOWING EXPENSES IN UTILITYCOSTS TABLE
INSERT INTO UtilityCosts (Expense, Description, Amount)
VALUES ('Beginner', 'Training Plan', 100.00);

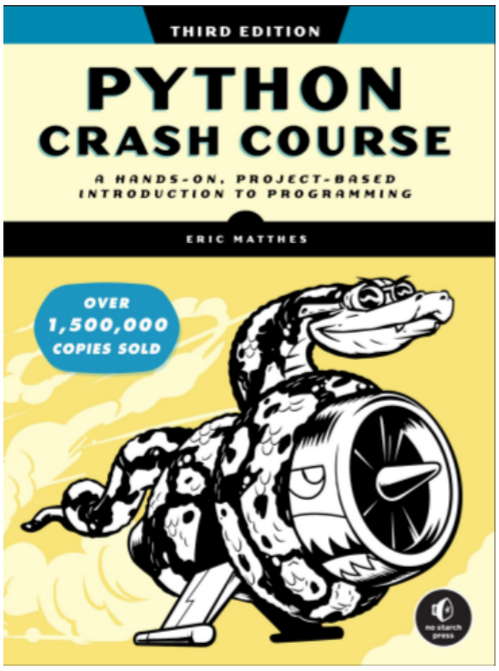
-- Inserting Training Plan - Intermediate
INSERT INTO UtilityCosts (Expense, Description, Amount)
VALUES ('Intermediate', 'Training Plan', 120.00);

-- Inserting Training Plan - Elite
INSERT INTO UtilityCosts (Expense, Description, Amount)
VALUES ('Elite', 'Training Plan', 140.00);

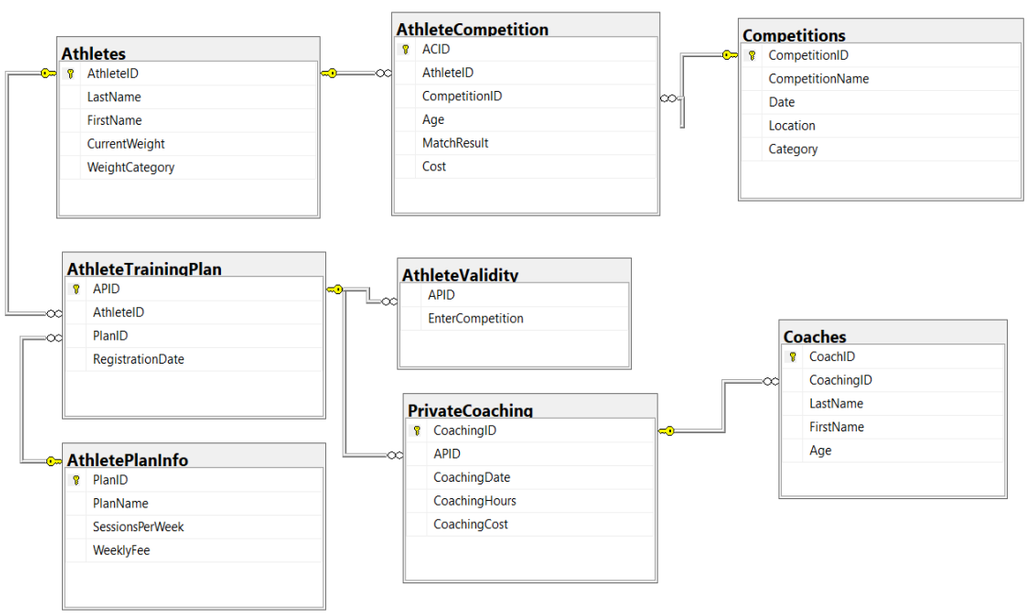
-- Inserting Private Coaching - Private Tuition (per/hr)
INSERT INTO UtilityCosts (Expense, Description, Amount)
VALUES ('Private Coaching', 'Private Tuition (per/hr)', 9.50);

-- Inserting Competition - Competition entry fee
INSERT INTO UtilityCosts (Expense, Description, Amount)
VALUES ('Competition', 'Competition entry fee', 22.00);
```

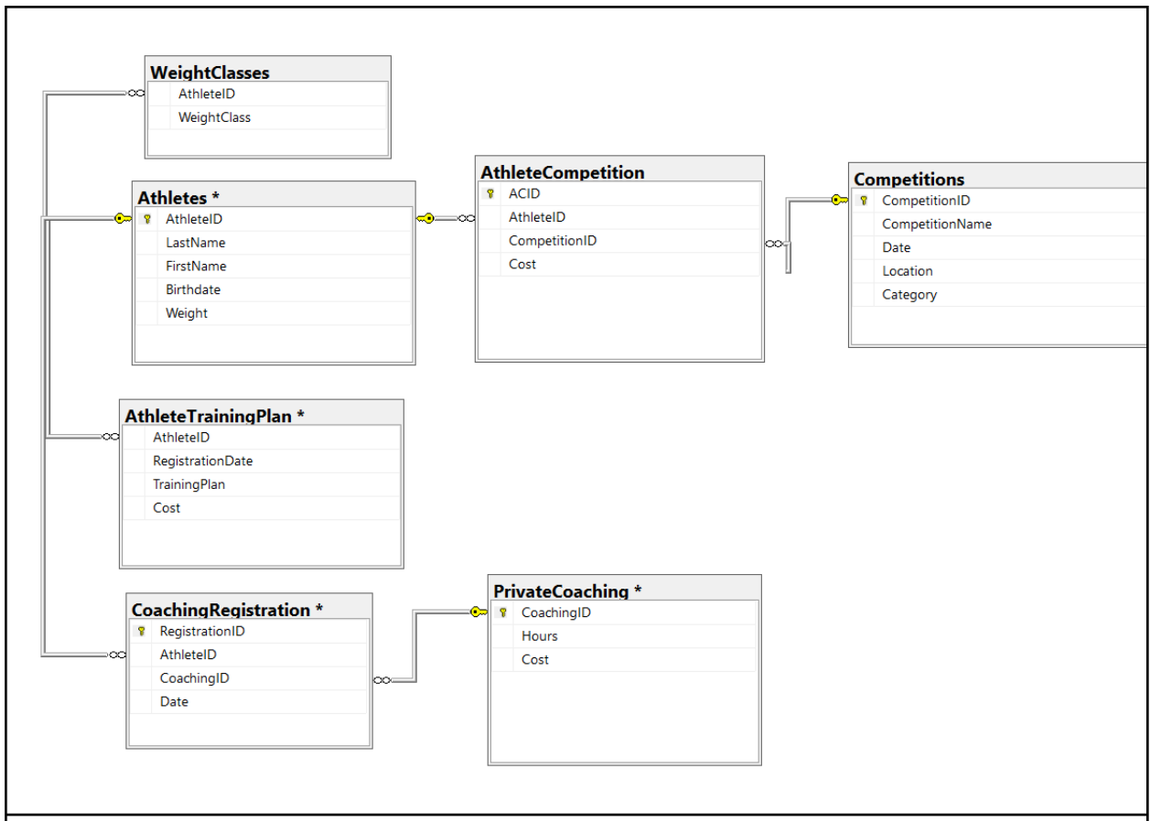
This SQL query statements helped me to test if the database operations worked successfully with the use of DELETE FROM and SELECT FROM statements. Resetting the identity specification means resetting the starting number of the auto-generated keys. The INSERT INTO statements are SQL commands for inserting data in the Utility Costs table.



Book: Python Crash Course (3rd Edition)
Author: Eric Matthes
I read halfway through this book before I started developing the program in order to get used to and understand Python syntax, challenged myself on problems, and properly use functions, classes, if-else statements, and more in my North Sussex Judo application.

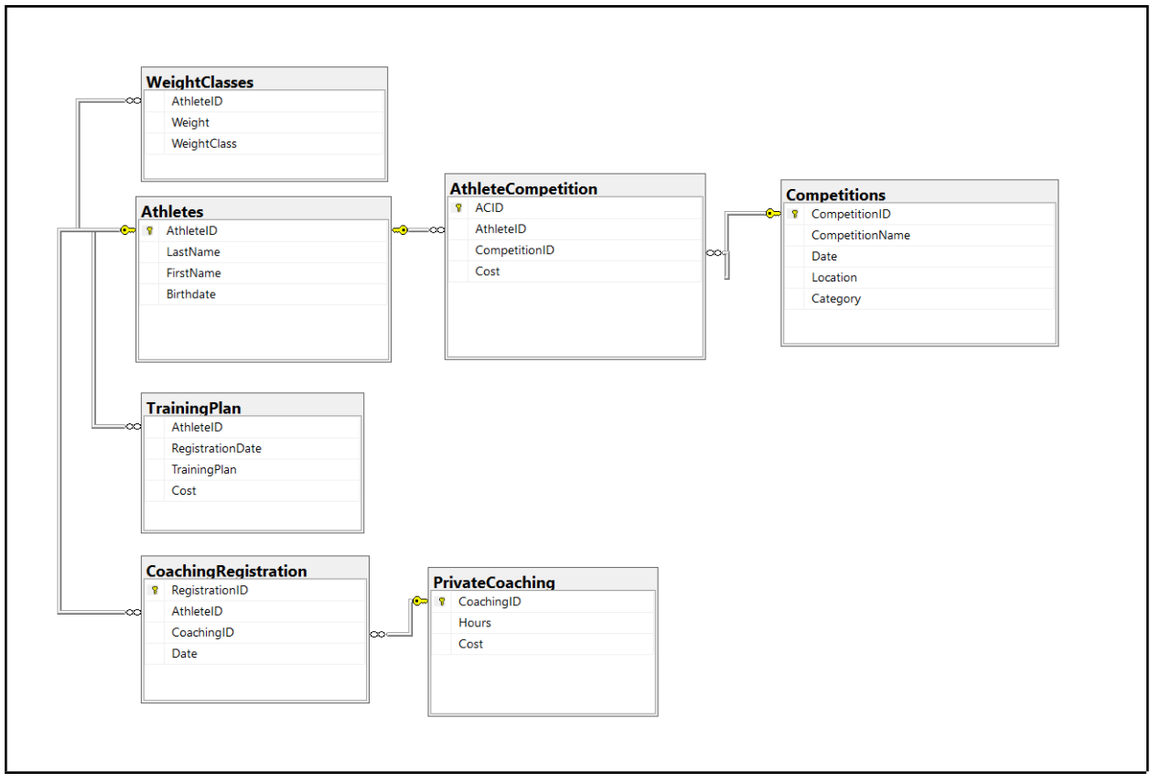


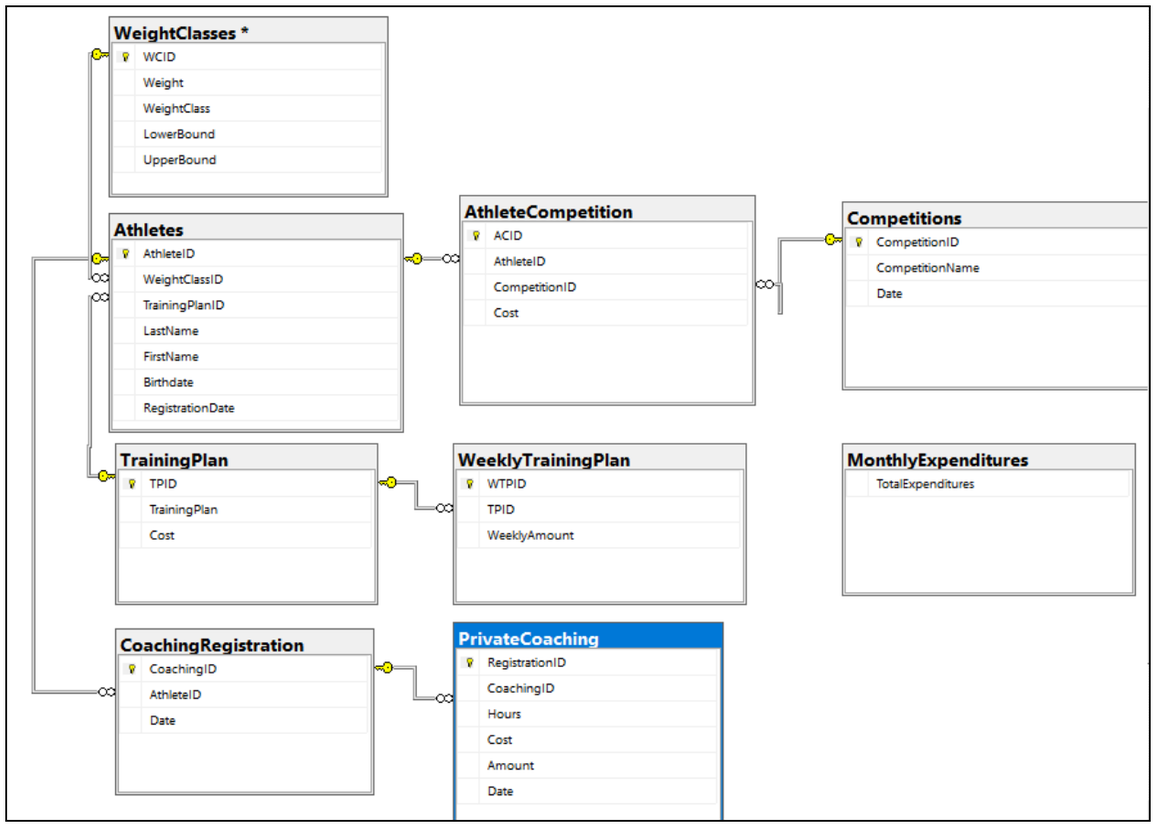
This is my first database design. I applied normalization to a training plan and private coaching. In the training plan, it should be a one-to-one relationship, therefore, the athlete ID wasn't necessary. I removed the Athlete Validity table and Coaches table since it was also unnecessary.

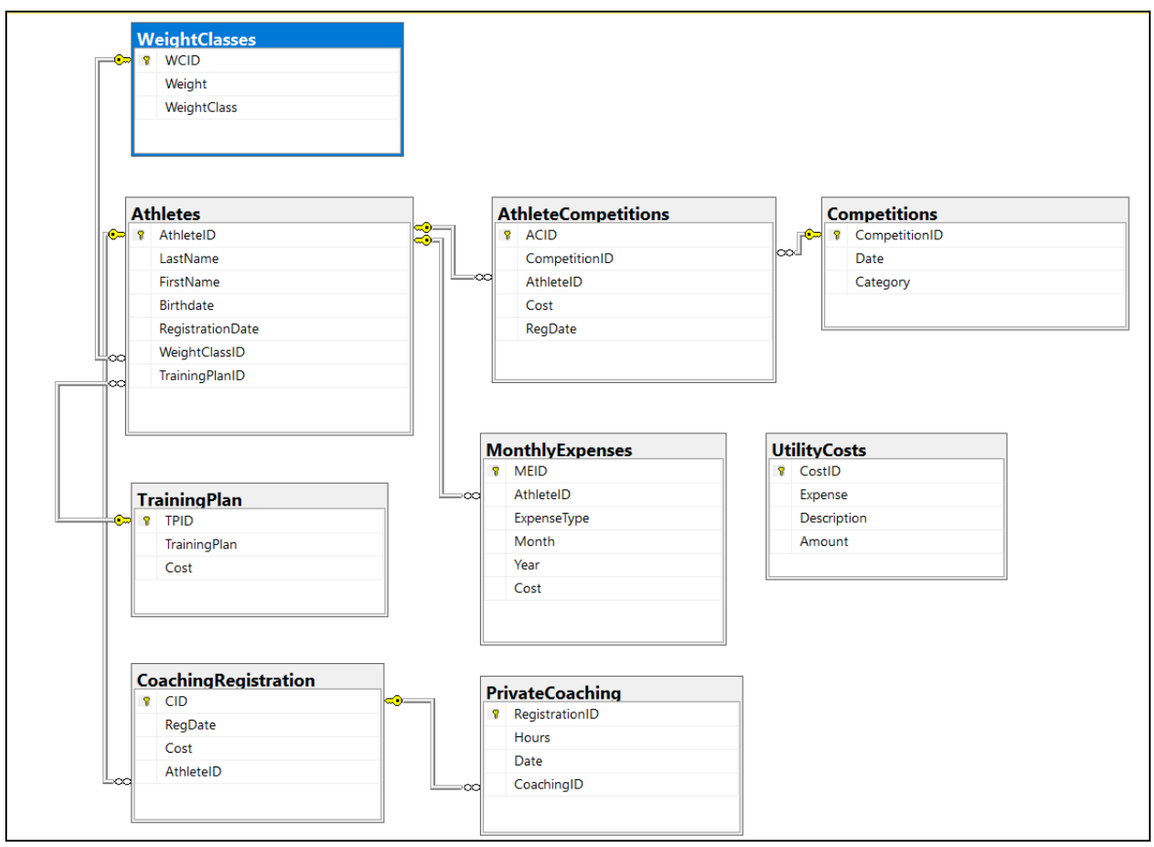


Again, the AthleteID on the TrainingPlan table and WeightCasses table was unnecessary. There should be a primary key instead to establish a relationship. Also, the RegistrationID in the Coaching Registration must be in the Private Coaching table. Location and Category in the Competitions were unnecessary. The weight must be in the weight classes table. The Registration ID in the Training Plan must be located in the Athletes table.

More documentation on my previous database designs:




WeightClasses *
WCID
Weight
WeightClass
LowerBound
UpperBound
Athletes
AthleteID
WeightClassID
TrainingPlanID
LastName
FirstName
Birthdate
RegistrationDate
AthleteCompetition
ACID
AthleteID
CompetitionID
Cost
Competitions
CompetitionID
CompetitionName
Date
TrainingPlan
TPID
TrainingPlan
Cost
WeeklyTrainingPlan
WTPID
TPID
WeeklyAmount
MonthlyExpenditures
TotalExpenditures
CoachingRegistration
CoachingID
AthleteID
Date
PrivateCoaching
RegistrationID
CoachingID
Hours
Cost
Amount
Date


WeightClasses
WCID
Weight
WeightClass
Athletes
AthleteID
LastName
FirstName
Birthdate
RegistrationDate
WeightClassID
TrainingPlanID
AthleteCompetitions
ACID
CompetitionID
AthleteID
Cost
RegDate
Competitions
CompetitionID
Date
Category
MonthlyExpenses
MEID
AthleteID
ExpenseType
Month
Year
Cost
UtilityCosts
CostID
Expense
Description
Amount
TrainingPlan
TPID
TrainingPlan
Cost
CoachingRegistration
CID
RegDate
Cost
AthleteID
PrivateCoaching
RegistrationID
Hours
Date
CoachingID

References

studysmarter (2023). *Event Driven Programming: Examples, Advantages & Features*. [online] StudySmarter UK. Available at: https://www.studysmarter.co.uk/explanations/computer-science/computer-programming/event-driven-programming/.

GeeksforGeeks. (2021). *Overview of Event-Driven Architecture (EDA)*. [online] Available at: https://www.geeksforgeeks.org/overview-of-event-driven-architecture-eda/.

Doherty, E. (2020). *What is Object Oriented Programming? OOP Explained in Depth*. [online] Educative: Interactive Courses for Software Developers. Available at: https://www.educative.io/blog/object-oriented-programming.

GeeksforGeeks. (2016). *Python OOPs Concepts*. [online] Available at: https://www.geeksforgeeks.org/python-oops-concepts/.

StudySmarter UK. (n.d.). *Procedural Programming: Functions, Procedures, Examples*. [online] Available at:

https://www.studysmarter.co.uk/explanations/computer-science/computer-programming/procedural-programming/.

JetBrains (2019). *PyCharm*. [online] JetBrains. Available at:

https://www.jetbrains.com/pycharm/

PyCharm Help. (n.d.). *Breakpoints - Help | PyCharm*. [online] Available at: https://www.jetbrains.com/help/pycharm/using-breakpoints.html.

IntelliJ IDEA Help. (n.d.). *Start the debugger session | IntelliJ IDEA*. [online] Available at: https://www.jetbrains.com/help/idea/starting-the-debugger-session.html [Accessed 5 Feb. 2024].

IntelliJ IDEA Help. (n.d.). *Examine suspended program | IntelliJ IDEA*. [online] Available at: https://www.jetbrains.com/help/idea/examining-suspended-program.html [Accessed 5 Feb. 2024].

Jetbrains (n.d.). *Debug Tool Window. Watches - Help | PyCharm*. [online] Available at: https://www.jetbrains.com/help/pycharm/debug-tool-window-watches.html.

IntelliJ IDEA Help. (n.d.). *Debug tool window | IntelliJ IDEA*. [online] Available at: https://www.jetbrains.com/help/idea/debug-tool-window.html [Accessed 5 Feb. 2024].

JetBrains Rider Help. (n.d.). *Evaluate Expressions | JetBrains Rider*. [online] Available at: https://www.jetbrains.com/help/rider/Evaluating_Expressions.html [Accessed 5 Feb. 2024].

Jetbrains (2019). *Stepping through the program - Help | PyCharm*. [online] Available at:

https://www.jetbrains.com/help/pycharm/stepping-through-the-program.html.

PyCharm Help. (n.d.). *Using Debug Console | PyCharm*. [online] Available at: https://www.jetbrains.com/help/pycharm/using-debug-console.html.

Jetbrains (n.d.). *Code completion - Help | PyCharm*. [online] Available at: https://www.jetbrains.com/help/pycharm/auto-completing-code.html.

PyCharm Help. (n.d.). *Using Debug Console | PyCharm*. [online] Available at: https://www.jetbrains.com/help/pycharm/using-debug-console.html.

Programiz (n.d.). *What is an Algorithm?* [online] www.programiz.com. Available at: https://www.programiz.com/dsa/algorithm.

GeeksforGeeks. (2018). *Difference between Algorithm, Pseudocode and Program - GeeksforGeeks*. [online] Available at: https://www.geeksforgeeks.org/difference-between-algorithm-pseudocode-and-program/.

Stack Abuse. (n.d.). *Python Circular Imports*. [online] Available at: https://stackabuse.com/python-circular-imports/.

van Rossum, G., Warsaw, B. and Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code | peps.python.org*. [online] peps.python.org. Available at: https://peps.python.org/pep-0008/.

GeeksforGeeks (2019). *System Testing - GeeksforGeeks*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/system-testing/.

GeeksforGeeks. (2017). *What is Software Testing?* [online] Available at: https://www.geeksforgeeks.org/software-testing-basics/ [Accessed 5 Feb. 2024].

Bose, S. (2021). *Coding Standards and Best Practices To Follow*. [online] BrowserStack. Available at: https://www.browserstack.com/guide/coding-standards-best-practices.

SmartBear (2019). *What is Automated Testing | Software Testing Basics*. [online] Smartbear.com. Available at: https://smartbear.com/learn/automated-testing/what-is-automated-testing/.

Odogwu, C. (2022). *What Is Database Maintenance and Why Is It Important?* [online] MUO. Available at: https://www.makeuseof.com/what-is-database-maintenance [Accessed 5 Feb. 2024].

Lynch, M. (2023). *What Is Database Maintenance and Why Is It Important?* [online] The Tech Edvocate. Available at:

https://www.thetechedvocate.org/what-is-database-maintenance-and-why-is-it-important/.

GitHub (2023). *GitHub*. [online] GitHub. Available at: https://github.com/.

Sagar Bhatia (2022). *What is Procedural Programming? Key Features of Procedural Programming*. [online] Hackr.io. Available at: https://hackr.io/blog/procedural-programming.

Cockroach Labs. (2023). What is a foreign key? (With SQL examples). [online] Available at: https://www.cockroachlabs.com/blog/what-is-a-foreign-key/

Rouse, M. (2019). What Is Normalization? - Definition from Techopedia. [online] Techopedia.com. Available at: https://www.techopedia.com/definition/1221/normalization.

GeeksforGeeks. (2018). *Concept of Comments in Computer Programming*. [online] Available at: https://www.geeksforgeeks.org/concept-of-comments-in-computer-programming/

GeeksforGeeks (2019). *System Testing - GeeksforGeeks*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/system-testing/.

Cockroach Labs. (2023). What is a foreign key? (With SQL examples). [online] Available at: https://www.cockroachlabs.com/blog/what-is-a-foreign-key

Rouse, M. (2019). What Is Normalization? - Definition from Techopedia. [online] Techopedia.com. Available at: https://www.techopedia.com/definition/1221/normalization.

Microsoft (2022). What is: Multifactor Authentication. [online] support.microsoft.com. Available at: https://support.microsoft.com/en-us/topic/what-is-multifactor-authentication-e5e39437-121c-be60-d123-eda06bddf661.

loginradius (n.d.). What is Login Authentication? Definition, Types & Practices. [online] Available at: https://www.loginradius.com/blog/identity/what-is-login-authentication/ [Accessed 17 Feb. 2024].

Smartsheet (2019). The Purpose and Importance of Audit Trails | Smartsheet. [online] Smartsheet. Available at: https://www.smartsheet.com/audit-trails-and-logs.

LearnSQL.com. (2021). 6 Advanced SQL Queries for Analyzing Financial Data. [online] Available at: https://learnsql.com/blog/advanced-sql-queries-for-financial-analysis/.

Nexus Integra EN. (2019). Database scalability: the future (and almost present) challenge for developers. [online] Available at:

https://nexusintegra.io/why-database-scalability-is-the-challenge-for-developers/.

GeeksforGeeks. (2021). What is Data Normalization and Why Is It Important? [online] Available at: https://www.geeksforgeeks.org/what-is-data-normalization-and-why-is-it-important/.

Marijan, B. (2021). 12 Best Cloud Databases and Why Use Them | phoenixNAP KB. [online] Knowledge Base by phoenixNAP. Available at: https://phoenixnap.com/kb/cloud-database.